

房產鏈金術

RWA 房產代幣化交易系統

指導老師:游佳萍 老師

組長:資管四 B 翁敬倫

組員:資管四 B 吳振維

資管四 B 陳政齊

資管四 B 廖偉哲

目 錄

摘要.....	8
1. 緒論.....	8
1.1 背景介紹.....	8
1.2 專題的目的及重要性.....	8
1.3 主要研究問題或目標.....	9
1.4 系統功能簡介.....	9
1.5 系統使用對象.....	11
1.5.1 技術端.....	11
1.5.2 業務端.....	12
1.5.3 投資人.....	12
1.6 系統特色.....	12
1.7 研究問題與對應表.....	14
2. 相關技術應用與重要文獻.....	16
2.1 描述與本專題相關的其他研究或技術.....	16
2.2 此類交易和系統的優缺點比較.....	16
2.3 系統安全性設計與已知限制.....	18
2.3.1 私鑰與金鑰管理.....	18
2.3.2 敏感欄位加密.....	19
2.3.3 KYC 文件存取控制.....	19
2.3.4 JWT 權杖管理.....	20
2.3.5 登入保護與端點速率限制.....	20
2.3.6 API 層級授權.....	20
2.3.7 職務分離.....	21
2.3.8 外部資料治理.....	21
3. 系統概要設計.....	23
3.1 本專題所採用的研究或開發方法及技術.....	23

3.2 工具和技術的選擇理由.....	23
3.2.1 前端技術.....	24
3.2.2 後端技術.....	24
3.2.3 區塊鏈技術.....	24
3.2.4 爬蟲技術.....	25
3.2.5 資料庫技術.....	25
3.2.6 技術方案比較.....	26
3.2.7 AMM 定價與撮合機制設計.....	26
3.3 處理流程.....	27
3.3.1 投資人購買代幣流程.....	27
3.3.2 KYC 審核流程.....	29
3.3.3 鏈上鏈下對帳排程.....	30
3.3.4 循序圖 Lifeline 與程式碼實體對照.....	31
3.4 模組關係與介面.....	33
3.5 系統架構圖.....	40
3.6 ER 模型圖.....	40
3.7 使用案例圖.....	46
3.8 甘特圖.....	49
3.9 活動圖.....	51
4. 系統開發工具與使用環境.....	54
4.1 系統技術架構與安全設計.....	54
4.2 系統的運行環境要求.....	55
5. 系統實作及實驗結果.....	56
5.1 系統功能的詳細描述.....	56
5.1.1 技術端介面.....	56
5.1.2 業務端介面.....	56
5.1.3 投資人介面.....	57

5.1.4 角色權限設計 (RBAC)	57
5.2 實驗數據.....	62
5.2.1 單元測試與故障注入測試.....	63
5.2.2 系統整合測試 (System Integration Test)	64
5.2.3 使用者驗收測試 (User Acceptance Test)	64
5.2.4 效能與負載測試.....	65
5.3 實作成果的評估.....	66
5.4 與其他相關技術應用的比較，包括優缺點.....	68
5.5 遭遇的問題和挑戰.....	69
5.5.1 遭遇的問題.....	69
5.5.2 面臨的挑戰.....	69
5.5.3 SWOT 分析	70
6. 結論及未來發展.....	72
6.1 總結本專題的主要貢獻.....	72
6.1.1 本節小結.....	72
6.2 對未來研究或發展的建議.....	73
6.2.1 AI 智慧資產推薦配置 (延伸自房產代幣市場與投資組合管理)	73
6.2.2 多源異質數據動態鑑價 (延伸自房產估值爬蟲與 AMM 動態定價)	73
6.2.3 房產投資願景清單 (延伸自房產代幣市場)	73
6.2.4 實價登錄資料整合 (延伸自房產估值爬蟲)	73
6.3 對於計劃的進一步研究或改進，以及預期的挑戰和機會.....	73
6.3.1 短期 (3 至 6 個月)	73
6.3.2 中期 (6 個月至 1 年)	74
6.3.3 長期 (1 年以上)	74
7. 參考文獻.....	76
7.1 中文文獻.....	76
7.2 英文文獻.....	76

附錄.....	78
附錄 A 自動化測試執行紀錄.....	78
附錄 B 效能測試原始輸出	83
附錄 C 後端 API 負載測試原始輸出	85
附錄 D 資料庫 Schema 設計	88
附錄 D-1 物件關聯對映 (Object-Relational Mapping, ORM) 設計	95
附錄 E 功能與循序圖對應表	95

圖 目 錄

圖 1：系統功能全景使用案例圖.....	11
圖 2：循序圖－投資人買入代幣流程.....	29
圖 3：循序圖－鏈上鏈下對帳排程.....	31
圖 4：循序圖－鏈上交易成功但資料庫寫入失敗之對帳處理.....	33
圖 5：模組關係圖（一）前端與後端 API.....	34
圖 6：模組關係圖（二）後端 API 與資料庫	35
圖 7：模組關係圖（三）後端 API 與區塊鏈	36
圖 8：模組關係圖（四）爬蟲服務與資料庫.....	37
圖 9：循序圖－業務端 KYC 審核流程	38
圖 10：循序圖－租金分潤批次處理.....	39
圖 11：系統架構圖.....	40
圖 12：資料庫－身分與 KYC 模組 (Identity & KYC)	42
圖 13：資料庫－房產與估值模組 (Property & Valuation)	43
圖 14：資料庫－訂單與交易模組 (Orders & Transactions)	44
圖 15：資料庫－租金分潤模組 (Rent & Payouts)	45
圖 16：資料庫－區塊鏈與系統對帳模組 (Blockchain & Reconciliation)	46
圖 17：使用案例圖－投資人.....	47
圖 18：使用案例圖－業務端.....	48

圖 19：使用案例圖－技術端.....	49
圖 20：專題時程甘特圖.....	50
圖 21：活動圖－投資人購買代幣之流程.....	52
圖 22：KYC 審核流程圖	53

表 目 錄

表 1：AspenCoin、富邦一號與本系統之三方比較.....	14
表 2：研究問題與設計、驗證方式對應表.....	15
表 3：本系統、富邦一號與喜來登度假村資產證券化之優缺點比較.....	18
表 4：各技術類別採用方案與落選方案比較.....	26
表 5：循序圖 Lifeline 與真實程式碼實體對照表	32
表 6：系統技術組成與版本.....	54
表 7：三角色功能權限矩陣.....	58
表 8：API 端點授權對照表	62
表 9：後端 API 負載測試結果（保留限流情境）	66
表 10：後端 API 負載測試結果（解除限流情境）	66
表 11：期初規劃與實際完成情形對照表.....	67
表 12：本專題 SWOT 分析	71
表 13：資料表欄位說明－roles.....	88
表 14：資料表欄位說明－users	89
表 15：資料表欄位說明－properties.....	90
表 16：資料表欄位說明－user_holdings	90
表 17：資料表欄位說明－transactions	91
表 18：資料表欄位說明－user_notifications	91
表 19：資料表欄位說明－system_alerts.....	92
表 20：資料表欄位說明－crawler_metrics.....	92
表 21：資料表欄位說明－property_ownership	92

表 22：資料表欄位說明－valuation_logs.....	93
表 23：資料表欄位說明－bank_trust_accounts.....	93
表 24：資料表欄位說明－bank_trust_transactions.....	93
表 25：資料表欄位說明－rent_payout_batches	94
表 26：資料表欄位說明－rent_payout_details	94
表 27：資料表欄位說明－chat_logs	94
表 28：資料表欄位說明－blockchain_config.....	94
表 29：功能與循序圖對應表.....	95

摘要

近年來，真實世界資產（Real World Asset, RWA）代幣化成為金融科技領域解決傳統房地產投資門檻過高、流動性匱乏及產權移轉程序繁瑣的重要契機。本專題以技術可行性驗證（Proof of Concept, PoC）為定位，設計並實作一套 RWA 房產代幣化交易系統原型，為小資族群開創低門檻的不動產權益投資管道，並在合規框架下建立兼具安全性與資料一致性的交易環境。

1. 緒論

1.1 背景介紹

近年來全球總體經濟面臨持續性的通膨壓力與貨幣購買力波動，不動產因具備實質租金收益與抗跌特性，長期以來成為重要資產配置標的。然而，傳統房地產市場具有資產總價門檻過高、交易流動性匱乏以及產權所有權移轉行政程序冗長等結構性痛點。在金融科技持續創新的浪潮下，分散式帳本技術為解決實體流動性困境帶來契機，進而推動真實世界資產（Real World Asset, RWA）代幣化概念的蓬勃發展。透過將高總價之實體物業拆分為數位化微型權益份額，資產發行方得以有效活化既有資本，二級市場亦能打破過往不動產流動性不足的限制。

隨著資產代幣化由早期小規模實驗走向機構級金融應用，代幣轉讓的鏈上合規性與交易安全性成為監管單位與系統架構關注的核心焦點。早期代幣化專案多採用一般的無權限代幣標準（例如 ERC-20），此類標準無法在智能合約底層施加身分查驗與轉帳限制，迫使系統必須仰賴鏈下中心化資料庫進行白名單比對。一旦後端系統發生中斷或遭穿透，非合規錢包依然能於公開區塊鏈上任意轉讓代幣，進而產生洗錢防制（AML）與客戶身分查核（KYC）的合規缺口。為了克服此一安全挑戰，國際技術社群發展出以 ERC-3643 為代表的受監管代幣標準，透過合約底層內嵌的身分註冊表與規則引擎，確保每一筆鏈上交易皆即時通過身分驗證，為鏈上金融交易奠定嚴密的合規基礎。

回顧臺灣現行之不動產投資環境，一般小額投資人（小資族）若欲參與優質房地產市場，往往面臨極高的資本門檻與狹窄的投資管道。傳統買賣動輒需要數百萬乃至數千萬元的自備款，使得缺乏龐大資本的小資族難以涉足。至於臺灣現存的不動產投資信託（REITs），其市場標的選擇有限，且交易機制受限於傳統證券交易時段與繁雜手續，次級市場流動性亦顯不足。透過本專題所聚焦的資產份額碎化技術，搭配自動化做市商（AMM）演算法與自動化租金分潤設計，小資投資人得以數千元至數萬元的極低門檻參與優質房產標的，並在合規且安全的次級市場中隨時進行份額撮合與收益結算，使過去受限於高門檻的不動產收益能真正普及於一般大眾。

1.2 專題的目的及重要性

本專題的第一項核心目的在於實作 RWA 不動產代幣化的完整技術概念，建構一套自鏈下實體建物資料擷取至鏈上份額發行的端到端技術原型。面對實體資產難以直接納入智能合約體系的問題，本系統透過程式化方式模擬不動產份額的微型拆分，並設計混合式自動做市商動態定價機制，提供次級市場即時報價依據。這項目的的重要性在於以具體程式碼驗證了實體物業碎化為數位代幣的可行路徑，讓原本笨重且缺乏流動性的大型房產轉化為能在數位環境中流通的資產模型，為未來金融科技系統的架構演進奠定實作基礎。

專題的第二項核心目的在於實踐代幣上鏈過程的全方位安全性設計與資料一致性防護。系統並非停留於概念展示，而是深度導入 ERC-3643 權限型合約架構，透過鏈上身份註冊表與動態風控規則引擎，確保每一筆代幣流轉皆通過嚴格的白名單與持倉上限檢驗，同時在系統底層實作鏈上不可逆交易與鏈下資料庫狀態的最終一致性對帳修復流程。這項目的的重要性在於跳脫傳統金融科技僅仰賴鏈下伺服器控管的單點脆弱性，在智能合約層築起主動合規屏障，為真實世界資產進入區塊鏈世界提供具備抗異常、防竄改特性的工程級安全防護框架。

專題的第三項核心目的在於落實小資族普惠金融願景，開創低門檻且權益透明的數位化投資管道。系統設計以小額份額認購不動產代幣的友善機制，讓散戶僅需數千元即可持有特定建物之收益權益，並以正規化資料庫結構實作自動化租金收益批次計算，使投資人能依持有比例精確獲得分潤。這項目的的重要性在於消弭傳統房地產投資的巨額資本壁壘，不僅大幅降低小額投資人的進場負擔，更透過智能合約與系統自動化確保收益分配透明可查，真正落實普惠金融的社會經濟價值。

實際發行與交易是否符合臺灣證券、信託、不動產、洗錢防制等相關法規，仍須另行進行法遵審查，不在本專題技術原型的驗證範圍內。至於法幣交割與信託、產權法定登記、外部機構 KYC 查核等未串接項目之具體限制，已移至 5.5 節「遭遇的問題和挑戰」說明，實價登錄資料整合等擴充構想則移至 6.2 節「對未來研究或發展的建議」一併說明。

1.3 主要研究問題或目標

本專題聚焦於以下三項核心研究問題：

（一）如何設計一套結合外部公開租屋行情與二級市場交易動能的混合式自動做市商（AMM）動態定價模型，以維持小額代幣在資金池深度有限時的定價穩定性，並降低單一鏈下數據來源被操縱的風險。

（二）如何在 ERC-3643 智能合約內嵌合規身分攔截的前提下，克服鏈上交易不可逆與非同步回傳的技術限制，設計出兼具資料庫行級鎖、冪等防重放與背景自動對帳修復的一致性容錯架構。

（三）如何設計符合第三正規化原則的關聯式資料庫結構，精確解決單一房產面臨眾多小資持有人時的一對多持倉動態變化，並無誤完成租金收益的批次結算與明細展開。

1.4 系統功能簡介

本系統之功能設計並非憑空羅列，而是逐一對應 1.3 節所提出之三項核心研究問題。房產代幣市場、鏈上轉帳與投資組合管理三項功能，回應研究問題（一）混合式 AMM 動態定價與次級市場流動性之設計需求。用戶註冊與 KYC 審核、動態風控與跨部門協作三項功能，回應研究問題

（二）鏈上合規攔截與一致性容錯之設計需求。房產估值爬蟲與租金收益分潤兩項功能，分別提供 AMM 定價所需之外部資料來源，並回應研究問題（三）多人持倉下租金收益批次結算與明細展開之設計需求。系統監控功能則貫穿三項研究問題，提供技術端掌握前述機制實際運作狀態之窗口。

本系統共規劃九項功能模組，各模組之內容分述如下。

（一）用戶註冊與 KYC 審核：受理投資人線上註冊，由業務端完成線下人工審核，審核通過後系統自動於鏈上發行 Identity 合約並簽發 KYC Claim，使該帳號取得交易資格。

(二) 房產代幣市場：提供建案資訊瀏覽與委託下單。投資人可依 AMM 即時報價立即成交，亦可設定限價條件單，由系統於資料庫層持續追蹤條件，達成時自動觸發成交。

(三) 鏈上轉帳：投資人買入時透過 ERC-3643 受合規檢查約束之 transfer 執行，由合約自動驗證交易雙方之合規身分。投資人賣出時因錢包內無 ETH 可支付鏈上 Gas 費，改由後端管理員錢包代為呼叫 forcedTransfer() 並代墊 Gas 費，將代幣收回平台池。此為一般投資人賣出交易之必經路徑，並非例外情境，相關授權說明詳見 5.1.4 節。

(四) 投資組合管理：供投資人查看持倉餘額、歷史交易紀錄與資產總值。

(五) 系統監控：技術端可即時查看資料庫查詢延遲、CPU 負載、系統告警與爬蟲狀態。

(六) 動態風控：具備持倉上限管控與持倉額度收緊等保護機制，並可由技術端或業務端發起全域交易暫停。

(七) 跨部門協作：技術端與業務端可透過平台內建聊天室即時溝通，並留存 chat_logs 紀錄。

(八) 房產估值爬蟲：自動擷取 591 新建案平台之公開行情，更新建案價格與基本資訊，作為 AMM 報價之外部校準來源。

(九) 租金收益分潤：依代幣持有比例，將房產租金收益精確計算並以批次方式分配至各投資人明細。

上述九項功能與三種使用者角色之互動全貌如圖 1 所示。

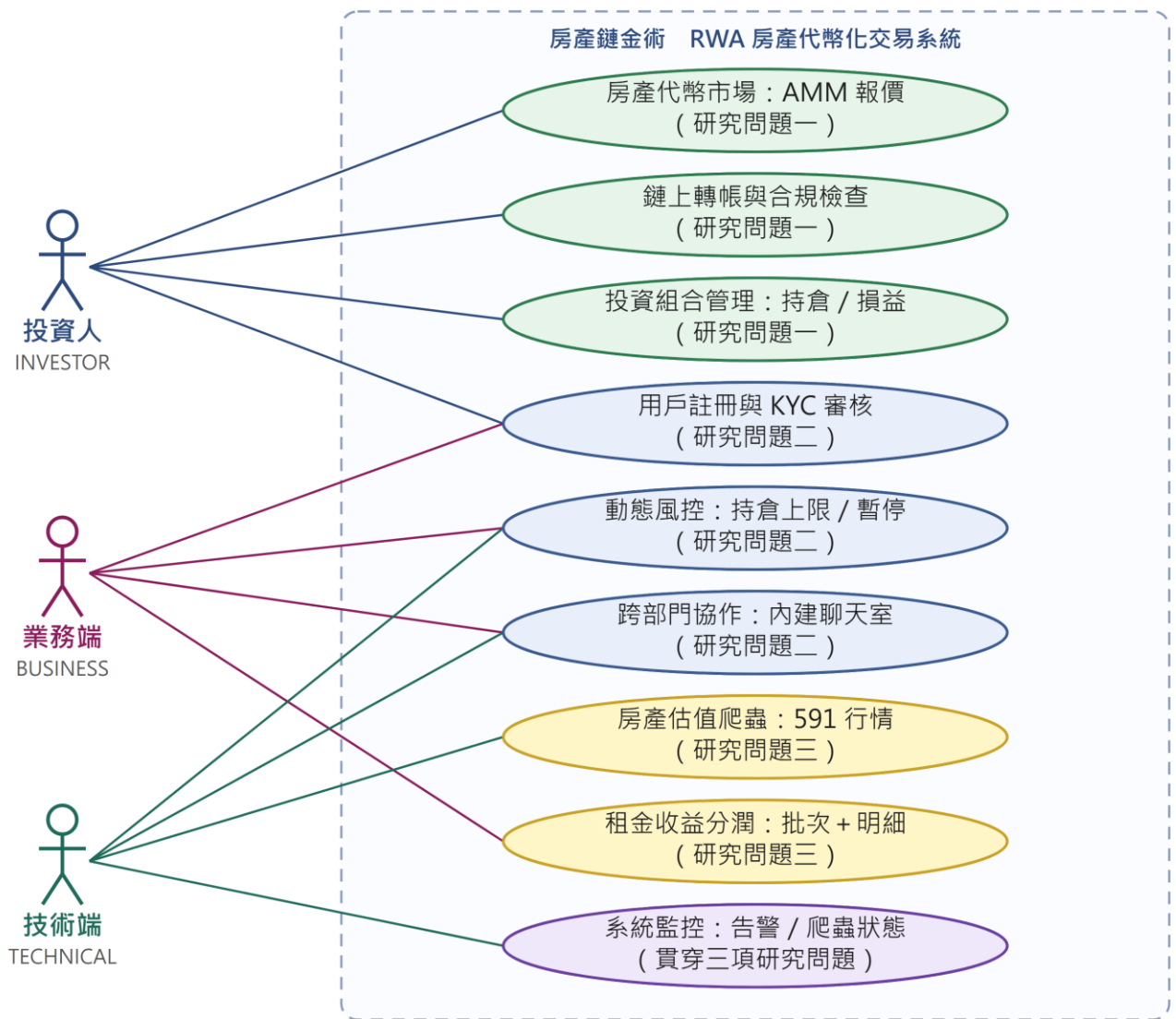


圖 1：系統功能全景使用案例圖

1.5 系統使用對象

1.5.1 技術端

技術端為負責系統維運的技術人員，可監控系統健康狀態、查看告警日誌、控制爬蟲監控儀表板，並在必要時暫停或恢復全平台交易。

情境 1（系統監控與異常處置）：技術端登入後台查看系統監控儀表板，對應「系統監控」功能。當 crawler_metrics 顯示失敗筆數超過門檻時，人工介入排查 591 爬蟲是否失效，對應「房產估值爬蟲」功能。必要時觸發全域暫停以避免異常價格影響交易，對應「動態風控」功能。

情境 2（跨部門協作處理異常）：當業務端發現 KYC 審核異常或白名單設定疑慮時，透過平台內建聊天室與技術端即時溝通，對應「跨部門協作」功能。技術端視需要呼叫區塊鏈補登 API 修復鏈上鏈下狀態不一致。

1.5.2 業務端

業務端為負責合規管理的業務人員，可審核投資人的 KYC 申請、管理用戶白名單狀態，以及於系統內查看各建案的信託帳戶餘額與待發放租金紀錄。現階段信託帳戶為系統內部模擬與測試資料，尚未與銀行系統實際串接，帳戶餘額由業務端於後台手動輸入或由系統模擬產生。

情境 3（KYC 審核與白名單管理）：業務端登入後台查看待審核用戶列表，對應「用戶註冊與 KYC 審核」功能。審查身分文件後執行核准或退件，核准後系統自動於鏈上發行 Identity 合約與 KYC Claim，使該用戶取得交易資格。

情境 4（租金分潤撥款管理）：業務端於信託帳戶查看待發放租金紀錄，對應「租金收益分潤」功能。於分潤批次執行時確認資金到位狀態，並可透過聊天室與技術端協調異常撥款情形，對應「跨部門協作」功能。

1.5.3 投資人

投資人為已完成 KYC 審核的一般使用者，可瀏覽房產代幣市場、執行買賣委託、查看個人投資組合與交易紀錄，並接收系統通知。

情境 5（瀏覽建案並執行代幣交易）：投資人瀏覽房產代幣市場，對應「房產代幣市場」功能。選定建案後以市價或限價方式下單，系統透過 ERC-3643 合約執行鏈上轉帳，對應「鏈上轉帳」功能，完成交易。

情境 6（查看投資組合與租金分潤）：投資人於投資組合頁面查看持倉與損益，對應「投資組合管理」功能。於租金分潤批次撥款後收到站內通知，確認依持有比例取得之分潤金額，對應「租金收益分潤」功能。

1.6 系統特色

對應第一項目的「RWA 不動產代幣化之概念實作」：本系統以資產份額碎化技術，將實體房產拆分為代幣化之微型權益份額，並設計混合式自動做市商（AMM）動態定價機制，結合鏈上代幣儲備量與鏈下 591 新建案平台爬蟲擷取之市場行情進行報價校準，降低單一資料來源被操縱之風險。租金分潤功能則採「批次+明細」二層式演算法，依代幣持有比例精確計算並自動化分配租金收益，將 RWA 代幣化由概念落實為可操作之技術原型。

對應第二項目的「代幣上鏈之安全性設計」：本系統採用 ERC-3643 權限型代幣標準，於智能合約層直接嵌入身分驗證與轉帳限制邏輯，取代傳統僅依賴鏈下資料庫比對的白名單機制。系統另設計對帳排程，以鏈上代幣合約之 Transfer 事件日誌為權威來源，定期與資料庫 UserHolding 紀錄比對，發現差異時以鏈上餘額覆寫資料庫，確保鏈上鏈下狀態最終一致。用戶錢包私鑰採 AES-256-CBC 加密儲存，並具備由技術端/業務端人工發起之全域交易暫停機制以因應極端異常狀況。惟尚未實作基於價格劇烈波動或交易量異常之自動化市場熔斷機制（Circuit Breaker），此為原型階段之已知限制。

對應第三項目的「提高小資族投資管道」：本系統以每枚代幣數千元至數萬元之低單價設計，搭配 AMM 演算法提供全年無休之次級市場流動性，取代傳統不動產交易冗長之產權過戶程序，使小額投資人得以低門檻參與優質房產標的，並可隨時調整持倉。

為具體呈現本系統相較既有 RWA 合規實例與傳統 REITs 之定位，以下就資產權利連結、身分／白名單、交易機制、鏈上／鏈下分工、租金分潤、資料來源、銀行交割、金鑰管理、稽核與完成成熟度十個構面進行比較：

就前述十個構面觀察，三者的差異可歸納為三點。第一，在權利連結與法遵基礎上，AspenCoin 具備 Reg D 私募與美國證管會監理，富邦一號受臺灣證券與信託法規規範，本系統之代幣則未與不動產產權正式連結，僅屬技術可行性驗證。第二，在交易與流動性機制上，AspenCoin 與富邦一號皆採集中式撮合並受交易時段限制，本系統則以資料庫訂單簿優先撮合、AMM 流動性池承接未成交餘量，可提供全年無休之報價。第三，在資金與資產保管上，前兩者皆有專屬股務機構或受託銀行負責金流與保管，本系統之信託帳戶餘額與撥款則為系統內部模擬數值，尚未串接真實銀行 API。十項構面之逐項對照如表 1 所示。

比較構面	AspenCoin (美國 RWA 合規實例)	富邦一號 (台灣傳統 REITs)	本系統 (RWA 技術展示原型)
資產權利連結	代幣代表 Aspen Digital Inc. 約 19% 股權，間接持有 St. Regis Aspen 度假村資產	受益證券直接連結信託不動產所有權	代幣未與不動產產權正式連結，屬技術可行性驗證，不具實體物權效力
身分／白名單	依 Reg D 506(c) 私募規範，限美國合格投資人，鏈上 KYC／AML 審查	集保帳戶實名制，依台灣證券法規監管	ERC-3643 IdentityRegistry 白名單，業務端線下人工審核後登記鏈上。屬技術層面合規檢查，非法定合規保證
交易機制	於 tZERO ATS 集中撮合交易	台灣證交所集中競價撮合	兩階段撮合：先於資料庫訂單簿與真實對手盤撮合（價格優先、時間優先，採 Maker 掛價），未成交餘量再由 AMM 固定乘積模型之平台流動性池承接。該流動性池為系統模擬，非真實資金
鏈上／鏈下分工	Tezos 鏈上證券化，搭配傳統股務系統管理	全鏈下運作，未使用區塊鏈技術	ERC-3643 合約管理身分與代幣轉帳，PostgreSQL 管理業務資料，並以鏈上餘額為權威來源、資料庫為鏡像快取
租金分潤	度假村營運利潤依代幣持有比例進行鏈上／法幣分派	依信託契約每年配發現金股利，90% 以上淨收益需分配	批次分潤演算法依持有比例計算應分配金額後，由管理員錢包於鏈上轉出等值原生代幣模擬撥款並標記為 PAID。未串接真實銀行撥款 API，鏈上金額為固定比例換算之展示值
資料來源	度假村實際營運財務報表	信託受託銀行公告之財務資訊	591 新建案平台爬蟲擷取
銀行交割	Computershare 等專屬股務與資產保管機	土地銀行擔任信託受託與金流保管機	無真實銀行系統對接，信託帳戶餘額為系統內部模擬數值

	構	構	
金鑰管理	投資人自託管錢包私鑰，或由合規平台代管	無私鑰概念，由券商集保體系保管	ADMIN_KEY 以環境變數存放於雲端後台，未導入 KMS/HSM 專用金鑰管理服務，屬原型階段已知限制
稽核	第三方合規平台 (Securitize) 進行稽核	會計師年度查核，並受主管機關監理	透過 system_alerts/valuation_logs/crawler_metrics 保留歷史軌跡，支援技術面事後回溯。非正式第三方外部稽核
完成成熟度與限制	已完成真實募資並穩定營運多年	台灣第一檔 REIT，已掛牌運作超過 20 年	教學與技術展示原型，尚未進入商業營運。法幣交割、產權登記、外部 KYC 查核均未串接，為明確揭露之限制

表 1：AspenCoin、富邦一號與本系統之三方比較

註: AspenCoin 的核心價值在於結合證券法遵循 (Reg D)、合格投資人審查與專屬股務保管等傳統金融基礎設施。富邦一號則展現台灣 REITs 制度下證交所集中撮合與受託銀行保管的高度成熟與法律保障。本系統的技術貢獻聚焦於 ERC-3643 權限控管、AMM 動態定價與鏈上鏈下對帳機制之可行性驗證，但在真實法幣交割、產權登記與第三方稽核等面向，如上表所示仍與 AspenCoin、富邦一號存在明確差距，此為技術原型階段之如實揭露，非隱瞞或迴避。

1.7 研究問題與對應表

為避免研究問題僅止於文字宣稱，本節將 1.3 節之三項核心研究問題，逐一對應至本專題實際採用之設計與驗證方式。研究問題 (一) 之混合式 AMM 動態定價模型，對應 2.1 節所述之定價理論基礎與 3.2.7 節之交易機制設計，並於 5.2 節以自動化測試比對公式結果與實際成交價。研究問題 (二) 之鏈上合規與一致性容錯架構，對應 3.2.5 節之對帳排程設計與 3.3 節之處理流程，並於 5.2 節以 89 項自動化測試 (含 28 項故障注入測試) 驗證異常情境行為。研究問題 (三) 之租金分潤資料庫正規化設計，對應 3.6 節之 ER 模型與附錄 D 之資料表結構，並於 5.2 節驗證批次與明細之展開正確性。三者皆已提出對應之測試案例與程式碼證據，而非僅憑文字宣稱達成。

三項研究問題、對應設計與驗證方式之完整對照如表 2 所示。

研究問題	對應需求	設計元件與實作模組	測試案例與實測證據	結論與已知限制
(一) 混合式 AMM 動態定價	結合 591 外部行情與鏈上交易動能，降低單一資料來源被操縱之風險，並維持資金池深度有限時之定價穩定	AMM 固定乘積定價公式 ($k = X \times Y$)、591 爬蟲擷取流程。實作於 transactions.service.ts (定價邏輯) 與 crawler/main.py (資料來源)	買入量佔比越高、成交均價偏離現貨價越多之滑價方向驗證，並以測試自行計算之公式結果比對服務實際寫入之成交價。證據：transactions.service.spec.ts (5.2.1 節)	現行公式已通過單元測試，惟尚未涵蓋大規模併發下的價格穩定性，屬 5.5 節所列之已知限制

研究問題	對應需求	設計元件與實作模組	測試案例與實測證據	結論與已知限制
(二) 鏈上合規與鏈上鏈下一致性容錯	鏈上交易不可逆之前提下，資料庫狀態須能與鏈上狀態最終一致，且合規檢查不得依賴鏈下服務持續運作	ERC-3643 合約層身分攔截、資料庫列級悲觀鎖、冪等鏈防重放、全量對帳與 system_alerts 告警。實作於 blockchain.service.ts、transactions.service.ts 與 contracts/ 目錄	DB commit 失敗回滾、鏈上逾時標記 CHAIN_FAILED、RPC 不可達時 reconcile 明確拒絕等故障注入情境。證據：transactions/fault-injection.spec.ts、blockchain/fault-injection.spec.ts、fault-injection-integrity.spec.ts (合計 89 項測試全數通過，詳見附錄 A)	已驗證單筆交易之異常處理正確。惟「鏈上已成功、DB 尚未 commit」之空窗重啟仍可能重複轉帳，現況僅能倚賴事後對帳偵測，為已揭露之架構層風險
(三) 一對多持倉與租金分潤之正規化資料庫設計	單一房產面對多位持有人時，須精確處理動態持倉變化並正確展開租金收益明細	符合第三正規化之 16 張資料表、user_holdings 三欄組合唯一約束，以及 rent_payout_batches 與 rent_payout_details 二層式分潤結構。實作於 properties.service.ts 與對應 Entity 類別	持倉唯一約束與重複扣款防護已由資料庫層 UNIQUE 約束與併發測試驗證。分潤批次與明細之展開則於技術端介面完成人工驗證 (5.2.3 節)	分潤金額之計算與明細展開邏輯已完成。尚未撰寫分潤計算之獨立自動化單元測試，且實際銀行撥款未串接，兩者均為已知限制

表 2：研究問題與設計、驗證方式對應表

2. 相關技術應用與重要文獻

2.1 描述與本專題相關的其他研究或技術

本專案應用之 RWA 財務概念主要有三。其一為資產份額碎化（Fractionalization），將高總價之單一房產拆分為可分割之微型代幣份額，以降低投資單位金額（Wang et al., 2025）。其二為次級市場流動性機制，透過 AMM 演算法提供無需仰賴傳統仲介之全年無休撮合管道（Adams, Zinsmeister, & Robinson, 2020）。其三為租金收益分潤（Rental Income Distribution），以代幣持有比例作為收益分配依據，將傳統不動產包租代管模式數位化為批次與明細二層式之自動化分潤演算法。

RWA（Real World Asset）代幣化近年成為區塊鏈應用的重要方向，其核心精神是將股權、債券、不動產等傳統資產的所有權以數位代幣形式呈現，使其能在鏈上被追蹤、移轉與交易（Wang et al., 2025）。國際上具代表性的實踐包括 RealT、Lofty 等美國不動產代幣化平台，其共同特徵是以有限合夥（LLC）架構持有實體物業，再將股權比例映射為 ERC-20 代幣供投資人認購。然而多數早期平台採用一般無權限代幣標準，合規驗證仍仰賴鏈下資料庫進行白名單比對，一旦後端服務中斷，代幣仍可能在鏈上被任意移轉，形成合規缺口。

為解決此問題，Tokeny Solutions 於 2019 年提出 ERC-3643（T-REX，Token for Regulated EXchanges）標準，將 IdentityRegistry、ClaimTopicsRegistry、TrustedIssuersRegistry 等合規元件內嵌於智能合約本身，使每一筆轉帳都必須通過鏈上驗證才能執行，不再依賴鏈下系統的可靠性（Ethereum Improvement Proposals, 2021。ERC-3643 Association, n.d.）。本專題採用此一標準作為代幣合規基礎，屬於此研究脈絡的延伸應用。

在次級市場定價機制方面，去中心化交易所（DEX）廣泛採用的自動化做市商（AMM）模型，例如 Uniswap 的固定乘積公式（Constant Product Market Maker），為本專題 AMM 定價邏輯的理論基礎（Adams et al., 2020）。本專題進一步結合 591 新建案平台之外部資料作為價格校準的預言機（Oracle）來源，屬於將鏈下真實世界資料導入鏈上定價機制的混合式設計，此類「鏈下資料驅動鏈上定價」的架構亦是目前 RWA 領域常見的研究方向之一。

本專案應用之外部資料整合與財務控管安全標準包含三項。其一為爬蟲資料治理，以 Python Playwright 定期擷取 591 新建案平台之公開市場行情，並以資料完整度（integrity）指標與失敗筆數監控來源品質。其二為動態風控機制，透過持倉上限管控（正常 5%、節流期 1%）與人工發起之全域交易暫停機制，作為交易量或狀態異常時的風險控管手段。其三為角色權限分工，依投資人、業務端與技術端三種角色劃分功能存取權限與 API 層級授權，並落實 KYC 審核與白名單管理之職務區隔。

2.2 此類交易和系統的優缺點比較

本節以富邦一號與美國喜來登度假村資產證券化兩個既有案例作為對照組，先分述兩者之優缺點，再說明本專案承接了哪些優點、又試圖解決哪些缺點。

首先是富邦一號。其優點在於制度成熟且法律保障明確，受益證券直接連結信託不動產所有權，由受託銀行保管金流、專業機構負責物業管理，投資人以每股面額 10 元即可參與，且依法規每年須將 90% 以上淨收益以現金股利分配。其缺點則在於流動性與即時性不足：次級市場交易不活躍，散戶買賣容易產生折價，受託銀行、物業管理與估價師等費用層層堆疊而稀釋投資報酬率，交易時間受限於台股盤中時段，物業財務揭露頻率低，投資人難以即時掌握標的價值變化。

其次是美國喜來登度假村資產證券化（AspenCoin）。其優點在於證實了區塊鏈與傳統證券法規接軌的可行性：該案以 1 美元面額募集 1,800 萬美元，符合美國證管會 Reg D 私募規範，並由專屬股務與資產保管機構負責投資人名冊與資產保管，使頂級資產得以平民化持有。其缺點在於早期僅開放合格投資人參與，未完全開放一般小額大眾，且發行初期缺乏 AMM 等成熟的去中心化交易機制，次級市場交易冷清，代幣持有人不易在需要時變現。

本專案承接了兩案例的三項優點：延續富邦一號的低面額設計，使小額投資人得以參與。延續喜來登案例以區塊鏈作為權益登記與移轉媒介的思路。並延續兩者以正式資料結構保留收益分配軌跡的做法，以批次與明細二層式演算法自動計算分潤金額。針對兩案例的缺點，本專案嘗試解決其中兩項：其一為次級市場流動性不足，做法是結合資料庫訂單簿 P2P 撮合與 AMM 流動性池承接，提供不受交易時段限制的全年無休報價。其二為定價缺乏即時性，做法是以 591 新建案平台爬蟲資料校準代幣參考價格，使定價能隨行情動態調整，此為兩個對照組皆未具備的機制。

本專案尚未能解決的部分亦須如實說明。喜來登案例已完成真實 Reg D 合規發行並受美國證管會監理，本系統則僅於合約層驗證 ERC-3643 白名單攔截機制，尚未具備對應之法遵結構。富邦一號由受託銀行實際撥發現金股利，本系統之分潤則僅能於資料庫標記為 PAID 並於鏈上以原生代幣模擬撥款，尚未串接真實銀行系統。三者於各構面之逐項差異如表 3 所示。

比較構面	本系統	富邦一號 (REITs)	美國喜來登度假村資產證券化
主要優點	降低投資門檻（數萬元即可投資）。採用經多次審計之 ERC-3643 標準合約庫	降低投資門檻。專業物業管理，招租維護收租皆由專業機構負責	頂級資產平民化。符合美證管會 Reg D 監管條例，證實區塊鏈與傳統證券法規接軌可行性
主要限制	詳見「5.5 節、遭遇的問題和挑戰」	台灣 REITs 次級市場交易不活絡，散戶買賣易有折價。受託銀行/物業管理/估價師等費用層層堆疊稀釋投報率。限台股交易時間，物業財務揭露頻率低缺乏即時性	早期僅開放合格投資人，未完全開放一般小額大眾。發行初期缺乏 AMM 等成熟去中心化交易機制，早期次級市場交易冷清
發行媒介／技術	本地開發節點 (Hardhat)	台股掛牌交易	以太坊區塊鏈
標的資產	台灣房市 (591 平台預售/新成屋)	富邦人壽大樓、富邦中山大樓等精華商辦	美國科羅拉多州喜來登五星級度假村 18.9% 股權
投資門檻與面額	每枚 token 數千元至數萬元不等	每股面額 10 元台幣，散戶數萬元即可買進	每個代幣面額 1 美元，募集總額 1,800 萬美元
收益分配	依持有比例分配收益	依法律規範，90% 以上淨收益以現金股利每年分配	依股權及代幣持有比例享有相對應權益
資料來源	本專題原始碼與測試資料	公開資訊觀測站、信託公會公開資料	Securitize 白皮書、STOWise 公開資訊

表 3：本系統、富邦一號與喜來登度假村資產證券化之優缺點比較

註：富邦一號代表台灣 REITs 制度下高度成熟但流動性受限的傳統模式，本系統與美國喜來登案例則共同指向 AMM 等去中心化交易機制對次級市場流動性的潛在助益。然而喜來登案例已完成真實 Reg D 合規發行與美證管會監理，本系統目前僅於合約層驗證 ERC-3643 白名單攔截機制，尚未具備對應之法遵結構，此差異已於 1.6 節之三方比較表明確揭露。

2.3 系統安全性設計與已知限制

本節依「安全問題、系統做法、已知限制」三段式格式，逐項說明本系統在交易安全與權限最小化兩大面向的設計。每一小節皆先界定該項設計所要解決的具體安全問題，再說明實作方式，最後如實揭露現階段尚未解決的部分。

2.3.1 私鑰與金鑰管理

要解決的安全問題：本系統的鏈上操作（代幣移轉、Identity 部署、KYC Claim 簽發）皆由平台管理員錢包簽署。若簽署用私鑰遭未授權者取得，攻擊者即可任意移轉平台代管之全部代幣，屬單點失守即全盤失守的風險。此外，KYC 證件影像若以明文保存，一旦資料庫外洩即造成個資大量曝光。

系統做法：管理員私鑰（ADMIN_KEY）與圖片加密金鑰（IMAGE_ENCRYPTION_KEY）以環境變數形式存放於 Render 後台，不隨原始碼進入版本控制，避免金鑰隨 Git 提交外流。系統並於啟動時檢查金鑰是否存在，正式環境若缺少 ADMIN_KEY 即拒絕啟動區塊鏈相關功能並記錄告警，避免以測試用私鑰誤啟動正式環境，此行為已由 blockchain/fault-injection.spec.ts 驗證（測試輸出詳見附錄 A）。

已知限制：金鑰仍以環境變數明文存放，未導入專用金鑰管理服務（如 AWS KMS 或 HSM 硬體安全模組），亦未搭配多重簽章（Multi-sig）機制，因此單一金鑰外洩仍會造成全面風險。此為原型階段之技術取舍，改善規劃詳見 6.3.1 節。

2.3.2 敏感欄位加密

要解決的安全問題：users 資料表中同時保存錢包私鑰、KYC 文件路徑與銀行帳號三類敏感欄位（對應圖 12 之 ERD）。若這些欄位以明文儲存，則資料庫備份外洩、雲端服務誤設為公開讀取，或內部人員越權查詢，都會直接造成資產遭盜用與個資外洩。

系統做法：wallet_private_key（錢包私鑰）採 AES-256-CBC 對稱加密儲存，每筆加密資料附帶隨機 IV，確保相同明文不會產生相同密文，避免攻擊者由密文比對推知哪些帳號共用同一組私鑰。系統並實作向後相容機制，可正確處理歷史資料中殘留的明文私鑰格式，使加密改造不需停機或重建資料。

已知限制：其一，kyc_document_path（KYC 文件路徑）與 bank_account_number（銀行帳號）目前仍為明文欄位，正式產品化應一併加密或改為遮罩顯示。其二，私鑰加密金鑰與 KYC 圖片加密共用同一把 IMAGE_ENCRYPTION_KEY，未依用途分離金鑰（key separation），使單一金鑰外洩的影響範圍擴大。其三，金鑰本身仍以環境變數明文存放於雲端平台，未使用 KMS 或 HSM。三者均屬原型階段之已知限制。

2.3.3 KYC 文件存取控制

要解決的安全問題：KYC 證件影像為高敏感個資，即使已加密儲存，只要解密端點的授權不夠嚴謹，具備一般後台帳號者仍可能大量下載他人證件。此類端點也容易成為金鑰暴力嘗試的攻擊目標。

系統做法：KYC 文件的解密操作（POST /api/kyc/:id/decrypt）限制僅 BUSINESS 角色可呼叫，且需額外提供 adminKey 參數才能執行，形成角色檢查與金鑰驗證的雙重驗證。該端點另設每分鐘 5 次之速率限制，以防金鑰暴力嘗試。KYC 審核流程（詳見圖 22）由業務端於鏈下完成人工審查後，才將使用者的合格狀態登記至鏈上 IdentityRegistry。

已知限制：此處比對之 adminKey 實為資料庫連線密碼（依序取用 SUPABASE_DB_PASSWORD、DATABASE_URL 中夾帶之密碼、DB_PASSWORD），並非另行簽發之獨立管理金鑰，等同再次擴大同一組祕密的使用範圍，與 2.3.2 節所述「未依用途分離金鑰」屬同一類限制。另本流程僅提供技術層面之合規檢查機制，不涉及對 KYC 審核結果本身正確性的保證。

2.3.4 JWT 權杖管理

要解決的安全問題：本系統為三角色分權設計，若身分權杖可被偽造或竄改，攻擊者即可自行宣稱為 TECHNICAL 角色而取得系統維運權限。另一方面，若權杖無法反映使用者權限的即時異動，被停權的帳戶在權杖失效前仍可繼續操作。

系統做法：身份驗證採用 JWT (JSON Web Token) 搭配 @nestjs/passport 的 AuthGuard('jwt') 實作，後端於每次請求時驗證簽章，確認權杖未遭竄改且尚未過期，通過後才解碼取出角色資訊。由於 JWT 屬簽章型令牌而非加密令牌，其內容可被任何人解讀，因此系統不在 payload 中放置任何敏感資訊，僅存放使用者識別碼與角色。權杖並設有過期時間 (TTL)，縮短權杖遭竊後的可用期間。

已知限制：JWT 於過期前無法主動撤銷，若使用者角色異動或帳戶遭停權，現階段須待權杖過期後才會生效。系統尚未導入權杖撤銷清單 (revocation list) 或 Token version 機制，此為已知限制。

2.3.5 登入保護與端點速率限制

要解決的安全問題：登入端點若無次數限制，攻擊者可以字典檔對已知帳號反覆嘗試密碼。註冊、下單與金鑰解密等端點若無限制，也可能被自動化腳本大量呼叫，造成資料污染或服務資源耗盡。

系統做法：/api/login 端點透過 @Throttle 設定速率限制 (5 分鐘內最多 10 次嘗試)，密碼並以雜湊值 (password_hash 欄位) 儲存，不保留明文密碼，因此即使資料庫外洩也無法直接取得原始密碼。除登入端點外，系統另於 /api/register 設每分鐘 10 次、/api/transactions 設每分鐘 50 次、/api/kyc/:id/decrypt 設每分鐘 5 次之端點層級限制，全站並套用 ThrottlerModule 之每 60 秒 100 次預設上限。

已知限制：速率限制以記憶體計數實作，於多執行個體水平擴充時各執行個體之計數彼此獨立，實際可嘗試次數會隨執行個體數量倍增，正式產品化應改以共用快取 (如 Redis) 集中計數。

2.3.6 API 層級授權

要解決的安全問題：三角色分權若僅在前端隱藏按鈕而未於後端強制檢查，任何人皆可直接以工具呼叫 API 取得不屬於其角色的資料或功能。此外，即使角色正確，使用者仍可能藉由竄改路徑參數存取他人資料，形成物件層級授權漏洞。

系統做法：授權分為兩層。第一層為角色層級，於 Controller 以 JwtAuthGuard 搭配角色檢查，限制每一支端點的可存取角色，完整對照詳見 5.1.4 節之表 8。第二層為物件層級，對涉及個人資料的端點於 Service 層以 id 與 user_id 並列作為資料庫查詢條件，非本人的資料查詢即回傳 null 並擋下請求。以取消掛單端點 POST /api/pending-orders/:id/cancel 為例，其 Controller 層雖未見明顯檢查，但 Service 層之並列查詢條件已確保物件層級授權，此行為並由 5.2.2 節之整合測試驗證。

設定過程中曾盤點出三項授權疏漏並已修正：GET /api/system/performance 與 GET

/api/system/crawler-status 兩支 API 原先缺少身份驗證，已補上 JwtAuthGuard 並限制僅 TECHNICAL 與 BUSINESS 角色可存取。POST /api/system/crawler-report 原先僅檢查登入狀態未限制角色，已限縮為僅 TECHNICAL 角色可呼叫。

2.3.7 職務分離

要解決的安全問題：KYC 核准、白名單調整、暫停或恢復交易、鏈上合約 setup 等操作，皆足以影響全體投資人的資產與交易資格。若單一人員即可獨力完成，則一名內部人員遭社交工程或帳號被盜，就足以造成系統性損害。

系統做法：本系統將 KYC 審核與系統維運分屬 BUSINESS 與 TECHNICAL 兩種角色，使身分核准與鏈上操作不由同一角色掌握，並於前端協作流程中設計「業務端提出風控暫停請求、技術端核准後執行」的順序約束。

已知限制：上述順序約束係由前端流程實現，後端 POST /api/system/state 仍對 BUSINESS 與 TECHNICAL 授予相同權限，未於伺服器端強制（鏈上合約之 POST /api/blockchain/pause-toggle 則確實限 TECHNICAL）。系統亦尚未實作「提出→核准→執行」分離或雙人核准機制，高風險操作目前皆僅需單一角色即可執行，屬原型階段已知限制，改善規劃詳見 6.3.1 節。

2.3.8 外部資料治理

要解決的安全問題：本系統之代幣參考價格部分取決於 591 新建案平台之公開行情。若該資料來源失效、頁面結構異動或回傳異常數值，錯誤價格將直接影響投資人的交易決策，形成「資料來源即攻擊面」的風險。爬蟲行為本身也須確保不逾越公開資訊之取用範圍。

資料來源與擷取方式：目標為 591.com.tw 新建案頁面，以 Python Playwright 進行動態渲染擷取，屬公開網頁資訊，僅作教學與展示用途。抓取前設定逾時秒數，連線失敗時累加 crawler_metrics.consecutive_failures 並寫入系統告警，供技術端於監控頁面掌握資料來源品質。

資料清洗與品質控制：爬蟲於擷取每個欄位後，依建案名稱、價格、坪數、地址、縣市、圖片六項是否成功取得計分，換算為 0 至 100 之資料完整度（integrity）指標，並將該次執行之平均完整度寫入 crawler_metrics.average_integrity。已知限制：目前僅計算此一指標，尚未實作「未通過驗證即不寫入」之攔截閘門，亦未對價格、坪數等數值欄位做正數檢核或日期格式解析檢核，欄位缺漏之資料仍會寫入 properties 資料表而僅反映為較低的完整度分數。

授權與合法依據：本系統爬取之 591 平台資料皆為不須登入即可檢視之公開資訊。爬蟲行為嚴格限制請求頻率，且所取得之資料僅供本學術專題之內部測試與 AMM 報價模擬使用，無任何商業營利行為。

來源失效處理：爬蟲每次執行後將該次失敗筆數寫入 crawler_metrics.consecutive_failures、狀態寫入 status（全數成功為 HEALTHY，否則為 WARNING），並同步寫入一筆 CRAWLER_REPORT 系統告警供技術端於監控頁面追蹤。已知限制：consecutive_failures 記錄的是「單次執行的失敗筆數」且每次執行以覆寫方式更新，並非跨次累計之連續失敗次數。系統亦尚未實作失敗達門檻即自

動凍結爬蟲排程，或改以最後一次成功之快照（Snapshot）價格暫代之機制，來源失效時須由技術端依告警人工介入修復，改善規劃詳見 6.3.1 節。

3. 系統概要設計

3.1 本專題所採用的研究或開發方法及技術

多人協作專案若無固定迭代與追蹤機制，容易發生範疇蔓延、進度不透明與待辦事項遺漏，此風險若未控管，將直接影響 1.2 節所述三項核心目的能否於期限內落實為可驗證之技術原型。為控管此風險，本專題採用敏捷開發（Agile）模式，以兩週為一個 Sprint 週期，於每個 Sprint 結束前召開團隊會議，回顧上一階段完成項目、檢視待解決問題並規劃下一階段任務優先順序，歷次會議紀錄統一以 GitHub Issues 留存，形成可回溯的議題與待辦清單。之所以選擇敏捷模式而非瀑布式開發，是因為本專題的關鍵參數（例如 AMM 權重、風控門檻與合約介面）必須依實測結果反覆調整，固定式規格難以在期初一次定案，兩週一迭代的節奏也讓四名成員的跨模組相依問題能在下一次會議前即被發現。GitHub Issues 之歷史紀錄可作為進度追蹤與 3.8 節甘特圖時程規劃之依據，供事後核對各階段實際完成情形。

導入 AI 輔助編碼工具若無治理規範，可能產生未經審查的程式碼、測試覆蓋不足，或敏感資料經提示詞外洩等風險，並連帶影響 5.2 節自動化測試涵蓋率之可信度。為此，本專題採用 AI-assisted development 治理方式，明訂審查、測試、授權三道關卡：保留工程師主導架構設計與程式碼審查之責任，AI 工具僅用於加速樣板程式碼撰寫、資料庫 schema 迭代與文件整理等工作，合規邏輯設計、風控機制與系統整合等需人為判斷的核心工作不假手 AI。採用此方式的理由在於，本專題為四人小組且開發期程有限，完全不使用 AI 將難以在期限內完成五大模組，但金融性質的合規與風控邏輯一旦出錯即形成系統性風險，因此以「AI 產出、人工審查、測試驗收」三段式分工，同時兼顧開發速度與正確性。核心交易與風控邏輯最終通過 5.2 節所述 89 項自動化測試（含故障注入測試），可作為此一分工下程式正確性之佐證。

多人協作若允許改寫提交歷史，可能導致他人本機分支遺失或衝突難以追溯。開發與正式環境設定不一致，也可能造成「本機正常、正式環境異常」的環境落差問題。因應方式為採用 Git 版本控制並訂定「嚴格禁止 Git Rebase、僅允許 Merge」之協作規範，全專案原始碼與資料庫 schema 統一透過 Git 管理，另以容器化與雲端代管服務統一開發與正式環境。開發環境透過 Docker Compose 容器化 PostgreSQL 資料庫服務，NestJS 後端 API 於本機以 `npm run start:dev` 直接啟動，正式環境則部署於 Render 雲端平台，資料庫統一連向 Supabase 雲端服務。選擇「只 Merge 不 Rebase」而非業界常見的 rebase 工作流，是因為本組成員的 Git 熟練度不一，rebase 一旦誤用即造成不可回復的歷史覆寫，而合併節點雖使分支線圖較為雜亂，卻能保證每一次協作行為都留下可稽核的紀錄。Git 提交歷史本身即為此政策落實與否之可稽核記錄，開發與正式環境是否一致，則可透過 Docker Compose 設定檔與 Render、Supabase 連線設定逐項比對確認。

3.2 工具和技術的選擇理由

本系統在技術選型上以「合規性優先、開發效率次之、可維護性並重」為原則，針對各子系統的功能特性分別評估並選用最適合的工具與框架。以下依子系統分類說明各項技術的選用理由。

3.2.1 前端技術

本系統前端需同時承載 KYC 審核介面、投資人交易頁面與技術管理後台三種角色的功能，且交易頁面需頻繁更新報價與持倉資料。基於此，選用 React 作為前端框架，主因在於其元件化（Component-based）架構能有效分離不同角色的功能模組，降低跨角色程式碼耦合度（Meta Open Source, n.d.）。搭配 Vite 作為建構工具，相較於傳統 webpack，其原生 ESM 熱更新（HMR）機制大幅縮短開發時的重新編譯時間，提升迭代效率（Vite, n.d.）。

樣式層選用 Tailwind CSS，其工具類（Utility-first）設計哲學可直接在 JSX 標記中完成樣式定義，無需切換獨立 CSS 檔案，適合快速搭建原型介面（Tailwind Labs, n.d.）。元件層則採用 shadcn/ui，其提供一組遵循 WAI-ARIA 無障礙標準的可組合元件，在符合規範的前提下仍保有完整的樣式客製彈性，且元件原始碼直接置於專案目錄而非封裝於 node_modules，便於針對金融平台特有的互動邏輯進行修改。

3.2.2 後端技術

後端選用 NestJS 並以 TypeScript 為第一語言，其模組（Module）、控制器（Controller）、服務（Service）三層架構與本系統的功能分區高度契合，BlockchainModule、TransactionsModule、SystemModule 各自封裝，不同子系統間透過依賴注入（Dependency Injection）進行解耦（NestJS, n.d.）。相較於 Express，NestJS 內建的 Pipe、Guard、Interceptor 機制使 JWT 鑑權、角色檢查與請求驗證等橫切關注點（Cross-cutting Concerns）得以在不侵入業務邏輯的前提下統一處理，這對本系統必須在每一支 API 上一致套用三角色權限檢查的需求特別重要。

身份驗證採用 JWT（JSON Web Token）無狀態方案，後端守衛（Guard）於每次請求時對 Token 進行簽章驗證（verify），確認其未遭篡改且尚未過期，驗證通過後才解碼（decode）取出內嵌之 TECHNICAL、BUSINESS 或 INVESTOR 角色資訊（Jones, Bradley, & Sakimura, 2015）。此設計可避免每次請求都查詢資料庫，降低延遲。Token 設有過期時間（TTL），若使用者角色異動或帳戶被停用，因 JWT 於過期前無法主動撤銷，現階段須待 Token 過期後才會生效，尚未導入 Token 撤銷清單（revocation list）或 Token version 機制，此為已知限制，詳見 2.3.4 節。

3.2.3 區塊鏈技術

本系統採用 ERC-3643（T-REX v4）標準而非一般 ERC-20，根本原因在於合規需求：房地產代幣屬於受監管的證券型資產，其轉讓必須僅限於身份已驗證的白名單投資人。ERC-3643 將 IdentityRegistry、TrustedIssuersRegistry、ClaimTopicsRegistry 等合規機制以智能合約形式內嵌於代幣本身，使每筆 transfer 在鏈上即時驗證持有人 KYC 狀態，無需仰賴鏈下中心化控管（ERC-3643 Association, n.d.）。此特性使合規稽核具備不可篡改性與可公開驗證性，符合金融監理的透明度要求，也是本專題選用此標準而非自行以鏈下白名單實作的關鍵理由。

系統以鏈上代幣合約餘額為持倉權威來源，資料庫則作為前端查詢之鏡像快取，兩者之對帳排程與異常重試機制詳見 3.3.3 節之循序圖說明。

開發與測試環境選用 Hardhat，其本地 EVM 節點（localhost:8545）可在無需實際 Gas 費用的環境下完整模擬鏈上部署與交易流程，並提供精細的堆疊追蹤（Stack Trace）輸出，大幅降低 Solidity 合約開發的除錯成本（Nomic Foundation, n.d.）。對本專題而言，此點尤其重要，因為故障注入測試需要能自由重設區塊高度、模擬 RPC 斷線與節點失憶等異常狀態，公開測試網無法提供此種控制能力。ethers.js 作為與合約互動的 JavaScript 函式庫，提供 NonceManager 封裝，有效解決後端在快速連續部署多個合約時可能發生的 Nonce 衝突問題（Ethers, n.d.）。

3.2.4 爬蟲技術

Python 長期以來是網路爬蟲的主流語言，其生態系中的 Playwright 函式庫支援真實瀏覽器環境（Chromium）下的動態頁面操作，能處理 591.com.tw 新建案頁面中需等待 JavaScript 渲染完成才能取得的房源資料，相較於靜態 HTTP 請求工具（如 requests）具備更高的資料擷取成功率（Microsoft, n.d.）。psycopg2 則作為 Python 原生的 PostgreSQL 驅動，在爬蟲抓取完成後直接以 SQL INSERT 寫入資料庫，避免跨語言 API 呼叫的額外網路往返開銷。選擇讓爬蟲直接寫入資料庫而非經由後端 REST API，是因為爬蟲屬批次型的獨立程序，資料量大且不需即時回應，直接寫入可省去序列化與網路傳輸成本。

爬取結果會依建案名稱、價格、坪數、地址、縣市、圖片六項必要欄位是否成功取得計算資料完整度（integrity），該次執行之平均完整度與失敗筆數分別寫入 crawler_metrics.average_integrity 與 consecutive_failures，供技術端於系統監控頁面掌握資料來源品質。此設計之取捨在於繞過後端驗證層以換取效能，因此改以完整度指標作為品質監控手段，相關已知限制詳見 5.5.2 節。

3.2.5 資料庫技術

PostgreSQL 的選用基於兩項關鍵考量。其一，房地產交易涉及資產餘額（UserHolding）、交易記錄（AppTransaction）與用戶資產總值（User.total_asset_value）的多表同步更新，需要嚴格的 ACID 交易保證以防止資料不一致。其二，PostgreSQL 成熟的列級鎖（Row-level Locking）機制能在高並發交易情境下正確處理持倉餘額的讀取、修改、寫入序列（PostgreSQL Global Development Group, 2022）。相較於 MongoDB 等文件型資料庫，金融撮合與分潤嚴格依賴外鍵、唯一約束與列級悲觀鎖，這正是關聯式資料庫的強項，故未採用文件型方案。

TypeORM 作為 ORM 層提供型別安全的查詢介面，並透過 QueryRunner API 確保「寫入交易雜湊（txHash）」與「更新資料庫持倉」兩項資料庫操作本身的原子性，避免系統中出現只更新一半的中間狀態（TypeORM, n.d.）。由於鏈上交易一旦確認即不可逆，鏈上與鏈下狀態無法被包進同一個原子交易中，因此系統以鏈上代幣合約餘額作為持倉的權威來源，資料庫 UserHolding 則作為供前端快速查詢的鏡像快取。

為銜接上述兩者，系統另設計對帳機制。BlockchainService 之 reconcile() 以 queryFilter 一次性拉取代幣合約完整的 Transfer 事件歷史，重建鏈上真實餘額後與資料庫 UserHolding 比對（BALANCE_MISMATCH），並以 txHash 比對交易紀錄找出「鏈上有轉帳、資料庫無對應紀錄」之情形（UNTRACKED_TRANSFER）。因鏈上交易不可逆，修復時一律以鏈上餘額覆寫資料庫，

並將每筆差異寫入 system_alerts 供技術端追蹤。此對帳機制之現行實作限制詳見 5.5.2 節。Docker Compose 則統一管理 PostgreSQL 容器的版本與啟動設定，使開發環境與生產環境保持一致，避免因本機資料庫版本差異導致的相容性問題（Docker, n.d.）。

3.2.6 技術方案比較

本系統於前端框架、全域狀態管理、後端框架、資料庫與區塊鏈環境五個技術類別上，皆曾比較至少一個替代方案，並依合規性、開發效率與可維護性等準則做出選擇。各類別最終採用方案與落選方案之比較準則、權衡理由，以及可對應查證之實作證據，整理如表 4 所示。

技術類別	採用方案	落選方案	選擇準則與權衡理由	實作證據與帶來的限制
前端框架	React + Vite	Next.js (SSR)	準則：客戶端即時互動與開發迭代速度。 本系統為高頻金融儀表板，不依賴 SEO，Vite 提供極速 HMR。	package.json 使用 @vitejs/plugin-react。限制：首頁無 SSR 預渲染。
全域狀態	React Context API	Redux / Zustand	準則：輕量化與低維護負擔。 系統僅需共享使用者身分、全域暫停與心跳計時，無需大型狀態庫。	實作 AuthContext.tsx 與 SystemControlContext.tsx。
後端框架	NestJS (TypeScript)	Express.js / FastAPI	準則：企業級模組化架構與強型別防護。 依賴注入 (DI) 與分層架構利於單元測試與大型專案維護。	完整 Controller-Service-Repository 結構。取代了舊版雛型 server/。
資料庫	PostgreSQL (Supabase)	MongoDB / MySQL	準則：ACID 交易保證與關聯完整性。 金融撮合與分潤嚴格依賴外鍵、唯一約束 (23505) 與列級悲觀鎖。	TypeORM 設定 synchronize: false，使用正式 Migration 控管。
區塊鏈環境	Hardhat 本地節點	Sepolia 測試網	準則：零 Gas 成本、即時打包與故障注入測試。 可自由重設區塊高度並測試 RPC 斷線與異常情況。	限制：單節點重啟後記憶體資料清空，需靠後端 autoRecoverNodeState 重建。

表 4：各技術類別採用方案與落選方案比較

3.2.7 AMM 定價與撮合機制設計

在定義定價公式之前，先說明本節反覆出現的「滑價」（slippage）一詞。滑價是指一筆委託的實際成交均價與下單當下現貨價之間的差距。在自動做市商機制中，成交價由資金池的儲備量比例決定，買入量越大、消耗的儲備越多，剩餘儲備所對應的價格就越高，因此同一筆委託中越後面成交的部分價格越差，平均下來即高於現貨價，此差額即為滑價。滑價並非平台收取的手續費，而是資金池深度有限所產生的自然成本。

本系統採用類似固定乘積（Constant Product）模型的 AMM 機制，以代幣總發行量（total_supply_x）作為代幣儲備量 X、以建案募資初始總額（fundraising_goal）作為對應的價值儲備量 Y，定義常數 $k = X \times Y$ 。當投資人買入 Δ 枚代幣時，代幣儲備量減少為 $X' = X - \Delta$ ，依 k 不變原則反推新的價值儲備量 $Y' = k / X'$ ，實際成交均價 = $(Y' - Y) / \Delta$ 。

選用固定乘積模型而非集中式訂單簿撮合或人工定價，主要考量本專案的三項特性。第一，本平台為原型階段，同一建案的真實對手盤數量極少，若僅依賴訂單簿撮合，投資人多數時候將無法成交，無從展示次級市場流動性。第二，固定乘積公式僅需儲備量兩個狀態變數即可產生連續報價，不需維護造市商團隊或外部報價來源，與本專題四人小組的維運量能相符。第三，該公式為 DEX 領域經長期驗證的成熟模型，其滑價行為可用數學式驗證，便於在 5.2 節以自動化測試比對理論值與實際成交價，符合本專題「以測試證據支撐設計主張」的驗證原則。

在交易對手安排上，系統採兩階段撮合。第一階段先於資料庫訂單簿中尋找價格重疊之反向掛單，依「價格優先、時間優先」原則與另一位真實使用者直接成交，成交價採 Maker 掛單價而不套用 AMM 滑價。第二階段才將未成交之剩餘數量交由上述公式模擬的「平台流動性池」承接，其初始儲備即為該建案上架時的總發行量與募資總額。此安排之理由在於，真實對手盤成交對雙方而言價格較佳且不消耗流動性池儲備，理應優先，AMM 僅作為保證「隨時可成交」的後備機制。已知限制為第一階段之 P2P 撮合僅更新資料庫持倉，並不觸發鏈上代幣轉帳，該部分之鏈上與鏈下差異須倚賴對帳機制事後偵測與修正。

在費用設計上，本原型未實作額外的交易手續費機制，投資人所承擔的價差僅來自 AMM 公式本身造成的自然滑價。此設計使測試時能將成交價偏差完全歸因於公式本身，便於驗證定價邏輯之正確性。

在委託類型上，市價單依當下 AMM 報價立即成交。限價單則由系統每 5 秒檢查一次是否達成觸發條件（背景排程 @Cron(EVERY_5_SECONDS)），且觸發判斷係以「該數量沿 AMM 曲線成交之含滑價均價」為準而非現貨價，以避免觸發後又被滑價檢查拒絕之重複循環，藉此模擬條件式自動觸發之延遲情境（測試紀錄詳見 5.2 節）。

以下以一筆市價買單為例，說明滑價的實際數值。假設某建案 total_supply_x = 100,000 枚、fundraising_goal = NT\$18,919,000，則 $k = 1,891,900,000,000$ ，現貨價 = $Y / X = 189.19$ 元。投資人以市價單買入 100 枚時， $X' = 99,900$ ， $Y' = k / 99,900 \approx \text{NT\$}18,937,938$ ，成交均價 $\approx (18,937,938 - 18,919,000) / 100 \approx \text{NT\$}189.38$ ，較現貨價 189.19 元高約 0.1%。此 0.1% 的差距即為買入量佔資金池 0.1% 比例所產生的自然滑價，與前段對滑價的定義一致。

3.3 處理流程

3.3.1 投資人購買代幣流程

前置條件為使用者已登入並持有有效 JWT、帳戶 KYC 狀態為 VERIFIED，且系統未處於全域暫停狀態。輸入為建案 ID、購買數量、委託類型（市價或限價），限價者另含觸發價格。投資人於

前端選擇建案、填入購買數量，並選擇依 AMM 即時報價直接成交，或設定限價條件由系統於資料庫層追蹤、條件滿足時自動觸發成交。

須說明的是，本流程僅完成代幣於鏈上與資料庫層級的轉移模擬，交易對手為系統預先配置之流動性池與平台保留持倉，並非真實買賣雙方之銀行法幣交割。代幣簽署交易之私鑰由後端伺服器本身持有及簽署，尚未導入正式的金鑰管理機制，此設計於教學原型階段為求開發簡便所採用。私鑰若外洩將導致平台代管資產遭盜用風險，未來應改採硬體安全模組（HSM）、多重簽章錢包（Multi-sig）或第三方金鑰管理服務（KMS）等方式強化保管。

完整處理步驟如下。

步驟 1（輸入）：前端攜帶 JWT 令牌呼叫 POST /api/transactions。

步驟 2（授權驗證）：後端驗證 JWT、確認系統未暫停、檢查交易參數合法性，驗證失敗則回傳 401 或 403 並終止流程。

步驟 3（參數與滑價驗證）：API 層執行請求頻率限制（rate limit），同一使用者短時間內下單次數超過門檻即拒絕該次請求。同時對限價單執行滑價保護，若該筆數量沿 AMM 曲線成交之含滑價均價已超出使用者設定之限價（買單均價高於限價、賣單均價低於限價），則拒絕本次成交並回傳錯誤訊息。已知限制為市價單並未設置系統層級之滑價容忍區間，投資人以市價單下單即視為接受當下 AMM 曲線所產生之全部滑價。

步驟 4（風控檢查）：風控引擎查詢投資人目前持倉，判斷是否超過上限（正常為 5%，節流期為 1%，節流期定義為系統自全域暫停恢復交易後之 2 小時內），超過上限則直接拒絕本次委託並回傳錯誤訊息。

步驟 5（鏈上執行）：BlockchainService 呼叫 ERC-3643 合約 transfer()，合約自動驗證交易雙方 KYC 狀態，未通過白名單者由合約直接攔截拒絕。

步驟 6（鏈上確認）：鏈上交易確認後取得 txHash。若鏈上呼叫逾時或失敗，交易狀態標記為 CHAIN_FAILED 並保留交易意圖，不回滾資料庫，供後續對帳機制修復。

步驟 7（資料庫同步）：資料庫事務寫入交易紀錄、更新持倉餘額與用戶資產總值。

步驟 8（通知與最終狀態）：系統發送成交通知至投資人通知中心。交易最終狀態為 SUCCESS 或 CHAIN_FAILED 二擇一，並持久化 txHash 與時間戳供稽核與對帳追溯。

上述流程之完整生命週期如圖 2 所示，涵蓋投資人從前端送出購買請求，歷經 JWT 驗證、幂等鍵重複攔截、資料庫悲觀鎖序列化、AMM 動態滑價計算，到最後鏈上代幣移轉的各個階段。

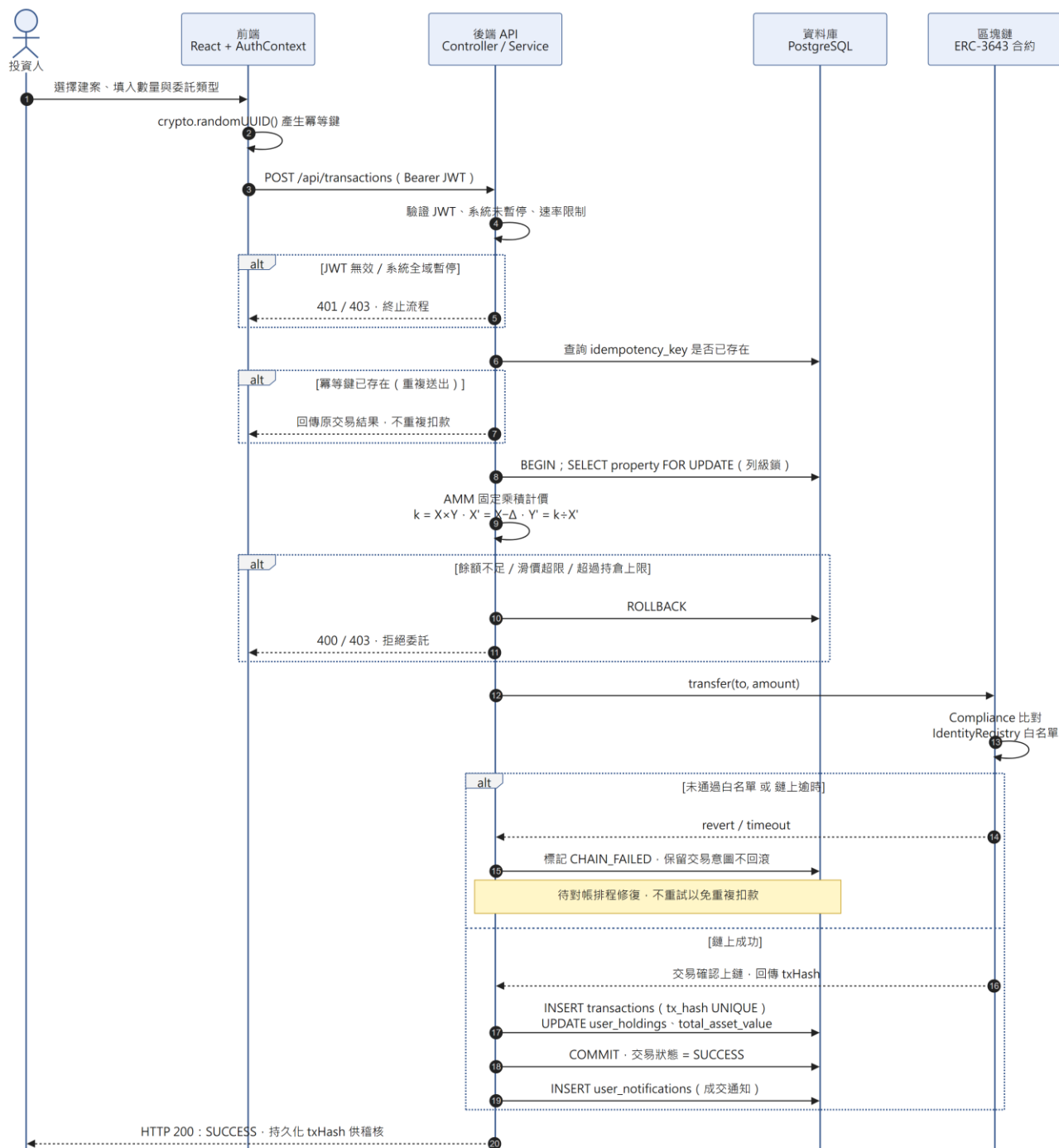


圖 2：循序圖－投資人買入代幣流程

註: 關鍵決策在於前端以 `crypto.randomUUID()` 生成冪等鍵，後端在進入資料庫交易前先查詢該鍵是否已存在，極端併發下則由資料庫 23505 唯一約束擋下重複請求。同時對該建案記錄取 `pessimistic_write` 鎖，確保高併發時 AMM 儲備量計算不會發生 Race Condition。例外處理上，若鏈上轉帳因節點逾時失敗，系統不回滾資料庫，而是將交易狀態設為 `CHAIN_FAILED`，確保投資人的交易意圖被保留，供後續對帳機制修復，避免因鏈上失敗導致整筆請求失敗、使用者需重新操作。

3.3.2 KYC 審核流程

前置條件為使用者已完成註冊並取得帳號，輸入為身分證件影像與基本資料。完整處理步驟如下。

步驟 1（輸入）：用戶完成線下身份文件提交，由投資人自行上傳、業務端進行人工審核，不涉及銀行系統查證。業務端登入後台後查看待審核用戶列表。

步驟 2（狀態轉換）：業務端點擊核准，後端於同一次資料庫更新中將 KYC 狀態設為 VERIFIED 並同時將 is_whitelisted 設為 true，使用者自此即具備交易權限。

步驟 3（鏈上登記）：資料庫更新完成後，BlockchainService 才為用戶部署 Identity 合約，並由管理員簽署 KYC Claim 寫入鏈上，將用戶錢包地址登記至 IdentityRegistry，最終狀態為 VERIFIED。

例外處理：若鏈上部署或 Claim 簽發失敗，系統僅記錄警告並於回應中夾帶錯誤訊息，資料庫狀態維持 VERIFIED 不回捲。

已知限制為此設計使「鏈下審核通過」與「鏈上登記成功」未形成原子操作，鏈上登記失敗時使用者於平台端已可下單，須由技術端呼叫 POST /api/blockchain/register-user/:userId 補登。正式產品化應改為兩階段狀態，待鏈上確認成功後才放行交易權限。另實際 KYC 狀態集為 UNSUBMITTED、PENDING、VERIFIED 與 REJECTED，退件後可由使用者透過 POST /api/kyc/resubmit 補件重審。

3.3.3 鏈上鏈下對帳排程

除上述兩項核心流程外，系統另設計背景對帳排程機制，用以解決鏈上交易不可逆與資料庫狀態同步的問題，並於偵測到資料庫寫入失敗等異常情境時，透過重試與告警機制修復兩邊狀態差異。以下分別以正常排程與異常重試兩種情境說明。

正常排程情境如圖 3 所示。對帳系統以鏈上代幣合約之 Transfer 事件日誌為權威來源，重建真實餘額後與資料庫 UserHolding 紀錄比對，確保雙邊狀態最終一致。

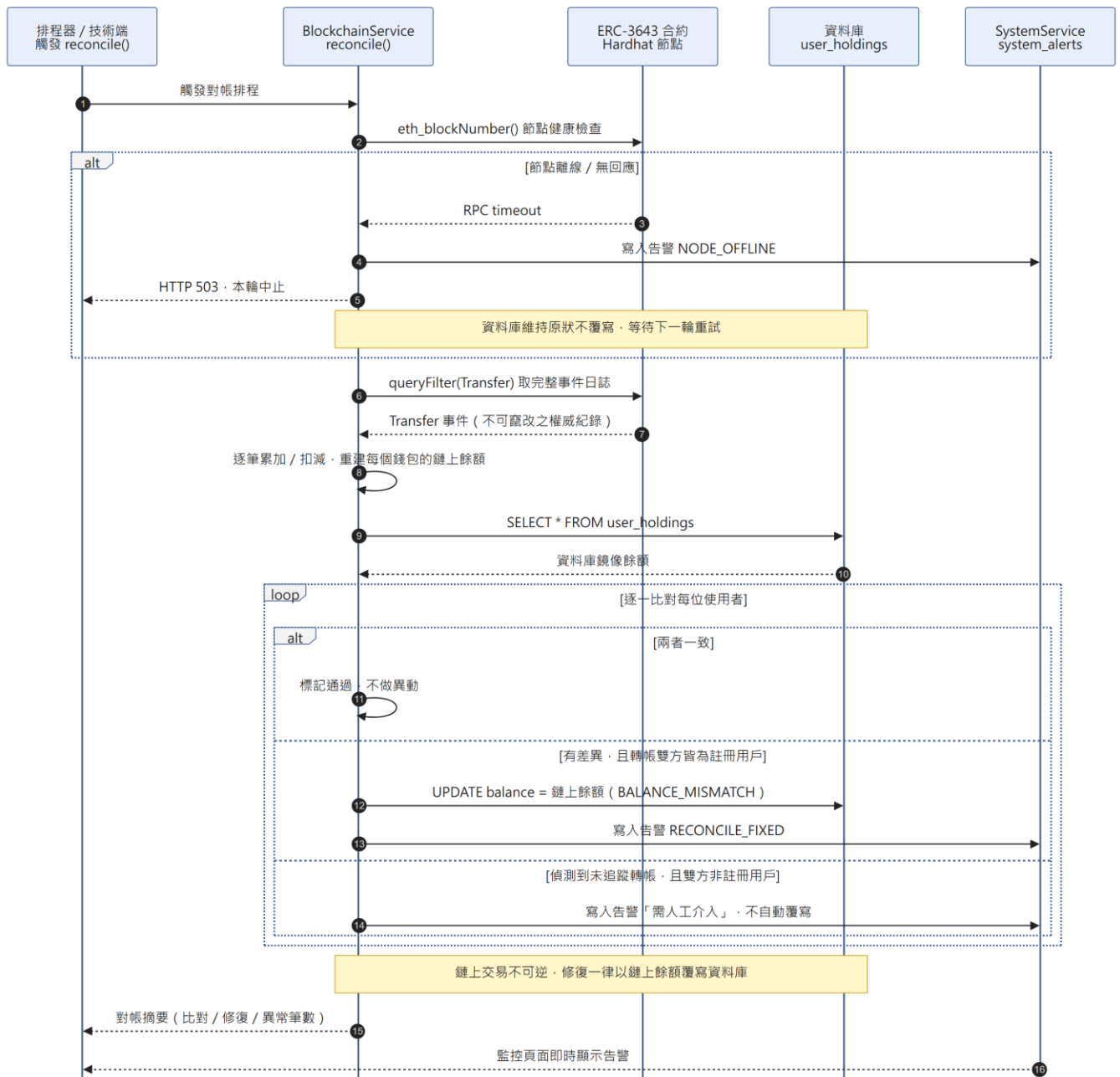


圖 3：循序圖－鏈上鏈下對帳排程

註：本圖之關鍵決策在於對帳系統以鏈上代幣合約之 Transfer 事件日誌為權威來源，重建真實餘額後與資料庫 UserHolding 紀錄比對。發現差異時，修復模式一律以鏈上餘額覆寫資料庫，因鏈上交易具不可逆性。例外情況包含：節點離線時 reconcile() 回傳 503（區塊鏈節點目前離線休眠中），以及偵測到未追蹤轉帳但轉帳雙方皆非系統註冊用戶時，系統標記為需人工介入，避免非法外部代幣干擾內部帳務。

3.3.4 循序圖 Lifeline 與程式碼實體對照

為避免架構圖與實作脫節，本專題所有循序圖上方之生命線皆對應專案原始碼中真實存在的層級與類別，逐項對照如表 5 所示。表中每一條生命線皆可在實際程式碼、模組或外部系統中找到對

應實體，用以佐證 3.3.1 至 3.3.3 節各循序圖並非概念示意，而是依實作結構繪製。

Lifeline 名稱	架構層級	真實程式碼對應位置	職責
Investor／Staff (Actor)	使用者角色	前端操作人員 (INVESTOR／BUSINESS／TECHNICAL)	人機介面互動
Frontend UI	前端介面層	React 元件 (訂單表單、用戶管理卡片等)	畫面渲染、使用者輸入
AuthContext	前端網路層	src/app/context/AuthContext.tsx	JWT 自動附加、401/403 自動登出
TransactionsController	後端控制層	src/transactions/transactions.controller.ts	REST API 端點宣告、Guard 驗證
TransactionsService	後端業務層	src/transactions/transactions.service.ts	交易邏輯、AMM 運算、悲觀鎖控制
BlockchainService	區塊鏈介面層	src/blockchain/blockchain.service.ts	封裝 ethers.js，執行 JSON-RPC 呼叫
TypeORM Repository	資料存取層	Repository<AppTransaction>等	執行 SQL 交易與實體映射
PostgreSQL Database	資料持久層	雲端代管 PostgreSQL	交易持久化、列級悲觀鎖、外鍵約束
智能合約層	區塊鏈合約	contracts/目錄下之 ERC-3643 合約	鏈上權限驗證與代幣轉帳
PropertiesService	後端業務服務層	src/properties/properties.service.ts	計算持倉百分比與應發租金、扣除信託專戶餘額、觸發鏈上撥款
SystemService	後端系統服務層	src/system/system.service.ts	管理全平台系統級告警 (system_alerts)

表 5：循序圖 Lifeline 與真實程式碼實體對照表

針對極端異常情境，圖 4 呈現當鏈上交易已成功、但資料庫因斷線或鎖定衝突寫入失敗時，系統如何透過重試與告警機制進行對帳處理。

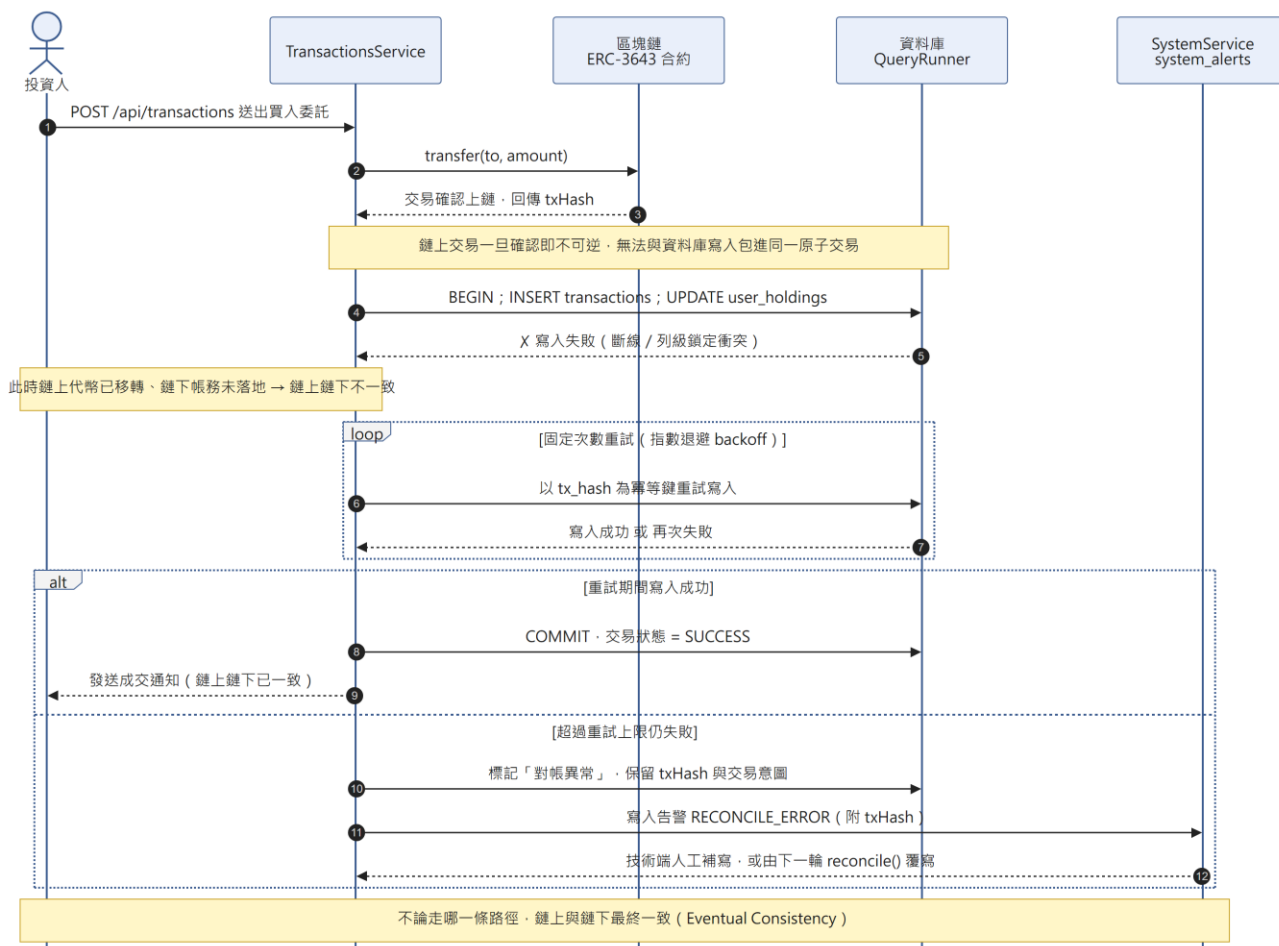


圖 4：循序圖－鏈上交易成功但資料庫寫入失敗之對帳處理

3.4 模組關係與介面

本系統依功能分為前端、後端 API、區塊鏈智能合約與爬蟲服務四大模組。前端負責畫面渲染與使用者輸入，並統一處理 JWT 附加與登出。後端 API 是唯一的業務邏輯集中處，負責身分驗證、AMM 計價、風控判斷與租金分潤運算。區塊鏈智能合約負責鏈上身分登記與受合規檢查約束的代幣移轉。爬蟲服務則負責定期擷取外部市場行情。

前端、後端 API 與區塊鏈三者間透過明確定義的介面相互呼叫，避免跨模組的直接耦合，爬蟲服務則例外地不透過介面呼叫，而是與後端共用同一資料庫。為避免單一大圖字體過小難以辨識，以下依「一張圖只描述兩個模組之間的關係」原則，拆為四張圖分別說明其介面與呼叫方式。

前端的主要負責範圍是「畫面渲染、使用者輸入蒐集與身分憑證的統一處理」，不承擔任何業務判斷。所有 AMM 計價、風控與權限決策皆由後端執行，前端僅負責把結果呈現給使用者。基於此範圍，前端 React 元件依角色分為投資人交易頁面、KYC 審核後台與技術監控儀表板三組，並共用 AuthContext 統一為每一次請求附加 JWT，以及在收到 401 或 403 時自動登出，避免每個元件各自處理憑證而產生行為不一致。後端 NestJS 則依業務領域切分為 AuthModule、UsersModule、TransactionsModule、PropertiesModule、SystemModule 等模組。兩者間唯一的溝通管道為帶 JWT 之 REST API，JWT 內嵌之角色 (INVESTOR、BUSINESS、TECHNICAL) 即為後端 Guard 進行存取

控制之依據。兩模組之介面關係如圖 5 所示。

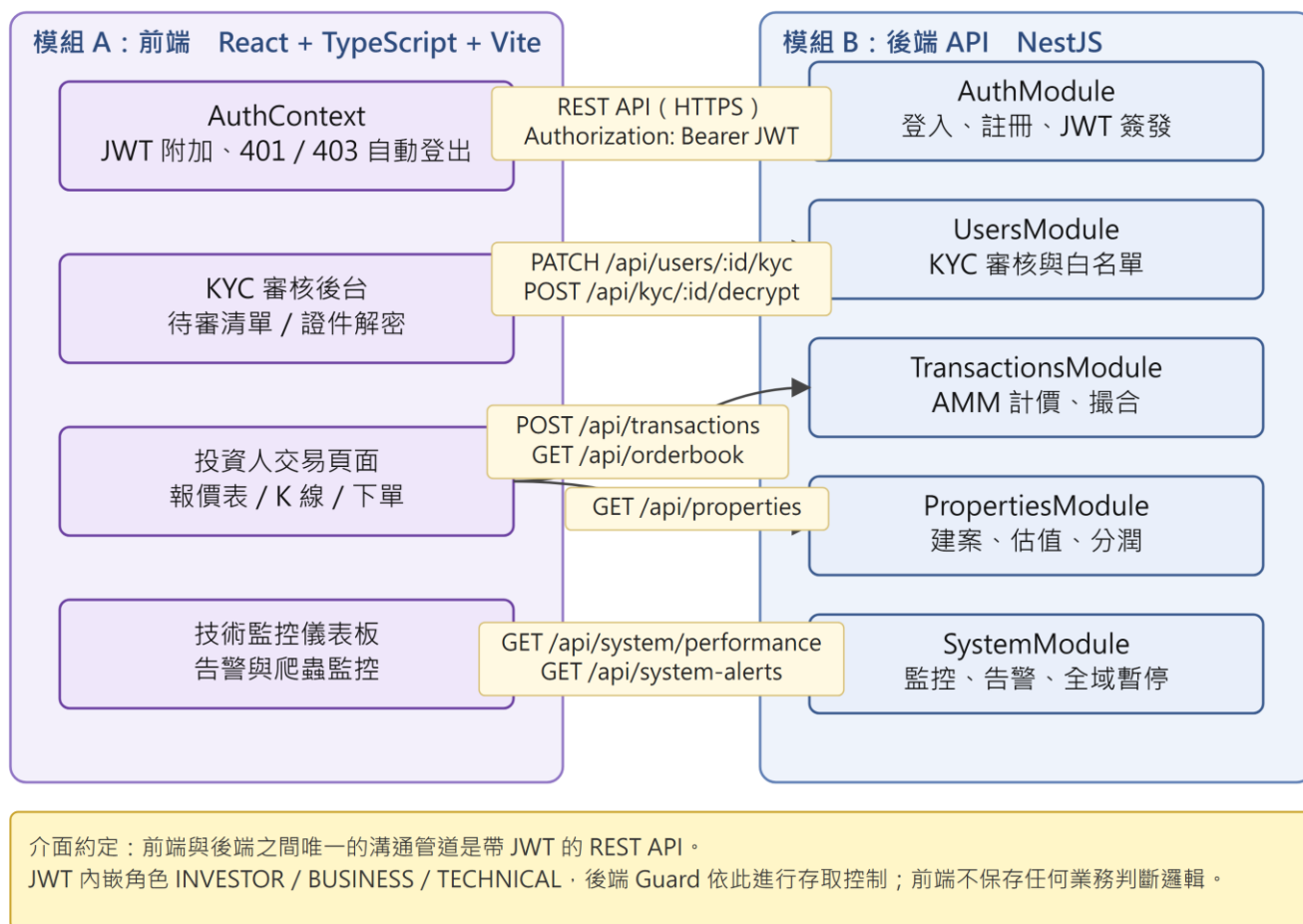


圖 5：模組關係圖（一）前端與後端 API

後端 API 對資料庫的主要負責範圍是「業務邏輯運算與交易完整性保證」，資料庫本身不含商業規則，所有計價、分潤與風控判斷皆在 Service 層完成後才落庫。基於此範圍，各 Service 層透過 TypeORM Repository 存取 16 個資料庫實體。其中 TransactionsService 於下單時對建案記錄取 pessimistic_write 列級鎖，以避免 AMM 儲備量計算產生競態條件。PropertiesService 則將整批租金分潤包裝於單一資料庫交易內，任一筆計算失敗即整批 ROLLBACK。transactions.idempotency_key 與 user_holdings 之 UNIQUE 約束則於資料庫層杜絕重複扣款，形成「應用層檢查、資料庫層兜底」的雙層防護。兩模組之介面關係如圖 6 所示。

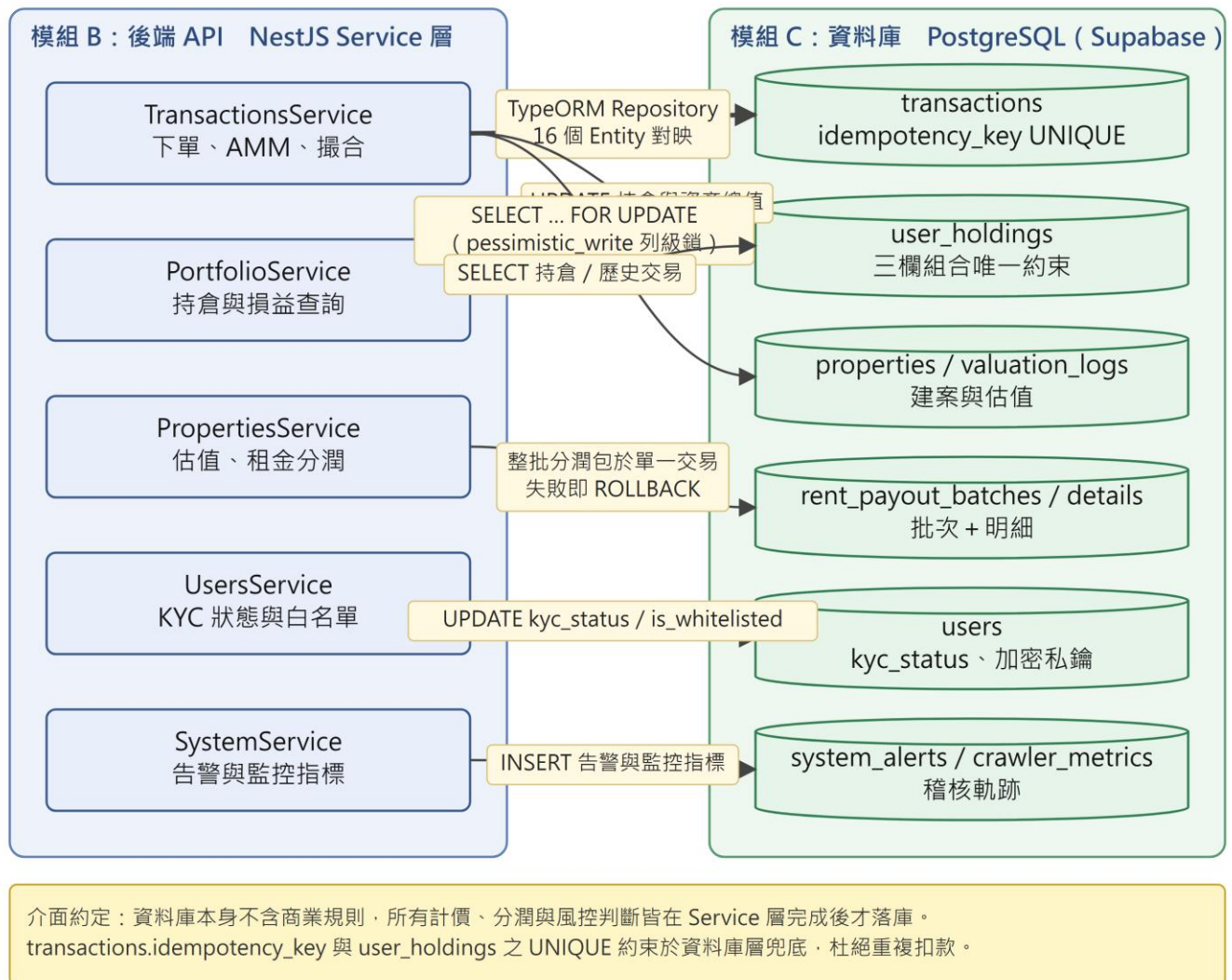


圖 6：模組關係圖（二）後端 API 與資料庫

後端 API 對區塊鏈的主要負責範圍是「將業務動作翻譯為鏈上呼叫，並處理鏈上回應的非同步與不可逆特性」。基於此範圍，BlockchainService 封裝 ethers.js，以 JSON-RPC 呼叫部署於 Hardhat 節點之 ERC-3643 合約群，並由 NonceManager 處理管理員錢包連續送出交易時之 nonce 競爭。合約群內部形成一條合規鏈：Token 合約每次 transfer 前先呼叫 Compliance，Compliance 向 IdentityRegistry 查驗雙方 Identity，再依 ClaimTopicsRegistry 規定之主題比對 KYC Claim 並確認簽發者列於 TrustedIssuersRegistry，任一項不符即 revert。此設計使合規檢查不因後端服務中斷而失效。兩模組之介面關係如圖 7 所示。

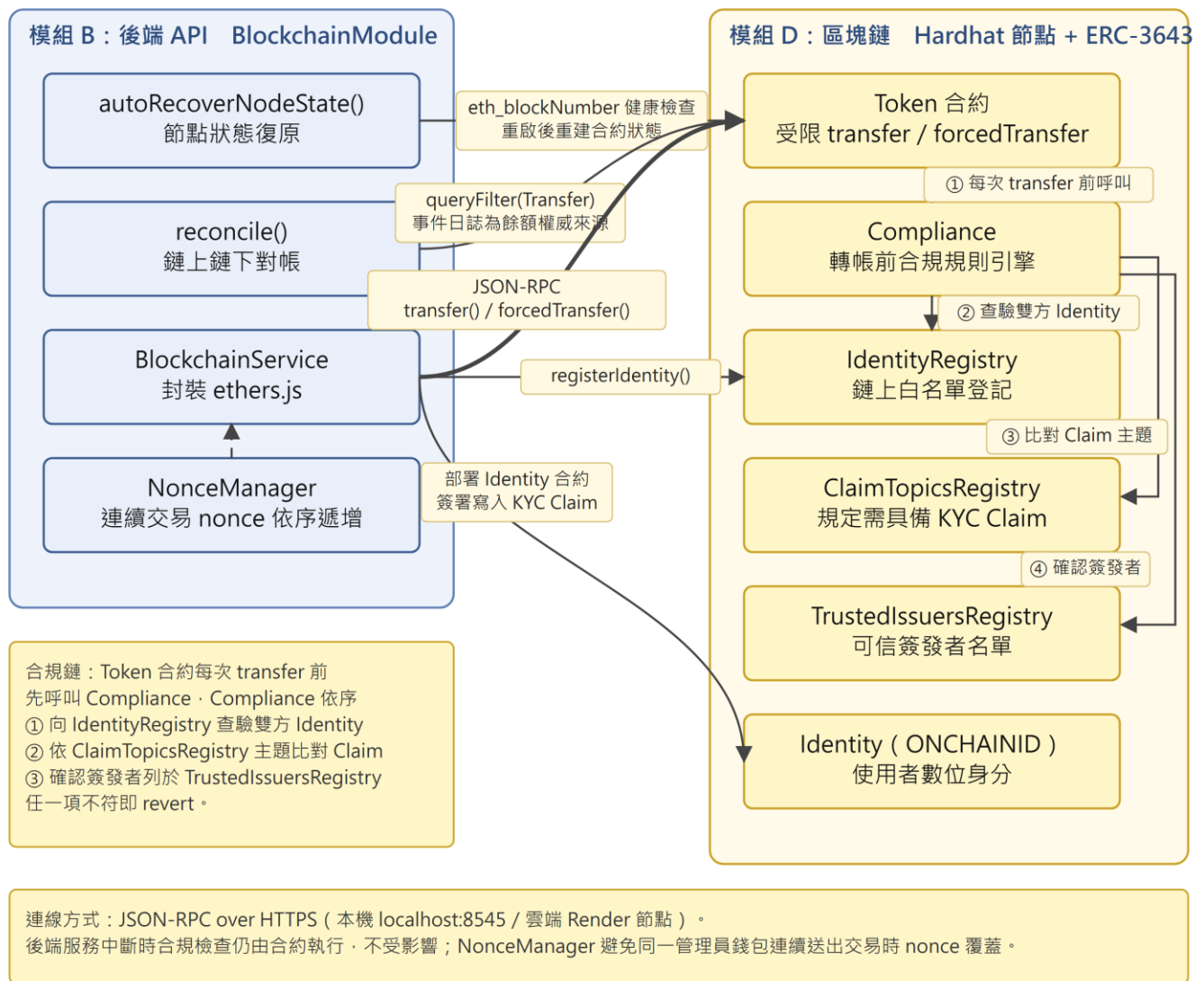


圖 7：模組關係圖（三）後端 API 與區塊鏈

爬蟲服務的主要負責範圍是「外部公開行情的定期擷取與品質評分」，它不參與任何交易或權限流程，是四大模組中唯一不經後端 REST API 者。基於此範圍，爬蟲以 Playwright 等待 591 新建案頁面之 JavaScript 渲染完成後擷取六項欄位，換算資料完整度指標，再以 psycopg2 直接寫入 properties、valuation_logs 與 crawler_metrics 三張資料表。此設計繞過後端驗證層以換取批次寫入效能，因此資料品質改以完整度指標與失敗筆數監控，必要時由技術端觸發全域暫停，避免異常價格影響交易。兩模組之介面關係如圖 8 所示。

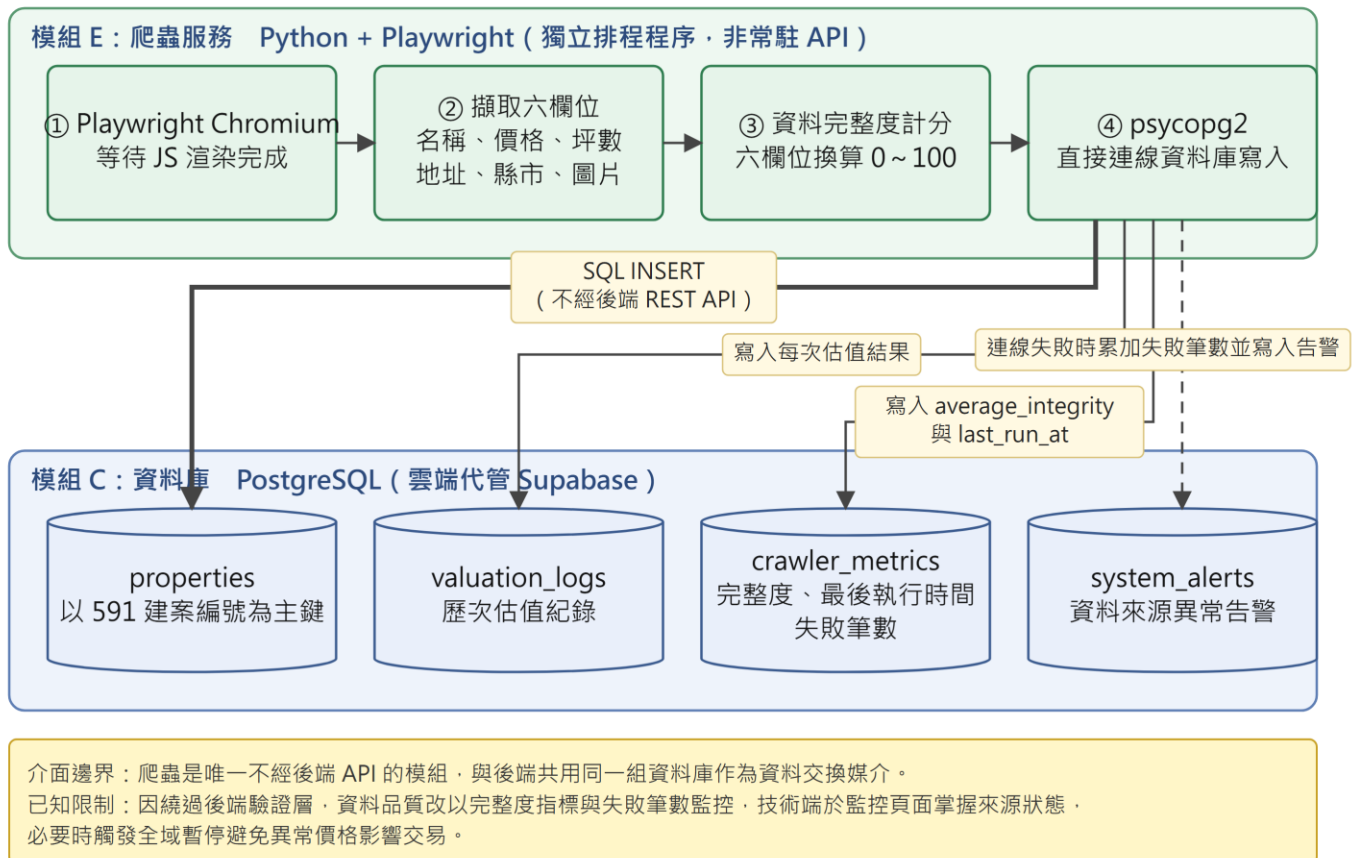


圖 8：模組關係圖（四）爬蟲服務與資料庫

圖 9 展示業務端如何落實身分驗證與職務分離：由投資人自行上傳身份文件、業務端於鏈下完成人工審查後，才在鏈上註冊 Identity 合約並簽發 KYC Claim，說明技術層面的合規檢查如何與人工判斷分工銜接。

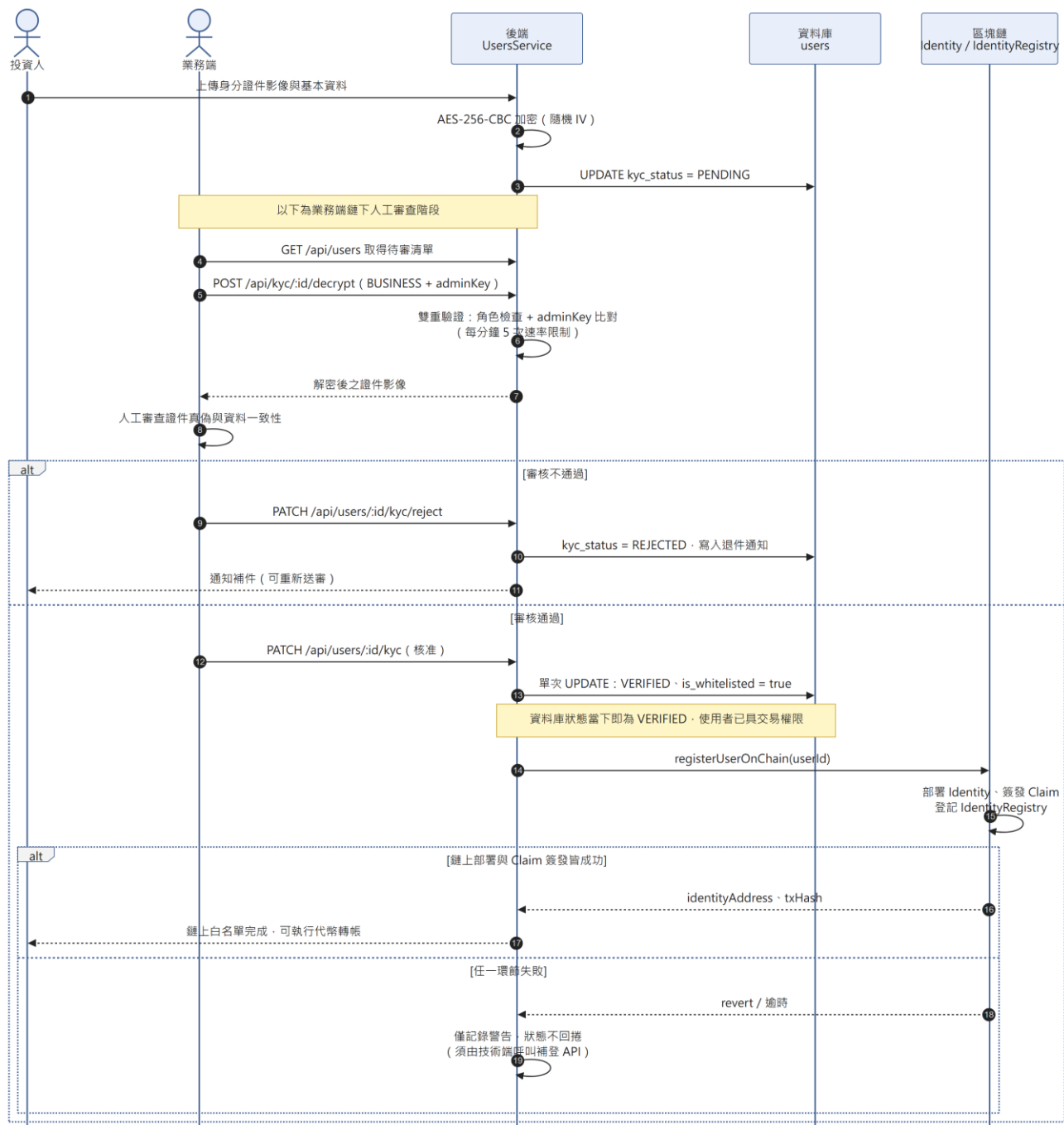


圖 9：循序圖－業務端 KYC 審核流程

註：業務端核准後，後端於單一次資料庫更新中同時將 kyc_status 設為 VERIFIED 並將 is_whitelisted 設為 true，隨後才呼叫 BlockchainService 部署 Identity 合約與簽發 KYC Claim。例外處理上，若鏈上部署或 Claim 簽發失敗，系統僅記錄警告並於回應中夾帶錯誤訊息，資料庫狀態不回捲。已知限制：鏈下審核與鏈上登記並非原子操作，鏈上登記失敗時使用者於平台端已具備交易權限，須由技術端呼叫 POST /api/blockchain/register-user/:userId 手動補登。正式產品化應導入「待鏈上確認」之中間狀態，待鏈上確認成功後才放行交易權限。

圖 10 展示平台依據單一建案代幣持有比例，將租金收益一次性批次分配予所有投資人的處理流程，並區分「計算應分配金額」與「實際銀行撥款」兩個階段的責任邊界。

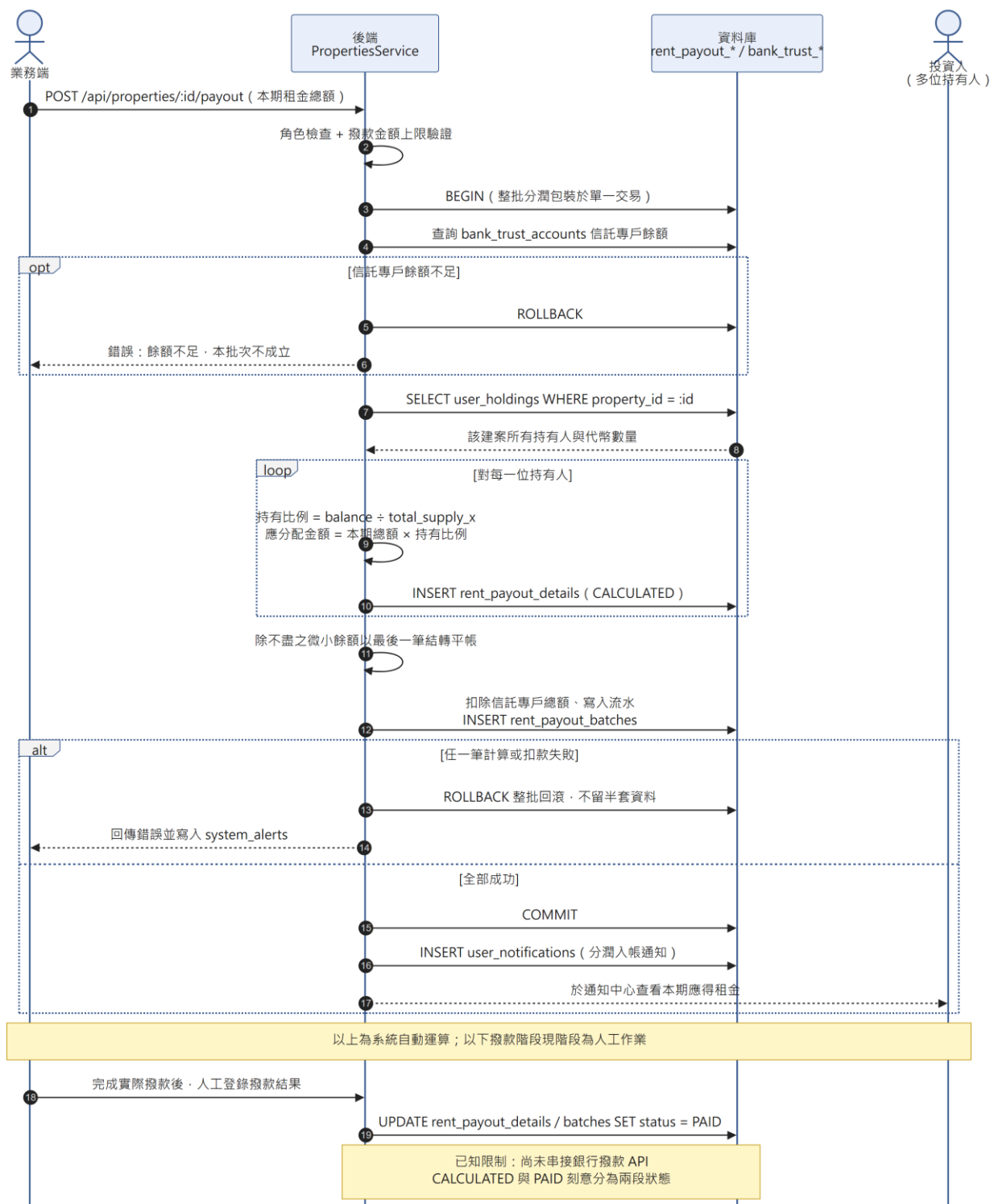


圖 10：循序圖－租金分潤批次處理

註:關鍵決策在於分潤採「批次 (rent_payout_batches) + 明細 (rent_payout_details)」二層式展開，依代幣持有比例計算各投資人應得金額後，由管理員錢包於鏈上轉出等值原生代幣以模擬撥款，成功者標記為 PAID、失敗者標記為 FAILED，並自信託專戶扣除對應金額且寫入一筆流水紀錄。

3.5 系統架構圖

本節聚焦於部署層級之系統架構，說明前端、後端、資料庫與區塊鏈節點之間的部署邊界與通訊協定，如圖 11 所示。

就部署邊界而言，前端為靜態 React SPA，後端 API 與 Hardhat 區塊鏈節點皆部署於 Render 雲端容器，資料庫則由 Supabase 雲端代管之 PostgreSQL 服務提供，三者分屬不同的執行環境。就通訊協定而言，前端與後端之間採用帶 JWT 的 HTTPS REST API，後端與區塊鏈節點之間採用 JSON-RPC，後端與資料庫之間則透過 TypeORM 之 PostgreSQL 連線池。爬蟲服務為唯一例外，它以獨立 Python 程序執行，不經應用層介面而直接連線資料庫寫入，此為刻意的架構取捨：優點是批次寫入效能高，代價是缺少應用層業務驗證，因此改以 3.2.4 節所述之資料完整度指標作為防護。

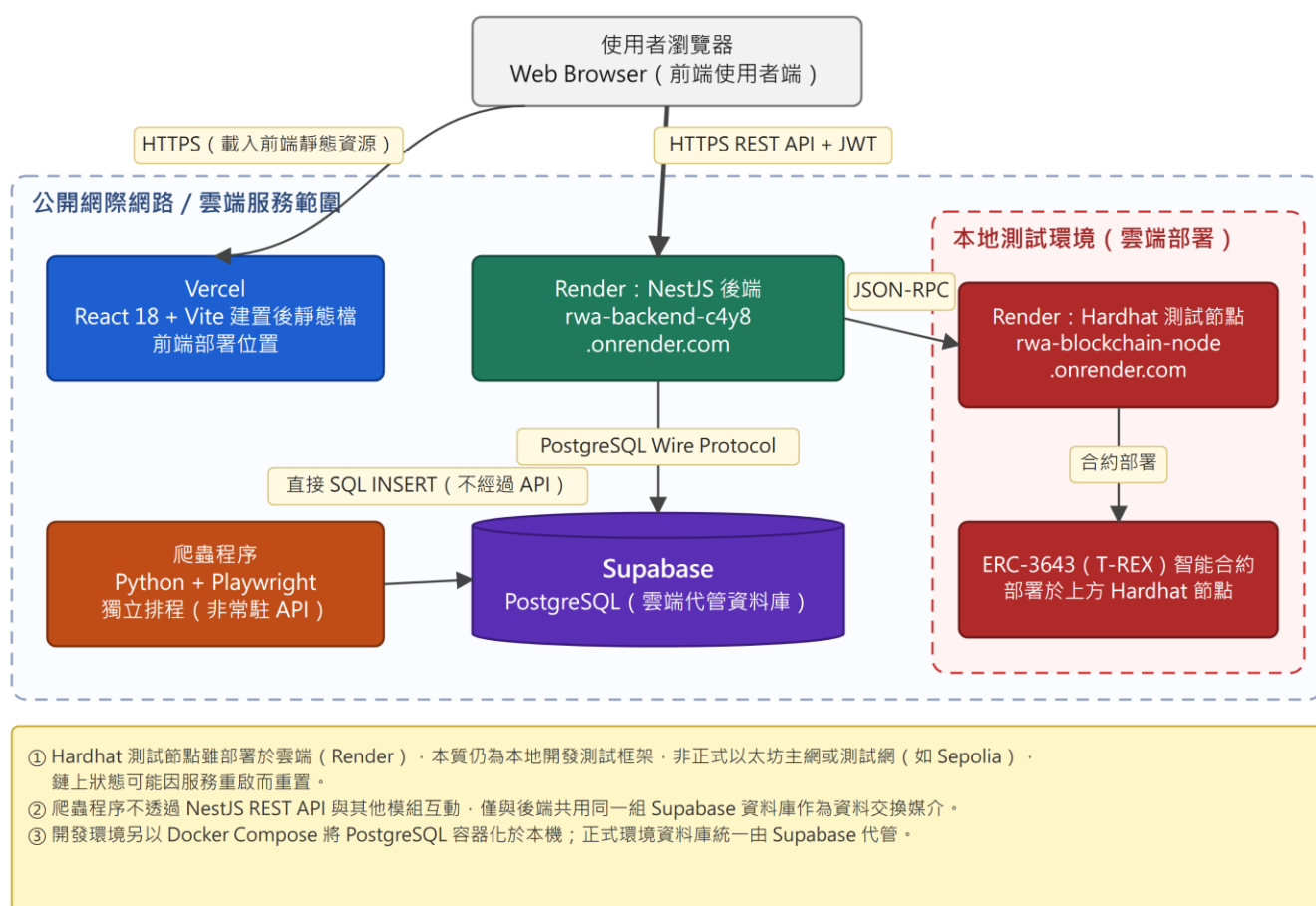


圖 11：系統架構圖

註：本圖之已知限制為 Hardhat 屬單節點測試環境，重啟後鏈上狀態會清空，須靠後端 autoRecoverNodeState 機制偵測並重建。

3.6 ER 模型圖

本節以五張 ER 子圖呈現資料表之間的關聯結構。本系統資料庫共 16 張核心資料表，為避免單一 ERD 因密度過高導致關係名稱、主外鍵與約束難以辨識，依身分與 KYC、房產與估值、訂單與

交易、租金分潤、區塊鏈與對帳五個業務領域拆分為五張子圖，分別為圖 12 至圖 16。各資料表之欄位、資料型態與約束則列於附錄 D，兩者依相同的五個業務領域順序排列，可逐一對照查閱。

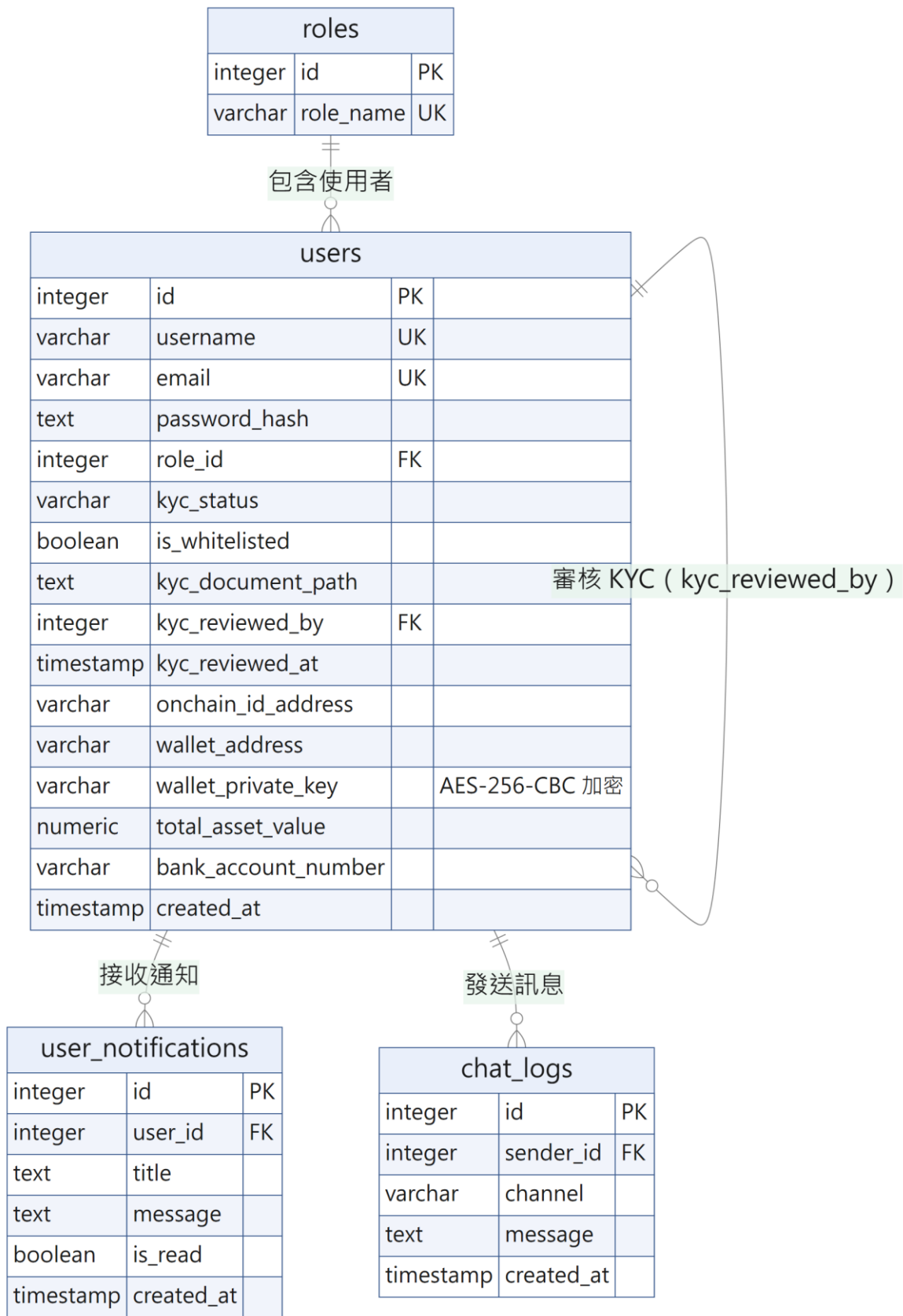


圖 12：資料庫—身分與 KYC 模組 (Identity & KYC)

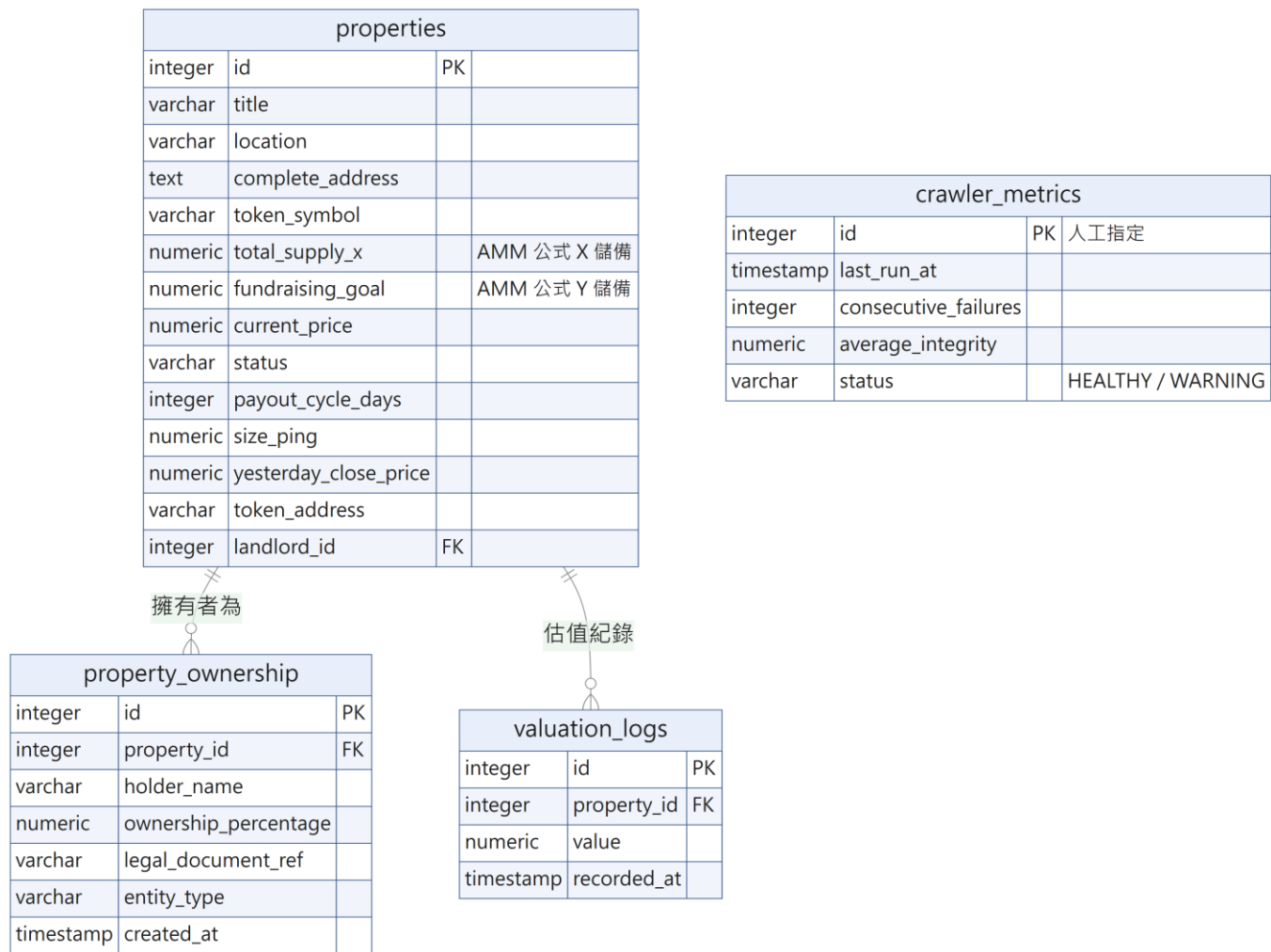


圖 13：資料庫－房產與估值模組（Property & Valuation）

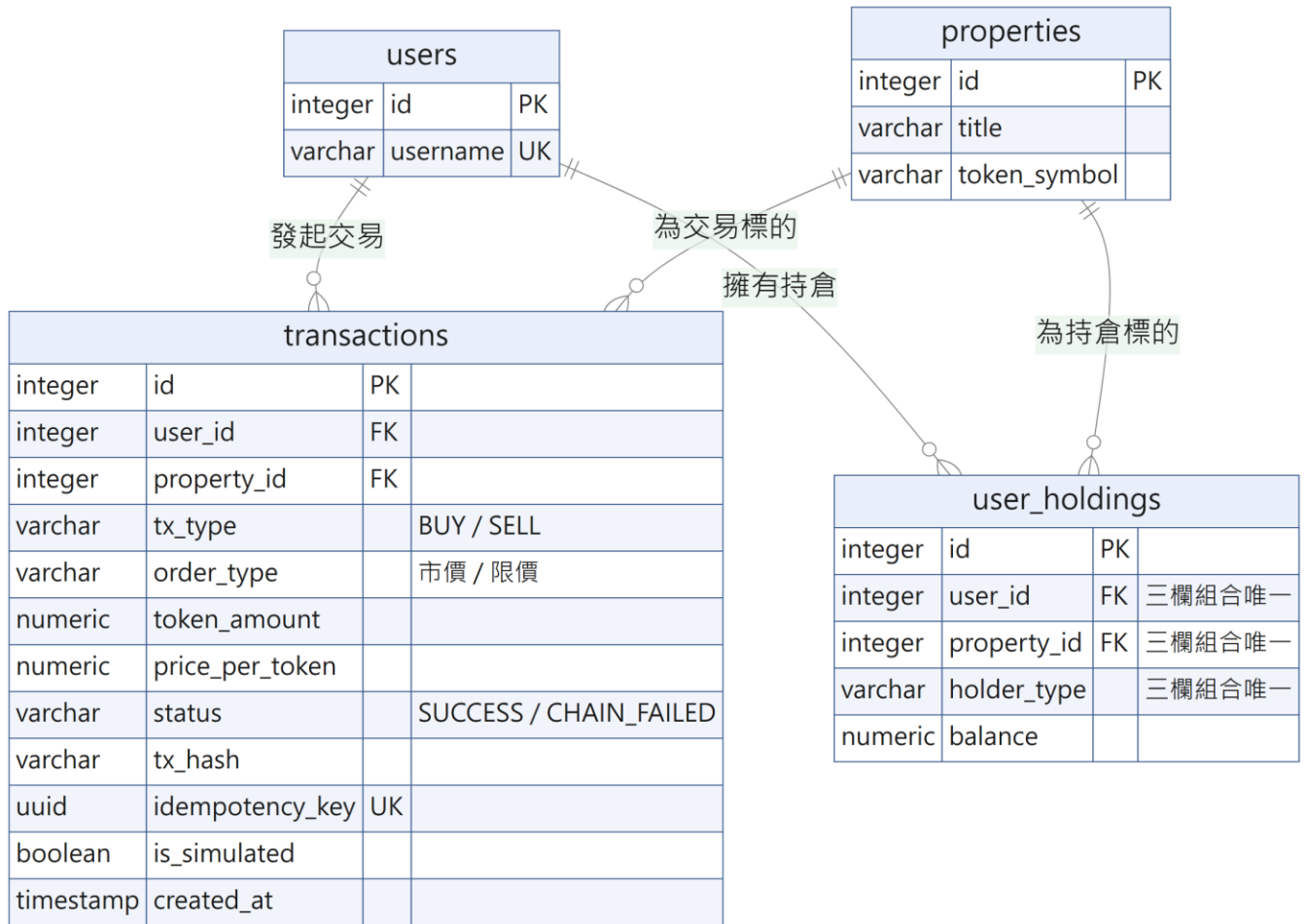


圖 14：資料庫－訂單與交易模組（Orders & Transactions）

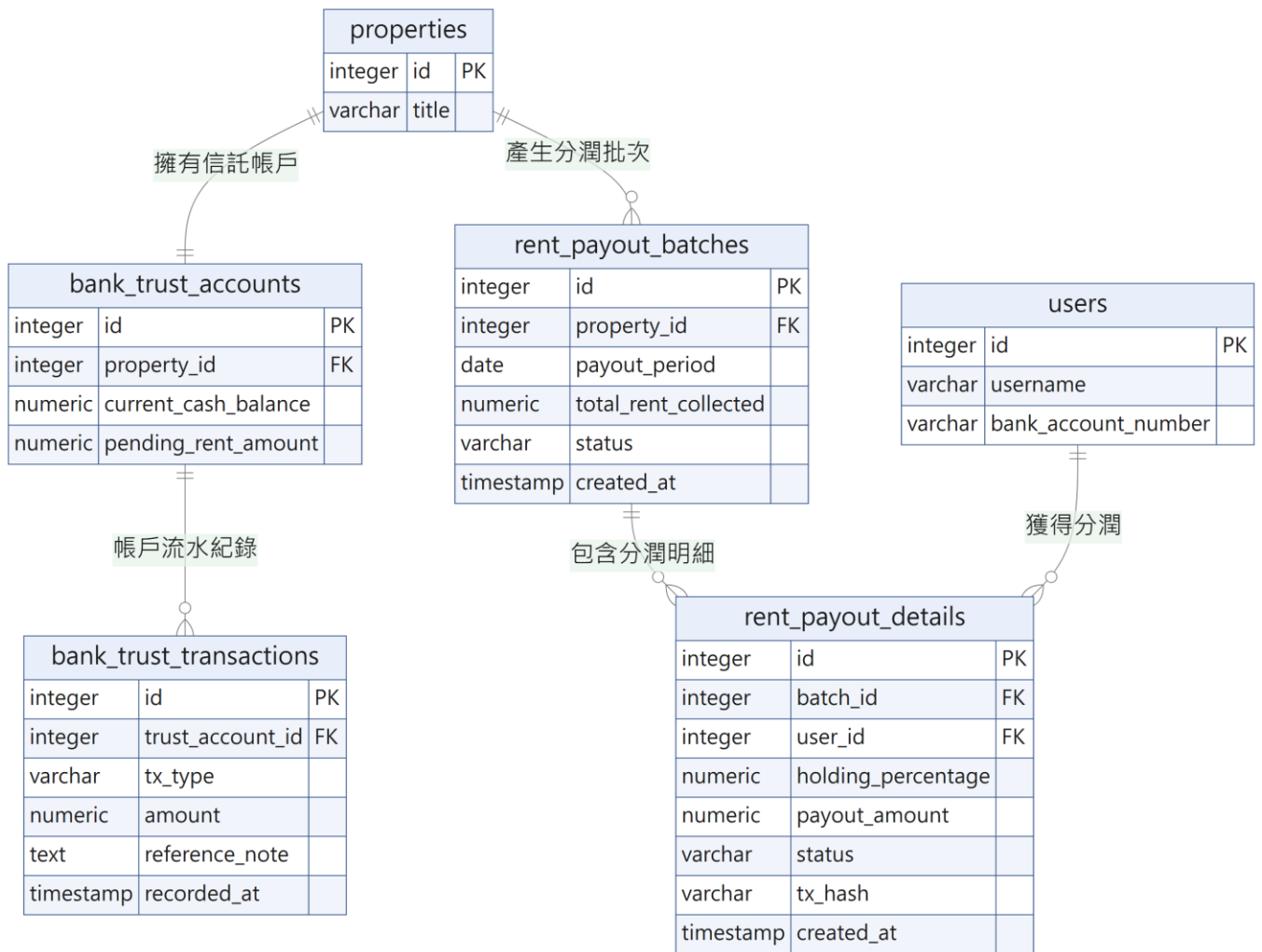


圖 15：資料庫－租金分潤模組（Rent & Payouts）

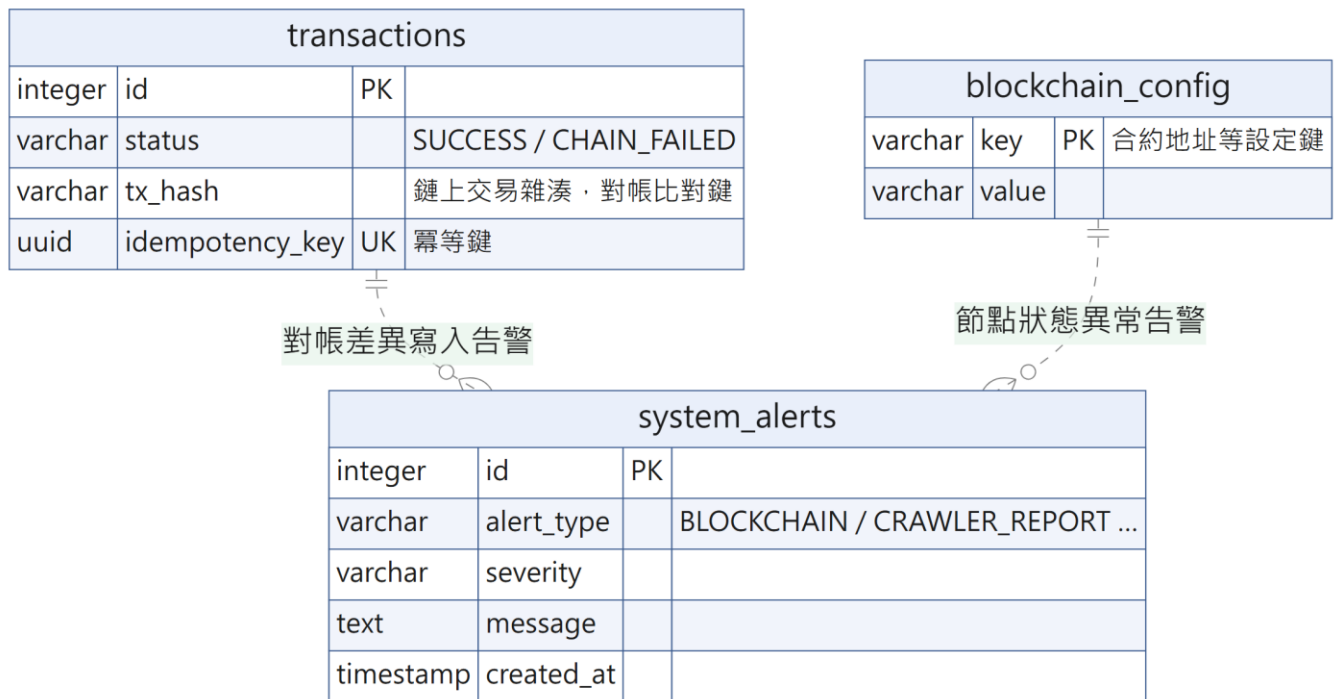


圖 16：資料庫－區塊鏈與系統對帳模組（Blockchain & Reconciliation）

註：五張子圖之關鍵約束在於 user_holdings (user_id, property_id, holder_type) 與 transactions (idempotency_key) 皆設有 UNIQUE 約束，於資料庫層級杜絕重複扣款。system_alerts、valuation_logs 與 crawler_metrics 三張表保留完整歷史軌跡，支援事後稽核與對帳回溯。跨子圖之外鍵關係（如 users.id 被 user_holdings、transactions 等表引用）已於各子圖標明 FK 對應，確保拆分後仍可追溯完整關聯。

3.7 使用案例圖

本節依據投資人、業務端與技術端三大角色，分別界定各角色可達成之使用者目標與功能邊界。投資人之使用案例集中於市場瀏覽、委託下單、投資組合查詢與通知接收，如圖 17 所示。業務端之使用案例集中於 KYC 審核、白名單管理與租金分潤撥款確認，如圖 18 所示。技術端之使用案例集中於系統監控、爬蟲狀態掌握、全域交易暫停與鏈上合約維運，如圖 19 所示。

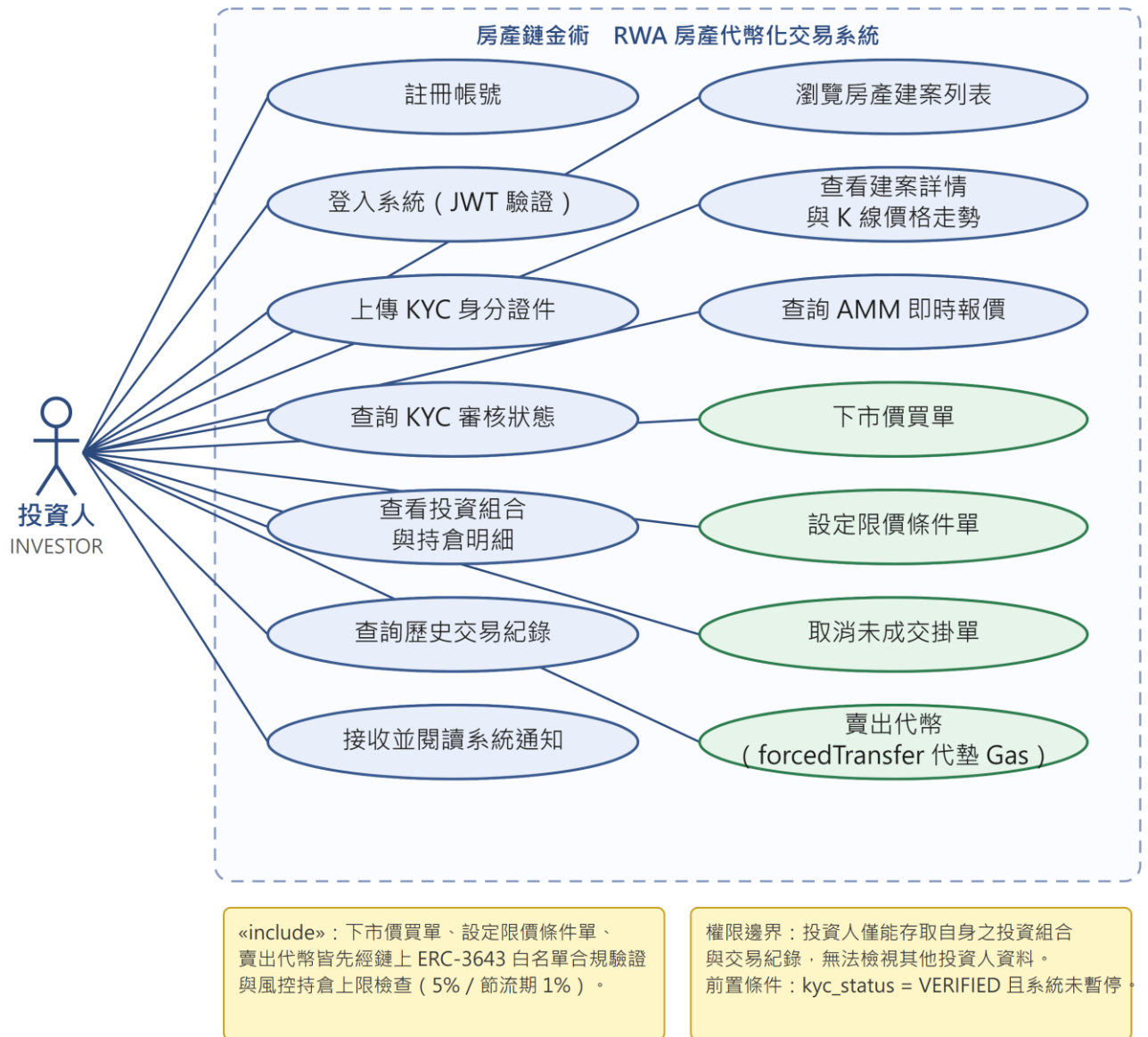


圖 17：使用案例圖－投資人

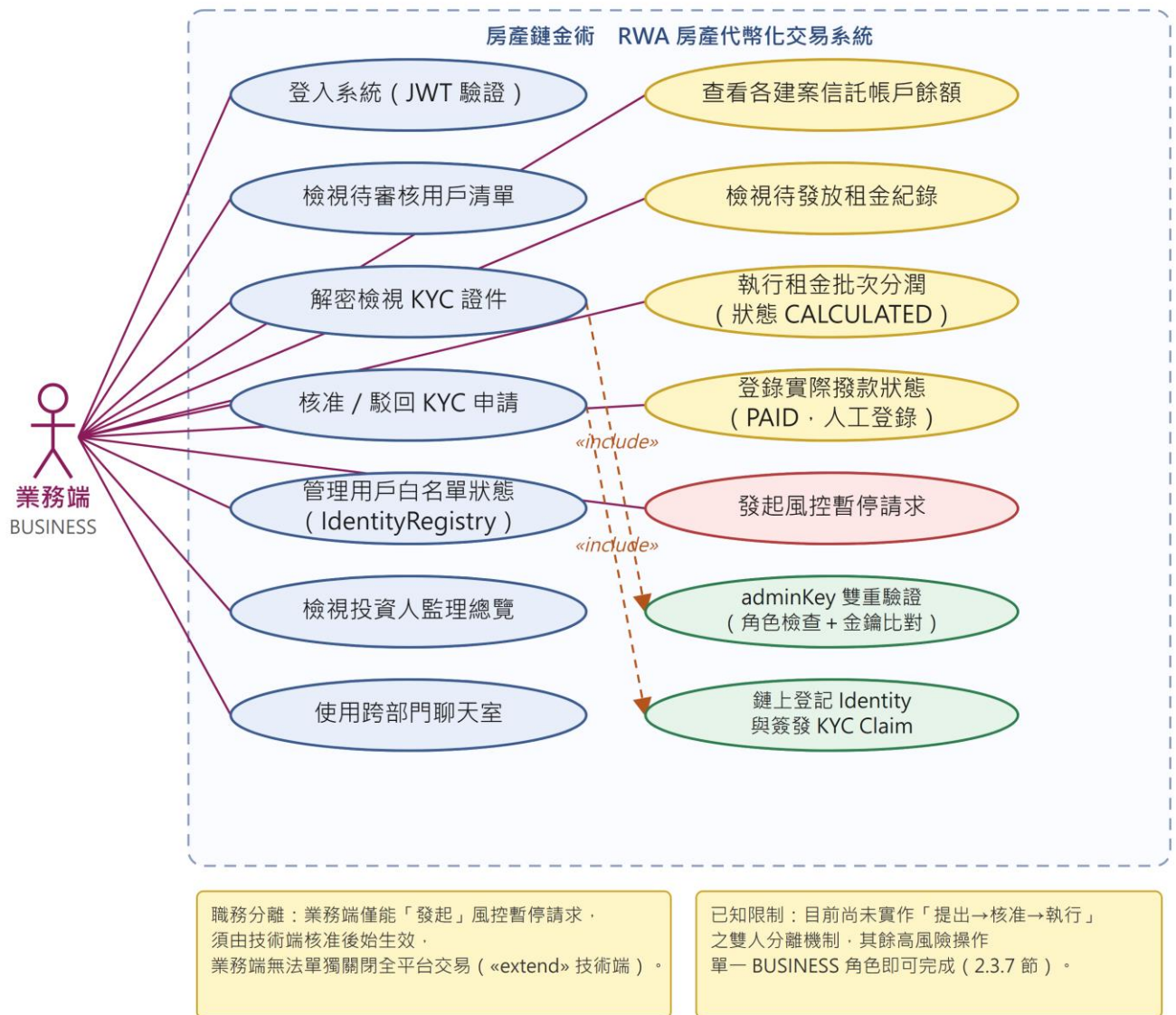


圖 18：使用案例圖－業務端

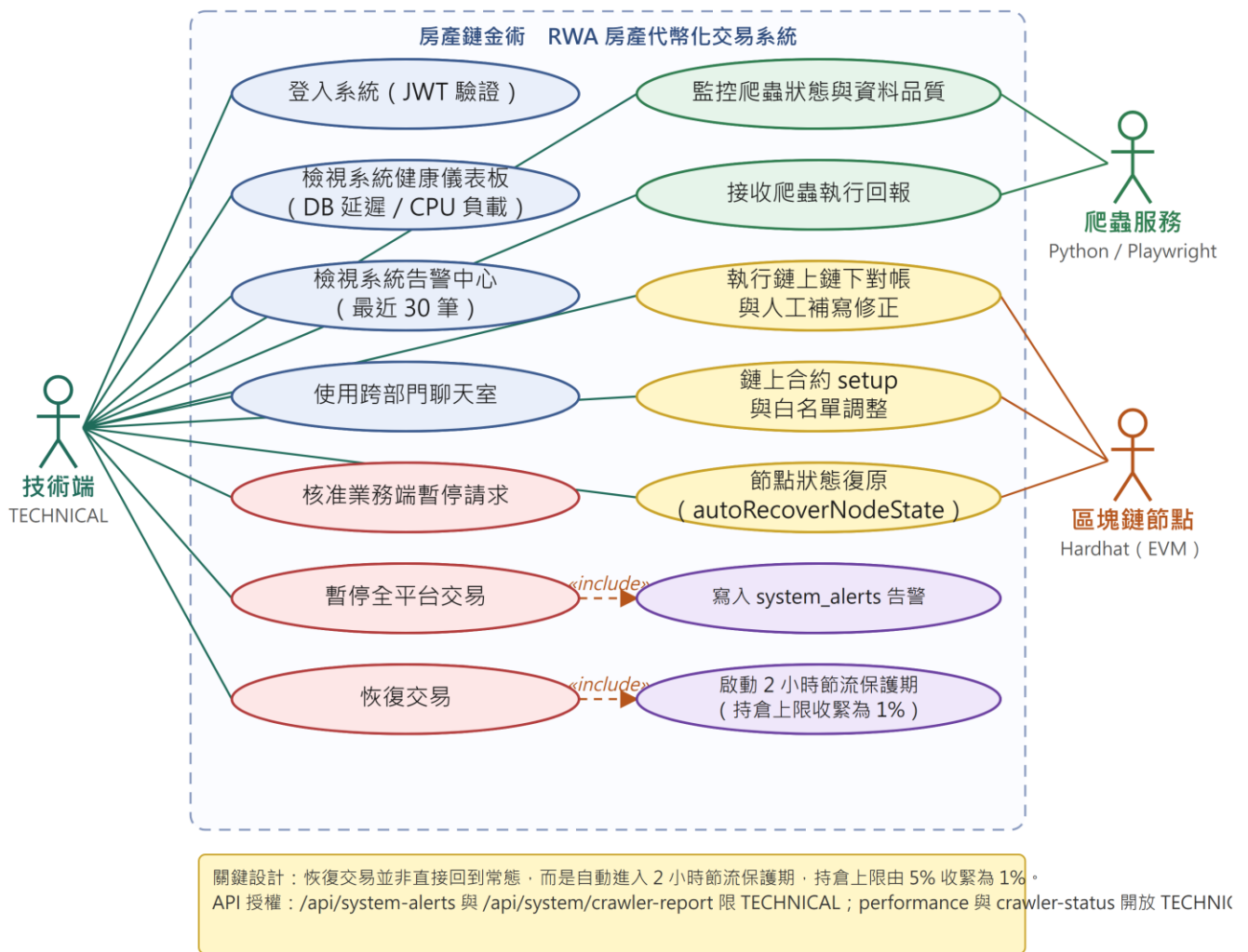


圖 19：使用案例圖－技術端

註：三張圖之核心設計為職務分離（Separation of Duties）：業務端於前端協作流程中僅能發起風控暫停請求，由技術端核准後才實際執行暫停。惟後端 POST /api/system/state 對 BUSINESS 與 TECHNICAL 授予相同權限，此「提出→核准」順序係由前端流程約束而非後端強制（鏈上合約之 POST /api/blockchain/pause-toggle 則確實限 TECHNICAL），屬原型階段已知限制。投資人僅能存取自身投資組合與交易紀錄，無法檢視其他投資人資料，落實最小權限原則。高風險管理操作（如白名單調整）以目標式命名呈現於技術端圖中，並與 5.1 節 RBAC 矩陣之 endpoint 層級授權規則一致。已知限制為目前尚未實作「提出→核准→執行」之雙人分離機制，高風險操作單一角色即可完成，此限制已於 2.3 節安全設計中揭露。

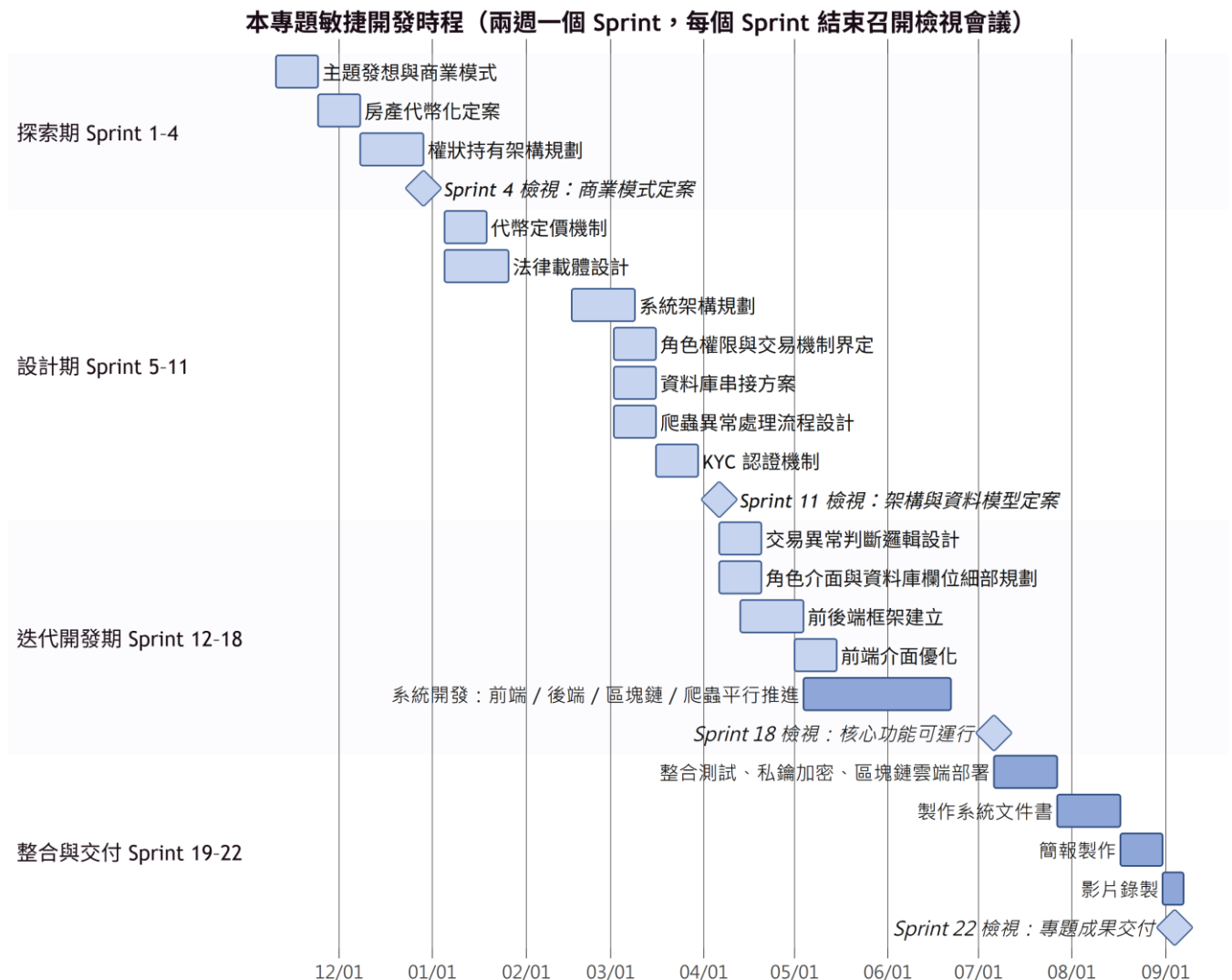
3.8 甘特圖

本專題採 3.1 節所述之敏捷開發模式，以兩週為一個 Sprint 週期進行迭代，時程規劃即依此原則劃分為多個 Sprint 區間，而非一次性的瀑布式排程。圖 20 為本專題自主題發想至簡報製作之時程規劃，依敏捷開發原則劃分為探索期、設計期、迭代開發期與整合交付期四個階段，共 22 個 Sprint。圖中菱形節點為各階段結束時之 Sprint 檢視會議與其驗收標的，涵蓋商業模式定案、系統架構設計、前後端與區塊鏈開發、整合測試至文件產出等工作。各區間對應之 GitHub Issues 紀錄可回

溯每個 Sprint 實際完成之項目，作為敏捷迭代確實落實之佐證。

圖 20：專題時程甘特圖

團隊分工依模組邊界劃分，四名成員各自負責可獨立交付的範圍，說明如下。



組長（翁敬倫）：負責整體專案管理與系統架構決策，並主導資料庫設計，包含 16 張資料表之正規化設計與 TypeORM Entity 對映。另負責主持每兩週一次的 Sprint 檢視會議，彙整跨模組相依議題並排定優先順序。

組員（吳振維）：負責後端 API 與區塊鏈智能合約開發，包含 NestJS 模組切分、ERC-3643（T-REX v4）合約群之部署腳本、IdentityRegistry 與 ClaimTopicsRegistry 之設定流程、AMM 定價與兩階段撮合邏輯、風控引擎與租金分潤批次演算法，以及後端 42 項單元、負向、併發與故障注入測試，與 20 項智能合約自動化測試之撰寫。

組員（陳政齊）：負責前端介面實作，包含投資人交易頁面、業務端 KYC 審核後台與技術端監控儀表板三組畫面，以及 AuthContext 之 JWT 統一附加與自動登出機制、K 線圖與下單表單

等互動元件之開發。

組員（廖偉哲）：負責爬蟲服務與外部資料整合，包含 Python Playwright 擷取 591 新建案頁面之流程設計、六欄位資料完整度計分機制、psycpg2 寫入 properties 與 crawler_metrics 之批次流程，以及爬蟲健康狀態告警與技術端監控頁面之資料串接。

各成員於每個 Sprint 內依上述時程並行推進所屬模組，並於 Sprint 會議中同步整合進度、排除跨模組相依問題，詳細議題紀錄留存於 GitHub Issues 供事後核對。跨模組介面（例如後端與區塊鏈之間的 BlockchainService 介面、爬蟲與資料庫之間的欄位定義）則於設計期即先行約定，以降低並行開發時的整合成本。

3.9 活動圖

本節之活動圖分別對應 3.7 節使用案例圖中「投資人買入代幣」與「業務端 KYC 審核」兩項使用案例，並已於各圖左上角標註其對應之使用案例名稱與 include 關係，供讀者逐項核對活動流程與使用案例之對應性。

圖 21 以狀態機角度描述「投資人購買代幣」之完整動作流程，補充循序圖側重訊息傳遞、活動圖側重決策分支的互補視角。圖中涵蓋市價單與限價單兩種分支路徑及其判斷節點，包含現金餘額檢查、持倉上限檢查，以及限價單成交均價是否超出使用者限價，並標示 CHAIN_FAILED 等例外終止狀態。

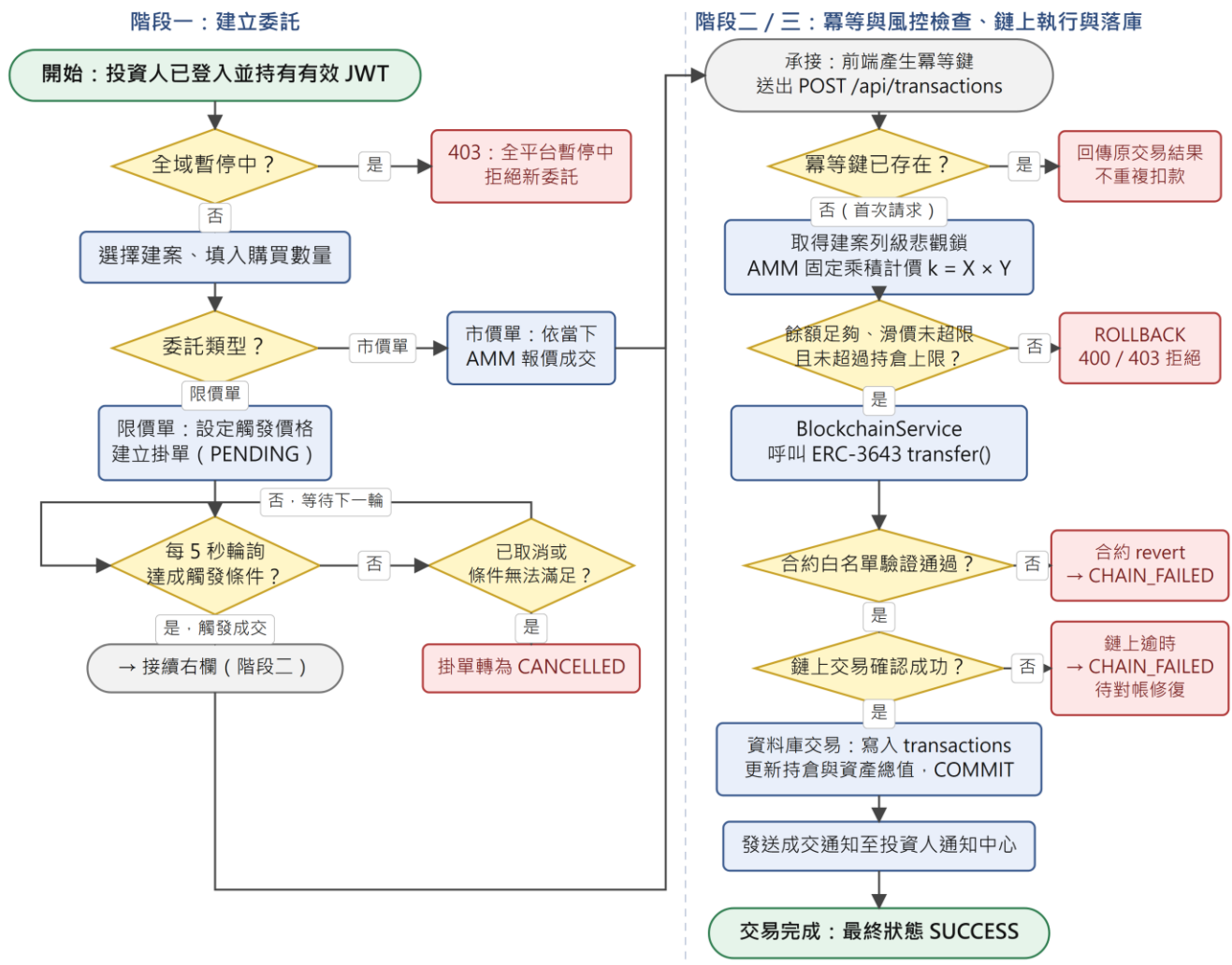


圖 21：活動圖－投資人購買代幣之流程

圖 22 以流程圖形式呈現 KYC 審核之判斷節點與分支路徑，與圖 9 之循序圖互補：循序圖側重系統元件間之訊息傳遞順序，本圖側重審核過程中的條件判斷與分支結果。其關鍵判斷節點在於「鏈上部署與 Claim 簽發是否皆成功」，唯有兩者皆確認成功才會進入 VERIFIED 終點，任一環節失敗則導向重試或人工介入分支。

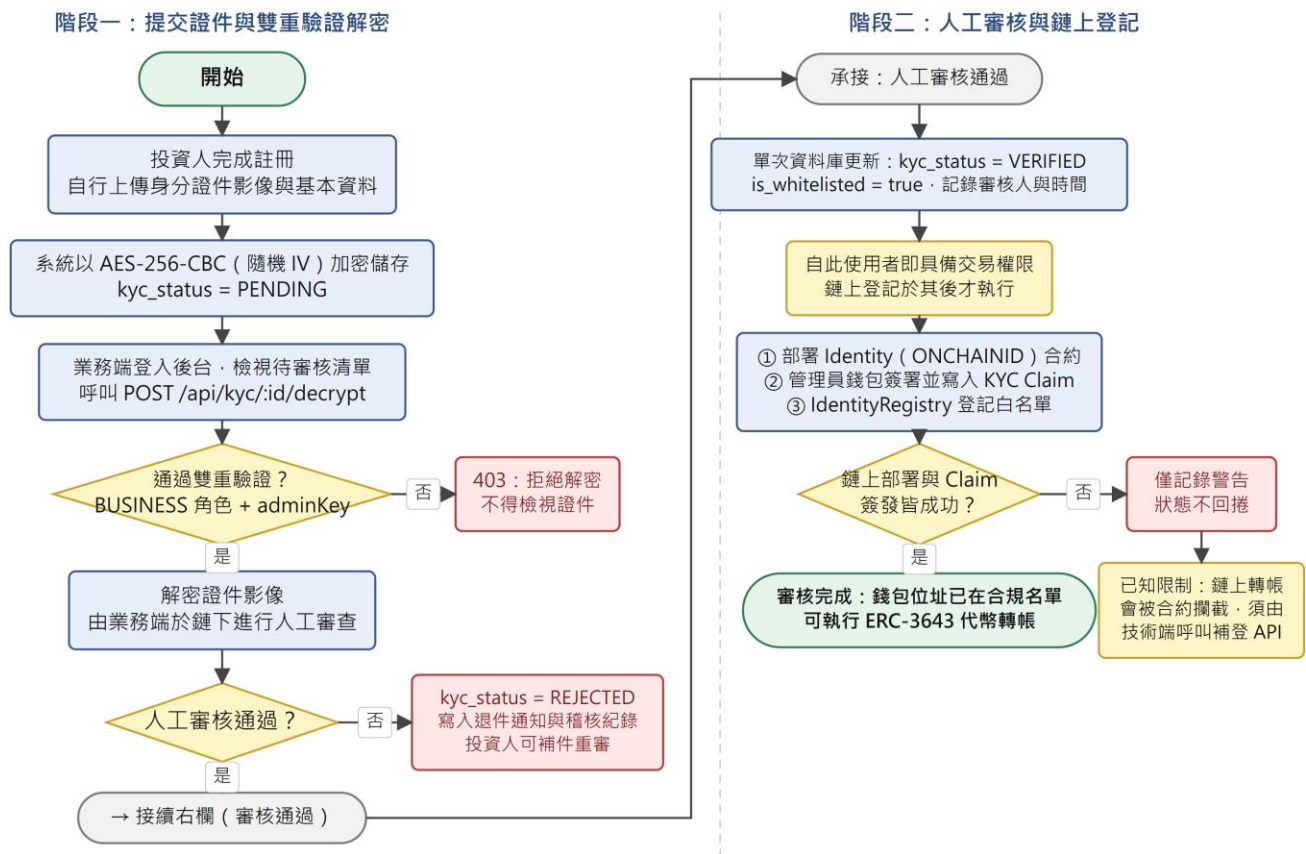


圖 22：KYC 審核流程圖

註：本圖之關鍵判斷節點在於「鏈上部署與 Claim 簽發是否皆成功」，唯有兩者皆確認成功才會進入 VERIFIED 終點。任一環節失敗則導向重試或人工介入分支，不會有中間狀態被誤判為審核完成，本圖為目標設計，實際實作中資料庫狀態於業務端核准當下即更新為 VERIFIED，鏈上部署或 Claim 簽發失敗時狀態並不回捲。此落差已於 3.3 節「處理流程」與圖 9 註中一併揭露，屬原型階段已知限制。

為確保所有核心業務與高風險例外皆具備架構設計依據，本專題另建立功能需求與循序圖之對應表，因篇幅較長且屬查考性質，列於附錄 E。

4. 系統開發工具與使用環境

4.1 系統技術架構與安全設計

本系統於前端、後端、區塊鏈、爬蟲、資料庫、容器化與部署七個層面所採用之最終技術組成如表 6 所示。各項工具之選用理由與比較準則已於 3.2 節說明，此處僅列出最終確定之版本組成，作為後續安全設計討論的基礎。

類別	元件名稱	版本
前端	React + TypeScript + Vite + Tailwind CSS + shadcn/ui	React 18+
後端	NestJS (TypeScript)	Node.js 18 LTS+
區塊鏈	Solidity 0.8.17 + Hardhat + ethers.js	ERC-3643 (T-REX v4)
爬蟲	Python + Playwright + psycpg2	—
資料庫	PostgreSQL + TypeORM	正式環境 synchronize: false
容器化	Docker Compose (僅本機資料庫)	port 5433
部署	Render 雲端平台 (後端／區塊鏈)	—

表 6：系統技術組成與版本

在上述技術組成之上，本系統的安全設計依循「分層防護」原則：每一層都假設上一層可能失守，因此各自保留獨立的檢查點。以下依信任邊界由外而內說明四道防線。

第一道防線位於前端與後端之間的網路邊界。前端 React SPA 不保存任何業務判斷邏輯，所有請求皆須攜帶 JWT，由後端 Guard 驗證簽章與有效期後才解出角色資訊。此處另套用 ThrottlerModule 之全站速率限制（每 60 秒 100 次），並對 /api/login、/api/register、/api/kyc/:id/decrypt 等敏感端點另設更嚴格的個別上限，用以阻擋自動化腳本的暴力嘗試。

第二道防線位於後端 API 內部的授權層。系統依 INVESTOR、BUSINESS、TECHNICAL 三種角色劃分端點存取權限，除角色檢查外，對涉及個人資料的端點另實作物件層級授權，於 Service 層以 id 與 user_id 並列作為查詢條件，確保使用者無法藉由竄改路徑參數存取他人資料。KYC 證件解密端點則採角色檢查與 adminKey 雙重驗證。完整之角色權限對照詳見 5.1.4 節。

第三道防線位於資料層。使用者錢包私鑰以 AES-256-CBC 加密儲存並附帶隨機 IV，KYC 證件影像同樣以加密形式保存。交易寫入時對建案記錄取列級悲觀鎖以序列化同一標的之計價，並以 transactions.idempotency_key 之 UNIQUE 約束在資料庫層杜絕重複扣款，形成「應用層先檢查、資料庫層再兜底」的雙重保險。

第四道防線位於鏈上。ERC-3643 合約於每一筆 transfer 前主動查驗雙方 Identity 與 KYC Claim，任一項不符即 revert，因此即使後端服務中斷或遭穿透，非合規錢包仍無法在鏈上任意移轉代幣。系統並以鏈上餘額作為權威來源，透過對帳排程與 system_alerts 告警，使鏈上與鏈下的狀態落差可被偵測與修復。

在可維護性方面，後端採模組化架構（AuthModule、TransactionsModule、BlockchainModule 等），新增業務功能可透過新增獨立 Module 並注入既有 Service 完成，不需修改既有模組內部邏輯。資料庫結構變更統一透過 Migration 腳本管理，確保異動可追蹤、可回滾。上述四道防線之行為正確性，已透過 28 項故障注入測試（Fault Injection Test）驗證，涵蓋資料庫連線中斷、鏈上呼叫逾時、環境變數缺失、區塊鏈節點無法連線等情境，測試結果詳見 5.2 節與附錄 A。

4.2 系統的運行環境要求

作業系統：開發與測試環境為 Ubuntu 22.04 LTS（或 Windows 11 + WSL2），生產環境建議採用 Linux 發行版以確保 Docker 容器相容性。

後端執行環境：Node.js 18 LTS 以上版本、npm 9 以上。NestJS 專案以 TypeScript 5 撰寫。本機開發以 npm run start:dev 直接啟動，正式環境部署於 Render 雲端平台，不透過 Docker Compose 容器化。

資料庫：PostgreSQL 15 以上版本，透過 Docker Compose 容器化部署於 port 5433。

區塊鏈環境：Hardhat 本地節點（localhost:8545）模擬 EVM 鏈上環境。正式部署前需另評估測試鏈（Testnet）或私有鏈環境的 Gas 成本與節點穩定性。

前端瀏覽器：支援最新版 Chrome、Edge 或 Firefox，需啟用 JavaScript 與 Cookie。

爬蟲服務：Python 3.10 以上，需安裝 Playwright 對應版本的 Chromium 執行檔。

硬體建議：開發機建議至少 8GB RAM、10GB 可用磁碟空間，以同時執行 PostgreSQL 容器、Hardhat 節點與前後端開發伺服器。

5. 系統實作及實驗結果

5.1 系統功能的詳細描述

本節依技術端、業務端與投資人三種角色，分別描述其實際操作介面與各介面所提供之功能。各角色之使用情境與子使用案例已於 3.7 節使用案例圖呈現，本節則聚焦於介面實作層級的具體功能。

5.1.1 技術端介面

技術端登入後可存取四個主要畫面，分別為系統健康儀表板、告警中心、爬蟲監控面板與區塊鏈維運面板。

（一）系統健康儀表板：即時顯示資料庫查詢延遲（毫秒級）與伺服器 CPU 負載百分比，資料由 GET /api/system/performance 每次輪詢取得，並同步寫入 system_alerts 資料表形成歷史軌跡。畫面另提供全域交易暫停與恢復的切換開關，觸發後全平台下單端點立即拒絕新委託，用於價格異常或資料來源失效時的緊急處置。

（二）告警中心：顯示最近 30 筆系統事件，涵蓋 SYSTEM_HEALTH、ORDER_MATCH、CRAWLER_REPORT、SECURITY_AUDIT、BLOCKCHAIN 五種類型，並以嚴重程度（severity）標示顏色。技術端可由此判斷異常來自資料庫、撮合引擎、爬蟲或區塊鏈節點，作為後續排查的入口。

（三）爬蟲監控面板：顯示 crawler_metrics 之最後執行時間、失敗筆數、平均資料完整度與健康狀態（HEALTHY 或 WARNING）。當完整度下降或失敗筆數升高時，技術端可據以判斷 591 頁面結構是否異動，並決定是否暫停以爬蟲資料校準的報價。

（四）區塊鏈維運面板：提供合約狀態查詢、鏈上鏈下對帳觸發（GET /api/blockchain/reconcile）與差異修復（POST /api/blockchain/reconcile/repair），以及節點失憶後的合約重建入口。對帳結果會列出 BALANCE_MISMATCH 與 UNTRACKED_TRANSFER 兩類差異明細，供技術端逐筆確認後再執行修復。

（五）跨部門聊天室：與業務端共用之即時溝通頻道，訊息留存於 chat_logs 資料表，用於協調 KYC 異常、撥款疑義與風控暫停等需跨角色確認的事項。

5.1.2 業務端介面

業務端登入後可存取三個主要畫面，分別為用戶與 KYC 審核清單、投資人監理總覽與信託帳戶管理。

（一）用戶與 KYC 審核清單：列出所有已註冊用戶的帳號、KYC 狀態與錢包地址，並可篩選出待審核者。點選單筆用戶後可檢視其上傳之身分證件影像，該影像需以角色檢查加上 adminKey 雙重驗證才會解密顯示。業務端可執行核准或退件，核准後系統於後端同步完成鏈上 Identity 合約部署與 KYC Claim 簽發，使該用戶的錢包地址正式進入 ERC-3643 IdentityRegistry

的合規名單。退件則寫入通知與稽核紀錄，並允許用戶透過補件流程重新送審。

（二）白名單管理：可個別調整用戶之 `is_whitelisted` 狀態，狀態變更即時反映於鏈上 IdentityRegistry，用於暫時停止特定帳戶的交易資格。

（三）投資人監理總覽：彙整各投資人之持倉分布與交易紀錄，供業務端檢視是否有單一帳戶持倉過度集中之情形，作為風控判斷依據。

（四）信託帳戶與租金分潤管理：可查看各建案的信託帳戶餘額（`current_cash_balance`）與待入帳租金（`pending_rent_amount`），並執行租金批次分潤。執行後系統依代幣持有比例展開每位投資人的應得金額明細，成功者標記 PAID、失敗者標記 FAILED，同時自信託專戶扣除對應金額並寫入一筆流水紀錄。現階段信託帳戶為系統內部模擬與測試資料，尚未與銀行系統實際串接，帳戶餘額由業務端於後台手動輸入或由系統模擬產生。

5.1.3 投資人介面

投資人登入後可存取四個主要畫面，分別為房產代幣市場、建案詳情頁、下單表單與投資組合頁。

（一）房產代幣市場：以卡片形式列出可交易建案，顯示建案名稱、地址、代幣單價、當日漲跌幅與建案圖片。展示資料之種子資料為三筆，爬蟲另以 591 建案編號作為主鍵寫入，故執行爬蟲後會新增而非更新既有建案。以上資料均取自公開網站資訊，僅作系統展示與測試用途，不涉及實際代售或產權移轉。

（二）建案詳情頁：顯示完整地址、坪數、代幣總發行之量、募資總額與租金分潤週期，並提供 K 線價格走勢圖，顯示區間為近 31 日。為使展示階段之圖表具備連續走勢，資料庫另以造市指令注入標記為 `is_simulated` 的模擬成交與模擬掛單作為背景資料，撮合引擎已將該類資料排除於真實撮合之外。頁面下方另顯示即時掛單簿，呈現目前買賣雙方的委託價量分布。

（三）下單表單：提供市價單與限價單兩種委託方式。市價單依當下 AMM 報價立即成交，並在送出前預估含滑價之成交均價供投資人確認。限價單則由系統於資料庫層持續追蹤 AMM 報價，達成條件後於下一輪檢查週期（模擬延遲 5 秒）自動觸發成交。表單同時顯示目前持有代幣數量與可用現金餘額，並於超過持倉上限或現金不足時即時提示。未成交之限價單可於掛單清單中查詢與取消。

（四）投資組合頁：顯示各建案之持倉數量、成本、目前市值與損益統計，以及完整歷史交易紀錄，包含交易類型、數量、成交價、鏈上交易雜湊與交易狀態。

（五）通知中心：每筆成交或租金分潤撥款後，投資人會收到站內通知，可於通知中心檢視並標記為已讀。

5.1.4 角色權限設計（RBAC）

本系統依角色劃分功能存取權限，共定義投資人（INVESTOR）、業務端（BUSINESS）與技術端（TECHNICAL）三種角色，其定義對應 1.5 節「系統使用對象」。表 7 為以角色為欄、以功能

為列之權限矩陣，用以呈現三種角色的職務邊界與職務分離設計。

功能	投資人 INVESTOR	業務端 BUSINESS	技術端 TECHNICAL
註冊、登入與變更密碼	可	可	可
提交或補件 KYC 證件	可（限本人）	不可	不可
審核或退件 KYC 申請	不可	可	不可
解密檢視 KYC 證件影像	不可	可（另需 adminKey）	不可
管理用戶白名單狀態	不可	可	不可
瀏覽建案列表與建案詳情	可	可	可
查詢 AMM 報價與掛單簿	可	可	可
下單買賣代幣	可（限本人）	不可	不可
取消未成交掛單	可（限本人）	不可	不可
查看投資組合與交易紀錄	可（限本人）	可（監理檢視）	可（監理檢視）
接收與閱讀系統通知	可（限本人）	不可	不可
執行租金批次分潤與調整分潤週期	不可	可	可
檢視系統健康儀表板	不可	可	可
監控爬蟲狀態與資料品質	不可	可	可
接收爬蟲執行回報	不可	不可	可
檢視系統告警中心	不可	不可	可
暫停或恢復全平台交易（應用層）	不可	可	可
暫停或恢復鏈上合約（pause/unpause）	不可	不可	可
使用跨部門聊天室	不可	可	可
鏈上合約 setup 與身分登記	不可	不可	可
執行鏈上鏈下對帳與補寫修正	不可	不可	可

表 7：三角色功能權限矩陣

註：「可（限本人）」表示該角色僅能操作屬於自己的資料，系統於 Service 層以 id 與 user_id 並列作為查詢條件實作物件層級授權。

表 7 之權限矩陣於實作上係由 API 端點層級的授權規則落實。表 8 逐支列出後端 API 端點及其對應之模組、使用案例、允許角色與授權方式，使每一支 API 皆可回溯至具體的功能設計來源，而非僅依 API 路徑分類。其中 AuthModule、UsersModule、PropertiesModule、TransactionsModule、SystemModule、BlockchainModule 六項已繪於 3.4 節模組關係圖，PortfolioModule 與 NotificationsModule 則為投資組合查詢與站內通知之獨立模組，於該圖中未單獨繪出。

模組／對應使用案例	Endpoint	HTTP 方法	允許角色	授權方式
AuthModule 登入系統（JWT 驗證）	/api/login	POST	所有人（未登入）	無需驗證（登入前）， 速率限制詳見 2.3 節第 5 項
AuthModule 註冊帳號	/api/register	POST	所有人（未登入）	無需驗證（註冊）
UsersModule 檢視待審核用戶清單	/api/users	GET	BUSINESS	Controller 內角色檢查
UsersModule 管理用戶白名單狀態 （IdentityRegistry）	/api/users/:id/whitelist	PATCH	BUSINESS	Controller 內角色檢查
UsersModule 核准／駁回 KYC 申請	/api/users/:id/kyc	PATCH	BUSINESS	Controller 內角色檢查
UsersModule 解密檢視 KYC 證件	/api/kyc/:id/decrypt	POST	BUSINESS	Controller 內角色檢查 + adminKey 雙重驗證
PropertiesModule 瀏覽房產建案列表	/api/properties	GET	INVESTOR ／ BUSINESS ／ TECHNICAL （已登入）	JwtAuthGuard
PropertiesModule 查看建案詳情與 K 線價格走勢	/api/properties/:id/valuation-logs	GET	已登入者皆可	JwtAuthGuard
PropertiesModule 查看建案詳情與 K 線價格走勢	/api/properties/:id/kline	GET	已登入者皆可	JwtAuthGuard
PropertiesModule 執行租金批次分潤 （狀態 CALCULATED）	/api/properties/:id/payout	POST	BUSINESS ／ TECHNICAL	Controller 內角色檢查 + 金額上限驗證
PropertiesModule 執行租金批次分潤 （狀態 CALCULATED）	/api/properties/:id/payout-cycle	PATCH	BUSINESS ／ TECHNICAL	Controller 內角色檢查 + 週期天數 1～365 範圍驗證
TransactionsModule 下市價買單／設定 限價條件單／賣出 代幣	/api/transactions	POST	本人（投資人下單）	Controller 內比對 req.user.id === user_id

模組／對應使用案例	Endpoint	HTTP 方法	允許角色	授權方式
TransactionsModule 設定限價條件單 (檢視未成交掛單)	/api/pending-orders	GET	已登入皆可 (依 user_id 過濾)	JwtAuthGuard
TransactionsModule 取消未成交掛單	/api/pending-orders/:id/cancel	POST	本人	已確認安全：service 層資料庫查詢以 {id, user_id} 並列條件，非本人的掛單查詢會回傳 null 並擋下
TransactionsModule 查詢 AMM 即時報價	/api/orderbook/:propertyId	GET	已登入者皆可 (公開市場資料)	JwtAuthGuard
TransactionsModule 查詢 AMM 即時報價	/api/stats/:propertyId	GET	已登入者皆可	JwtAuthGuard
PortfolioModule 查看投資組合與持倉明細	/api/portfolio/:userId	GET	本人／ TECHNICAL／ BUSINESS	Controller 內比對
PortfolioModule 查詢歷史交易紀錄	/api/transactions/:userId	GET	本人／ TECHNICAL ／ BUSINESS	Controller 內比對
PortfolioModule 檢視投資人監理總覽	/api/oversight	GET	BUSINESS	Controller 內角色檢查
NotificationsModule 接收並閱讀系統通知	/api/notifications/:userId	GET	本人	Controller 內比對
NotificationsModule 接收並閱讀系統通知	/api/notifications/:userId/read	PATCH	本人	Controller 內比對
SystemModule 檢視系統健康儀表板 (DB 延遲／CPU 負載)	/api/health	GET	所有人	無需驗證 (健康檢查)
SystemModule 檢視系統健康儀表板 (DB 延遲／CPU 負載)	/api/system/performance	GET	TECHNICAL ／ BUSINESS	JwtAuthGuard + 角色檢查

模組／對應使用案例	Endpoint	HTTP 方法	允許角色	授權方式
SystemModule 監控爬蟲狀態與資料品質	/api/system/crawler-status	GET	TECHNICAL ／ BUSINESS	JwtAuthGuard + 角色檢查
SystemModule 接收爬蟲執行回報	/api/system/crawler-report	POST	TECHNICAL	角色檢查，限縮為 TECHNICAL 專用
SystemModule 暫停全平台交易／ 恢復交易	/api/system/state	GET	已登入者皆可	JwtAuthGuard
SystemModule 暫停全平台交易／ 恢復交易	/api/system/state	POST	TECHNICAL ／ BUSINESS	Controller 內角色檢查
SystemModule 使用跨部門聊天室	/api/chat	GET	TECHNICAL ／ BUSINESS	Controller 內角色檢查
SystemModule 使用跨部門聊天室	/api/chat	POST	TECHNICAL ／ BUSINESS	Controller 內角色檢查
SystemModule 檢視系統告警中心 (system_alerts)	/api/system-alerts	GET	TECHNICAL	Controller 內角色檢查
BlockchainModule 節點狀態復原 (autoRecoverNodeState)	/api/blockchain/status	GET	已登入者皆可	JwtAuthGuard (曾為公開，已修正需登入)
BlockchainModule 鏈上合約 setup 與白 名單調整	/api/blockchain/setup	POST	TECHNICAL	Controller 內角色檢查
BlockchainModule 鏈上登記 Identity 與 簽發 KYC Claim	/api/blockchain/register-user/:userId	POST	TECHNICAL	Controller 內角色檢查
BlockchainModule 執行鏈上鏈下對帳 與人工補寫修正	/api/blockchain/reconcile	GET	TECHNICAL	Controller 內角色檢查
BlockchainModule 執行鏈上鏈下對帳 與人工補寫修正	/api/blockchain/reconcile/repair	POST	TECHNICAL	Controller 內角色檢查
BlockchainModule 查看建案詳情與 K 線價格走勢	/api/metadata	GET	已登入者皆可	JwtAuthGuard

模組／對應使用案例	Endpoint	HTTP 方法	允許角色	授權方式
AuthModule 登入系統（JWT 驗證）（延伸：變更密碼）	/api/auth/change-password	POST	本人（已登入）	JwtAuthGuard + 舊密碼驗證
UsersModule 查詢 KYC 審核狀態	/api/users/profile/me	GET	本人（已登入）	JwtAuthGuard，僅回傳自身資料
UsersModule 核准／駁回 KYC 申請	/api/users/:id/kyc/reject	PATCH	BUSINESS	Controller 內角色檢查。退件並寫入通知與稽核紀錄
UsersModule 上傳 KYC 身分證件	/api/kyc/resubmit	POST	本人（已登入）	JwtAuthGuard + 副檔名白名單、Magic Bytes 與 5MB 大小驗證
BlockchainModule 暫停全平台交易／恢復交易	/api/blockchain/pause-toggle	POST	TECHNICAL	Controller 內角色檢查。為鏈上合約 pause／unpause 之實際入口

表 8：API 端點授權對照表

註：表中之模組名稱取自後端 NestJS 模組結構（主要模組見 3.4 節模組關係圖），使用案例名稱取自 3.7 節三張使用案例圖，角色定義則取自 1.5 節「系統使用對象」。後端 Guard 依 JWT 內嵌之角色資訊進行存取控制，若使用者嘗試呼叫超出其角色權限之 API，會回傳權限不足錯誤（403 Forbidden）。

本系統目前未實作提出、核准、執行三者分離或雙人核准機制，高風險操作（如白名單調整、鏈上暫停、資產強制轉移）僅需單一 BUSINESS 或 TECHNICAL 角色即可完成，屬原型階段已知限制，相關說明詳見 2.3.7 節。

註（forcedTransfer 補充說明）：forcedTransfer 並非獨立對外之 API 端點，而是封裝於投資人賣出代幣流程內部，父級端點為表 8 之 POST /api/transactions（txType = "SELL"）。允許角色為 INVESTOR 本人，Controller 內比對 req.user.id 與 user_id，僅能操作自己的持倉。因投資人錢包內無 ETH 可支付鏈上 Gas 費，後端於 transactions.service.ts 之 runTrade() 判斷 txType 為 SELL 時，呼叫 blockchain.service.ts 之 executeOnChainSell() 執行 token.forcedTransfer(userWalletAddress, adminAddress, amountWei)，由平台管理員錢包代付 Gas 費，將代幣收回平台池。智能合約層為 SimpleRWA.sol（繼承自 ERC-3643 標準之 Token.sol）。此設計使 forcedTransfer 成為一般投資人賣出交易之必經路徑，而非僅限法令命令或資產回復等例外情境使用，與表 8 中 POST /api/transactions 之角色限制一致，未額外開放獨立呼叫權限。

5.2 實驗數據

本節之驗證工作依測試層級分為四類，依序為單元測試與故障注入測試、系統整合測試、使用者驗收測試，以及效能與負載測試。四類測試合計 89 項自動化測試全數通過，另有人工操作驗證與效能量測，分述如下。

5.2.1 單元測試與故障注入測試

本層級聚焦於單一函式或單一服務在正常與異常輸入下的行為正確性，以 Jest（後端）與 Hardhat（智能合約）撰寫，合計 74 項，其中後端單元、負向、併發與故障注入 42 項、智能合約 20 項、鏈上鏈下狀態一致性故障注入 12 項。測試涵蓋範圍如下。

（一）AMM 定價公式驗證：以測試自行計算之公式結果比對服務實際寫入資料庫之成交價，驗證滑價方向與數值正確性（transactions.service.spec.ts）。

（二）資料庫交易失敗恢復：模擬資料庫連線於 commit 階段中斷，驗證系統是否正確執行 rollback 並釋放連線，避免留下半寫入的髒資料或連線洩漏（transactions/fault-injection.spec.ts）。

（三）鏈上呼叫逾時處理：模擬區塊鏈 RPC 呼叫逾時，驗證資料庫端交易仍正確完成並標記為 CHAIN_FAILED 狀態，而非讓整筆請求失敗（transactions/fault-injection.spec.ts）。

（四）必要環境變數缺失防護：驗證系統於缺少關鍵環境變數（如 SUPABASE_SERVICE_KEY、IMAGE_ENCRYPTION_KEY）時，可於服務啟動當下即 fail-fast 拋出明確錯誤，而非延遲到實際呼叫 API 時才發生未預期錯誤（auth/fault-injection.spec.ts）。

（五）區塊鏈節點故障處理：模擬 RPC 節點無法連線，驗證系統各項功能（狀態查詢、部署、對帳）皆能明確拒絕或回報錯誤，而非讓服務整體崩潰或無回應（blockchain/fault-injection.spec.ts）。

（六）節點失憶的自動重建判斷：針對 autoRecoverNodeState()（每 30 秒排程檢查合約狀態）與 isConfigStale() 的判斷分支新增 4 項測試，涵蓋節點不可達時應跳過重建、RPC 瞬斷不應誤判為合約消失、getCode 確實回傳 0x 時才觸發重建，以及重建進行中應被 isRecovering 旗標擋下避免重複部署。此為系統中風險最高的自動化路徑，誤判將導致合約重新部署、鏈上餘額歸零。其中第 2 筆測試另以突變測試（Mutation Testing）反向驗證：刻意將判斷邏輯改錯後重跑，確認測試會如預期失敗，驗證後還原程式碼，證實測試確實守住行為而非恰好通過。第 4 筆因權限限制未能做同等驗證，為誠實揭露之已知缺口。

（七）鏈上與鏈下狀態一致性五情境測試：針對服務重啟、重複事件、Nonce 衝突、區塊重組、通知失敗五種情境，以 fault-injection-integrity.spec.ts 撰寫 12 項測試並全數通過。其中「重複事件」與「區塊重組」於本系統架構下並無字面對應機制，因全專案未使用鏈上事件訂閱（無 contract.on、provider.on 或 WebSocket listener），reconcile() 採全量比對式讀取而非持續監聽，.wait() 亦均未帶確認數參數，未實作 reorg 偵測與回捲邏輯。因此測試改針對等價風險驗證：以「連續執行兩次對帳是否重複累計不一致」驗證重複事件風險，結果顯示對帳為無狀態全量比對，天生冪等。以「reorg 後鏈上查無事件、資料庫仍有餘額時對帳是否偵測」驗證區塊重組風險，結果正確偵測為 BALANCE_MISMATCH 並寫入 system_alerts。Nonce 衝突與通知失敗則直接測試，兩者經確認後皆已修正：前者於失敗路徑加入 resetNonceIfDesynced() 清除 NonceManager 快取，後者將通知寫入移至 commitTransaction() 之後並改用獨立 repository，避

免純通知層故障回滾已上鏈交易。

(八) 服務重啟情境測出一項尚未修正的已知風險：鏈上轉帳呼叫位於 DB commit 之前，若程序恰於「鏈上已成功、DB 尚未 commit」的空窗中斷，重啟後重新結算會導致鏈上轉帳被重複執行（實測 executeOnChainBuy 遭呼叫 2 次，使用者僅入帳 1 次）。現況安全網僅有 reconcile() 事後偵測而非事前防止，修正需將鏈上呼叫前置一筆「已送出」意圖紀錄並改為兩階段提交，屬架構層面改動，現以測試釘住現況行為並標記為已知風險。

已知限制：併發測試發現 idempotency_key 檢查與交易寫入之間原存在 TOCTOU 競態條件，經追查確認資料庫層級 UNIQUE 約束足以防止資料不一致，惟原本市價單路徑對此類衝突的錯誤處理不完整（會回傳未預期的 500 而非明確的 400 訊息），現已修正並補上對應測試。另針對 AMM 讀取建案資料時的 lost update 風險，已於資料庫交易中加入 pessimistic_write 排他鎖，確保同一建案交易序列化執行。

5.2.2 系統整合測試 (System Integration Test)

本層級聚焦於跨模組串接後的端到端行為，特別是身分驗證、角色授權與物件層級授權三者串接是否正確，以 Jest 之 e2e 測試撰寫，合計 15 項全數通過（integration-auth.e2e-spec.ts 等，完整輸出詳見附錄 A）。測試涵蓋以下情境。

(一) 未攜帶 JWT 或攜帶過期 JWT 呼叫受保護端點時，應回傳 401 Unauthorized。

(二) 以投資人身分呼叫僅限業務端或技術端之端點（如 KYC 審核、爬蟲回報、系統告警）時，應回傳 403 Forbidden。

(三) 物件層級授權驗證：當投資人嘗試查詢非本人的投資組合或交易紀錄（user_id 與 token 內的 id 不一致）時，系統正確回傳 403 Forbidden，確實落實最小權限原則。此項對應 5.1.4 節表 7 中「可（限本人）」之權限定義。

(四) 取消非本人掛單時，Service 層以 id 與 user_id 並列之查詢條件回傳 null 並擋下請求，確認 Controller 層雖未見明顯檢查亦非實際漏洞。

(五) KYC 核准後鏈上 Identity 部署與 Claim 簽發之串接驗證，包含鏈上失敗時資料庫狀態不回捲、並可由技術端補登之行為。

5.2.3 使用者驗收測試 (User Acceptance Test)

本層級以實際操作介面的方式，驗證系統是否滿足 1.4 節所列九項功能之使用者需求，由團隊成員分別扮演三種角色於開發環境執行，驗收項目與結果如下。

(一) 交易撮合驗收：以投資人身分下市價單，確認依 AMM 報價立即成交且成交均價含滑價。下限價單後確認系統每 5 秒檢查一次觸發條件，於條件達成後自動成交，驗收通過。

(二) 風控機制驗收：以投資人身分持續買入同一建案至超過 5% 持倉上限，確認系統拒絕該次委託並回傳錯誤訊息，同時於 system_alerts 資料表寫入一筆告警紀錄供技術端於監控頁面查

核，驗收通過。

（三）KYC 審核驗收：以投資人身分上傳證件、業務端執行核准，確認用戶狀態轉為 VERIFIED、鏈上 IdentityRegistry 完成登記，且該用戶隨即可下單。另測試退件流程與補件重審，驗收通過。

（四）租金分潤驗收：以業務端身分對單一建案執行批次分潤，確認系統依持有比例展開每位投資人明細、標記 PAID、扣除信託專戶餘額並寫入流水紀錄，投資人端亦收到站內通知，驗收通過。

（五）系統監控與全域暫停驗收：以技術端身分觸發全域交易暫停，確認投資人端下單即被拒絕，恢復後於節流期內持倉上限收緊為 1%，驗收通過。

（六）跨部門協作驗收：以業務端與技術端分別登入，確認聊天室訊息即時送達並留存於 chat_logs，驗收通過。

未通過或無法於本階段驗收之項目為法幣交割、產權法定登記與外部機構 KYC 查核三項，因其均未串接外部系統，僅能以資料表模擬，相關原因詳見 5.5.2 節。

5.2.4 效能與負載測試

效能測試分為資料庫查詢、區塊鏈 RPC 與後端 API 負載三部分。資料庫查詢效能測試於 5 萬筆資料規模下，查詢執行時間為 3.677 ms，索引運作正常未退化為全表掃描。區塊鏈 RPC 效能測試顯示雲端節點穩定狀態平均延遲約 105 ms，20 併發下吞吐量為 215.35 req/s 且零錯誤，原始輸出詳見附錄 B。

後端 API 負載測試以 autocannon 對兩支代表性端點進行 20 併發連線、持續 20 秒之測試，兩支端點分別為 K 線查詢（GET /api/properties/:id/kline）與掛單簿查詢（GET /api/orderbook/:id）。測試區分「保留 Throttler 限流（100 次／60 秒，全站套用）」與「暫時解除限流」兩種情境，藉以區分限流本身的攔截效果與系統實際處理能力，原始輸出詳見附錄 C。

保留限流情境之結果如表 9 所示。掛單簿查詢平均延遲 3.95 ms 且零錯誤，限流在請求觸及業務邏輯前即攔下，系統幾乎不耗資源。K 線查詢平均延遲則達 4071.69 ms，79,000 筆請求中有 16,000 筆（約 20%）於連線層失敗，顯示限流雖能保護業務邏輯不被打爆，但無法阻止重運算端點於高併發下在連線層崩解。表中「非 2xx」一欄即為被限流機制以 429 拒絕的請求數，其數量遠高於實際處理的 2xx 請求數，可見表面上的高吞吐量主要來自快速拒絕。

項目	K 線查詢（保留限流）	掛單簿查詢（保留限流）
平均延遲	4071.69 ms	3.95 ms
p99 延遲	12,617 ms	4 ms
平均吞吐量	3,155.15 req/s	4,400.7 req/s
2xx 成功	200	100
非 2xx（429 限流拒絕）	62,900	87,901
連線層錯誤	16,000	0

表 9：後端 API 負載測試結果（保留限流情境）

解除限流情境之結果如表 10 所示。吞吐量從限流情境下的每秒數千次驟降至 15 至 31 req/s，證實前述高吞吐量主要來自 429 的快速拒絕而非系統真實處理能力。此情境下無任何錯誤，顯示系統本身穩定，瓶頸在於單筆請求延遲過高。

項目	K 線查詢（解除限流）	掛單簿查詢（解除限流）
平均延遲	632.88 ms	1277.33 ms
p99 延遲	2370 ms	2998 ms
平均吞吐量	31.25 req/s	15.15 req/s
錯誤/逾時	0/0	0/0

表 10：後端 API 負載測試結果（解除限流情境）

經以 EXPLAIN ANALYZE、序列請求基線測試、純 TCP 連線測試及連線池設定檢查逐步排除，判斷延遲瓶頸主要來自三項疊加因素。其一為本機至 Supabase 雲端資料庫（ap-southeast-2 雪梨機房）之跨國網路延遲，純 TCP 連線測試顯示穩定狀態下單次連線即需 530 至 560 ms。其二為 PgBouncer Transaction Pooling 模式下連線無法有效重用。其三為 TypeORM 未設定連線池上限（預設 10），於 20 併發下造成排隊等待。三者疊加導致 K 線查詢 p99 延遲達 2.4 至 3.0 秒。此結果已作為後續優化方向：短期導入 Redis 快取層攔截高頻重複查詢，長期將 OHLC 聚合邏輯下放至資料庫端，並評估應用伺服器與資料庫同區域部署。

5.3 實作成果的評估

本節以期初規劃為基準，逐項檢視本專題是否如期、如實、如質完成。表 11 就功能範圍、時程、技術、開發方法、品質與成本六個構面，對照當初預計項目與實際完成情形，未達成者一併說明原因。

構面	期初規劃	實際完成情形	達成評估與未達成原因
功能範圍	1.4 節所列九項功能模組	九項全部實作完成，並可於開發環境完整操作展示	如實完成
時程	22 個 Sprint，自探索期、設計期、迭代開發期至整合交付期共四階段	四階段均依甘特圖推進，程式碼、測試與文件於期末產出，各 Sprint 之議題紀錄留存於 GitHub Issues	如期完成
技術	React + Vite、NestJS、PostgreSQL、ERC-3643 (T-REX v4)、Python Playwright	全數採用並實際落地，未因技術困難而中途更換方案	如實完成
開發方法	兩週一 Sprint 之敏捷迭代，並以 GitHub Issues 追蹤待辦	確實執行，會議紀錄與待辦清單可回溯	如期完成
品質（功能正確性）	核心交易與風控邏輯須具備自動化測試	89 項自動化測試全數通過，其中 28 項為故障注入測試	如實完成
品質（效能）	未列入本階段驗證目標	已完成資料庫查詢、區塊鏈 RPC 與 API 負載測試	部分未達。K 線端點於 20 併發下平均延遲 4 秒，原因為跨國網路延遲、PgBouncer 連線無法重用與 TypeORM 連線池未調校三者疊加，非業務邏輯缺陷
法幣交割與信託金流	期初列為選擇性目標	未串接，僅以 bank_trust_accounts 與 bank_trust_transactions 兩張資料表模擬	未完成。原因為學生專題無法取得銀行金流測試環境與相關法遵資格
產權法定登記	期初列為選擇性目標	未串接，僅於 property_ownership 保留對接欄位	未完成。原因為內政部地政系統無公開介接管道
外部機構 KYC 查核	期初列為選擇性目標	未串接，改由業務端以加密證件影像人工審核	未完成。原因為第三方 KYC 服務需付費商用授權
成本	以免費或教育方案為限，不產生額外費用	使用 Render 與 Supabase 免費方案、Hardhat 本地節點，開發期間未產生實際支出	如實控制於預算內

表 11：期初規劃與實際完成情形對照表

整體而言，本專題在功能範圍、時程、技術選用、開發方法與功能正確性五個構面均達成期初規劃。效能構面部分未達，瓶頸經測試確認來自部署環境的網路與連線池設定而非業務邏輯本身。法幣交割、產權登記與外部 KYC 查核三項未完成，原因均為外部資源取得限制，非技術可行性問題，此三項已於期初即列為選擇性目標，並自始於文件中揭露為技術原型的邊界。

以下就五大模組分述實際完成內容。

前端：已完成投資人、業務端、技術端三種角色的完整操作介面，採用 React + TypeScript 搭配 Tailwind CSS 與 shadcn/ui 元件庫，涵蓋房產代幣市場瀏覽、K 線走勢圖、委託下單、投資組合查詢、系統監控儀表板、協作聊天室等頁面，介面風格統一、互動流程完整。

後端：NestJS API 服務已完成依業務領域切分的模組化架構，涵蓋 AuthModule、TransactionsModule、PropertiesModule、BlockchainModule、SystemModule 等模組，委託下單、動態風控（持倉上限、持倉額度收緊、緊急暫停）等邏輯已實作並通過單元測試與故障注入測試驗證（詳見 5.2 節測試結果）。JWT 三角色鑑權機制已實作並可運作，其涵蓋範圍與限制詳見 2.3 節安全性說明。

區塊鏈：ERC-3643 (T-REX) 合規代幣合約已於 Hardhat 測試節點完成部署，IdentityRegistry 白名單機制與 KYC Claim 簽發流程已可實際觸發並驗證，代幣轉帳的鏈上合規檢查已能正確攔截未通過白名單的交易對象，落實「合規驗證內嵌於代幣本身」的設計方向。惟大規模併發情境下之邊界行為（如本文 5.2 節所述之 idempotency 競態條件限制）尚待後續優化。

爬蟲與動態定價：Python + Playwright 爬蟲已完成自動擷取 591 新建案平台的功能，並已完成 AMM 動態定價邏輯，作為房產代幣次級市場交易的即時報價依據。

資料庫：共 16 張資料表已全數完成建置，架構符合 3NF 正規化原則，並已將 K 線歷史行情、資金信託流水帳、租金分潤明細等資料正確拆解為可追溯、可對帳的資料表結構，完整支援前述所有業務模組的資料存取需求。

5.4 與其他相關技術應用的比較，包括優缺點

本節先整理本系統相較於既有技術方案的優點，再整理其缺點。

就優點而言，共有四項。第一，合規驗證內嵌於代幣本身。相較於傳統 REITs 系統多以中心化資料庫搭配人工對帳流程管理股權與收益分配，本系統將合規驗證邏輯內嵌於 ERC-3643 智能合約層，交易雙方的白名單資格於鏈上即時驗證，不需仰賴人工複核每一筆交易，理論上可降低合規稽核的人力成本，且鏈上紀錄具備不可篡改性，稽核軌跡更為透明。第二，合規邏輯不依賴鏈下系統的持續運作。相較於一般採用 ERC-20 標準的 RWA 代幣化平台，即使後端服務中斷或遭穿透，非合規錢包仍無法在鏈上任意移轉代幣。第三，交易明細可用一般 SQL 查詢。相較於完全鏈上化的系統，混合式架構使前端儀表板與報表產出的開發效率較高。第四，流動性與定價即時性較高。以 AMM 演算法結合 591 爬蟲資料提供全年無休之次級市場報價，不受證券交易時段限制。

就缺點而言，同樣有四項，且多數與上述優點互為代價。第一，智能合約複雜度較高。ERC-3643 的合約群較單純的 ERC-20 龐大，開發與稽核成本相對增加，且每筆轉帳需額外執行鏈上合規檢查，在 Gas 成本與交易確認速度上略遜於單純的 ERC-20 轉帳。第二，多了一層對帳負擔。混合式架構需維護鏈上鏈下對帳機制，相較於完全鏈上化的系統增加開發與維運成本，且對帳異常時仍需人工介入。第三，AMM 定價高度仰賴資金池深度。募資初期流動性不足時容易產生較大滑價，早期投資人將承擔較高的隱含交易成本。第四，外部資料存在時效落差風險。591 平台資料非即時更新，代幣參考價格可能與實際市場行情脫節。

綜合而言，本系統以較高的合約複雜度與對帳維運成本，換取合規檢查的不可繞過性與次級市場的即時流動性，此為混合式 (Hybrid On-chain/Off-chain) 架構相對於純鏈上系統與傳統 REITs 系統的核心權衡取捨。上述四項缺點對應之因應方式與後續規劃，詳見 5.5 節。

5.5 遭遇的問題和挑戰

本節將本專題所面臨之事項分為兩類。第一類為「問題」，指開發過程中已實際發生、且系統現行設計尚未能完全解決的技術缺口。第二類為「挑戰」，指非本團隊技術能力所能單獨克服、需依賴外部資源或後續產品化階段才能處理的限制。兩類分述於 5.5.1 與 5.5.2 節，並於節末以 SWOT 分析歸納本專題的整體處境。

5.5.1 遭遇的問題

（一）外部資料（591）的時效落差：591 平台資料非即時更新，若代幣參考價格與實際市場行情產生落差，可能導致投資人依過時價格下單而承擔非預期損益。目前之因應方式為爬蟲定期排程更新，並由 `crawler_metrics` 記錄最後執行時間供技術端掌握資料新鮮度，惟尚未實作依落差程度自動調整報價權重之機制。

（二）外部資料的異常值與可信度風險：若 591 頁面結構異動或回傳異常數值（如坪數為 0 或價格為負），可能污染代幣定價依據，影響投資人交易決策之公平性。目前僅以資料完整度（`integrity`）指標反映欄位缺漏情形，尚未針對數值合理性進行正數檢核或範圍檢查，亦未實作「未通過驗證即不寫入」之攔截閘門，未來應於爬蟲層加入異常值攔截規則。

（三）募資初期流動性不足時的 AMM 滑價問題：建案剛上架時代幣總發行量與募資總額構成之資金池儲備較小，投資人買入時容易產生較大滑價，可能使早期投資人承擔較高之隱含交易成本，降低參與意願。目前之因應方式為以資料庫訂單簿之 P2P 撮合優先於 AMM 成交，以價格優先、時間優先原則降低對 AMM 資金池之依賴，惟尚未針對初期流動性另行設計做市誘因機制。

（四）定價權重參數為人為設定，缺乏實證數據校準：AMM 定價公式中 591 行情與鏈上交易動能之權重比例目前由開發團隊依經驗設定，若權重設定不當，可能使定價偏離真實市場行情，影響投資人對代幣價值之判斷。此問題目前尚無實證數據驗證最適權重組合，待累積足夠交易資料後，可透過歷史成交價與 591 行情之相關性分析進行校準。

（五）服務重啟空窗期可能造成鏈上轉帳重複執行：故障注入測試發現，鏈上轉帳呼叫位於資料庫 `commit` 之前，服務若於「鏈上已成功、資料庫尚未 `commit`」的空窗中斷重啟，重新結算會導致鏈上轉帳被重複執行。現況安全網僅有 `reconcile()` 事後偵測，尚未事前防止，修正需改為兩階段提交，屬架構層面改動。

5.5.2 面臨的挑戰

（一）大規模併發與效能基準測試尚未執行：本專題現階段聚焦於核心業務邏輯之正確性驗證，若未經高併發壓力測試即投入實際營運，可能在多位投資人同時對同一建案下單時，因資料庫列級悲觀鎖競爭而出現撮合延遲或請求逾時，影響成交成功率與使用者體驗。目前之因應方式為以 89 項自動化測試確保單筆交易路徑之正確性，並於 5.2.4 節以負載測試觀察 API 限流機制之實際效果，惟長時間、多使用者之壓力測試與效能基準比較須待下一階段產品化再行處

理。

(二) 法幣交割與信託金流未串接：本系統之銀行信託帳戶餘額與租金撥款金額皆為系統內部數值模擬，未串接真實銀行法幣金流 API，因此租金分潤僅能於資料庫標記為 PAID，無法產生可供投資人以銀行對帳單核對之實際資金移轉憑證，也無法驗證信託專戶餘額是否足以支應本次撥款。目前之因應方式為將撥款流程拆為「計算應分配金額」與「實際銀行撥款」兩個階段，並以 bank_trust_accounts 與 bank_trust_transactions 兩張資料表完整保留專戶餘額與流水紀錄，使未來串接銀行 API 時僅需替換撥款階段之實作，分潤計算邏輯不需更動。

(三) 產權法定登記未串接：本系統未串接內政部地政線上登記系統，代幣僅代表平台內部記錄之收益權益份額，不具備實體土地產權之法律物權效力，若投資人誤將代幣認知為產權憑證，將產生權利範圍之認知落差與潛在爭議。目前之因應方式為於 property_ownership 資料表保留 legal_document_ref 與 entity_type 欄位，作為未來對接法定登記文件之介面，並於系統介面與 2.3 節中明確揭露代幣不具物權效力之事實。

(四) 外部機構 KYC 查核未串接：本系統未串接外購 KYC 身分查核服務商，改由業務端以加密後之證件影像進行內部人工審核。隨投資人數量增加，人工審核量能可能成為系統瓶頸，且審核標準取決於個別承辦人員之判斷，缺乏一致的查核基準與國際制裁名單比對能力。目前之因應方式為以 AES-256-CBC 加密儲存證件影像、限制僅 BUSINESS 角色搭配 adminKey 雙重驗證方可解密，並將審核結果與鏈上 Identity 合約發行綁定為單一流程以確保紀錄可追溯，惟自動化查核與第三方 AML 名單比對仍待未來導入。

(五) 法遵資格與監理定位未確立：實際發行與交易是否符合臺灣證券、信託、不動產與洗錢防制等相關法規，須另行進行法遵審查。本專題為技術原型，不具備發行主體資格，此為學生專題無法自行克服之外部條件。

5.5.3 SWOT 分析

綜合上述問題與挑戰，並對照 1.6 節所述之系統特色，本專題之整體處境以 SWOT 分析歸納如表 12 所示。

內部因素	外部因素
優勢 (Strengths) 1. 合規檢查內嵌於 ERC-3643 合約層，不因鏈下服務中斷而失效。 2. 具備完整的鏈上鏈下對帳與故障注入測試，89 項自動化測試全數通過。 3. 資料庫符合第三正規化，租金分潤以批次與明細二層式展開，可追溯可對帳。 4. 兩階段撮合設計兼顧真實對手盤成交與隨時可成交的流動性保證。	機會 (Opportunities) 1. RWA 代幣化為國際金融科技明確趨勢，已有 AspenCoin 等成功先例。 2. 金管會已就證券型代幣發行 (STO) 提出相關規範說明，制度環境逐步明朗。 3. 政府實價登錄等公開資料可作為未來多源鑑價的額外校準來源。 4. 小資族對低門檻不動產投資存在明確需求，市場空間尚未被充分滿足。

內部因素	外部因素
劣勢 (Weaknesses) 1. AMM 定價權重為人為設定，缺乏實證數據校準。 2. 募資初期資金池深度不足時滑價偏高。 3. 金鑰以環境變數存放，未導入 KMS 或 HSM，且未實作雙人核准機制。 4. K 線等重運算端點於 20 併發下延遲過高，尚未進行效能調校。 5. 服務重啟空窗期仍可能重複執行鏈上轉帳，僅能事後對帳偵測。	威脅 (Threats) 1. 591 為第三方平台，頁面結構異動或反爬蟲政策調整將直接影響資料來源。 2. 法幣交割、產權登記與外部 KYC 查核皆須外部機構配合，非團隊可自行取得。 3. 若無正式法遵資格，代幣與產權之權利落差可能引發認知爭議。 4. 公鏈部署後 Gas 成本波動將影響交易體驗與 AMM 定價頻率。

表 12：本專題 SWOT 分析

就 SWOT 之交叉策略而言，本專題應以優勢承接機會，將已驗證的合約層合規機制與對帳架構，作為未來銜接 STO 法遵環境的技術基礎。針對劣勢與威脅，短期應優先補足金鑰管理與效能調校兩項可由團隊自行控制的項目，中長期則須以多源資料來源分散對 591 單一平台的依賴，並待取得正式法遵資格後才推進真實金流串接，相關時程規劃詳見 6.3 節。

6. 結論及未來發展

6.1 總結本專題的主要貢獻

本專題完成了一套結合 ERC-3643 合規代幣標準、動態 AMM 定價機制與自動化租金分潤設計的 RWA 房產代幣化系統原型。以下四項貢獻並非陳述「做了什麼」，而是說明各項成果對使用者、對系統可靠度與對後續研究所帶來的價值。

（一）正規化資料庫設計帶來的價值在於「可稽核」。K 線歷史行情、資金信託流水帳與租金分潤明細，原本在多數原型中容易被簡化為單一快照欄位，一旦數值有誤便無從回溯。本專題將其拆解為符合第三正規化的資料表結構後，每一次估值、每一筆金流與每一位投資人的分潤金額都留下獨立且可追溯的紀錄，使系統具備事後對帳與爭議釐清的能力，這正是金融性質系統與一般展示型原型的分野。

（二）以 ERC-3643 智能合約取代傳統鏈下白名單管理，其價值在於「合規不因系統故障而失效」。傳統做法將白名單比對放在後端資料庫，後端一旦中斷或遭穿透，代幣仍可在公開鏈上被任意移轉，形成洗錢防制與客戶身分查核的缺口。本專題將驗證邏輯下沉至合約層後，合規檢查成為代幣移轉的前置條件而非附加步驟，使系統的合規保證不再依賴任何單一伺服器的可用性，對監理單位而言也提供了可公開驗證的稽核軌跡。

（三）「批次+明細」二層式分潤展開的價值在於「讓小額投資人也能驗證自己拿到多少」。單一房產的租金收益若僅記錄總額，投資人無從確認自身應得金額是否正確。本專題將批次與明細分離後，每位投資人都能看到自己的持有比例、應得金額與撥款狀態，使收益分配從平台的單方宣告轉為可逐筆核對的紀錄，這是普惠金融中「透明」二字的具體落實。需說明的是，本專題完成之範圍為分潤金額之計算與展開，實際銀行撥款仍屬模擬階段。

（四）三角色分權架構與動態風控機制的價值在於「降低單點濫權與系統性風險」。技術端、業務端與投資人的權限邊界明確劃分後，KYC 審核與系統維運分屬不同角色，避免單一人員同時掌握身分核准與資產移轉能力。持倉上限、持倉額度收緊與全域交易暫停三項風控機制，則使系統在資料來源失效或價格異常時具備可立即啟動的煞車，而非只能被動等待人工發現。

6.1.1 本節小結

綜合上述四項，本專題的整體價值可從三個層面理解。就金融科技概念的實作而言，資產份額碎化、混合式 AMM 動態定價與租金收益批次分潤三項概念，在本專題中被落實為可運行且可測試的技術原型，而非僅止於文獻探討，後續研究者得以在此基礎上直接驗證改良方案。就對小資族的意義而言，每枚數千元至數萬元的低單價設計搭配次級市場流動性機制，實質降低了不動產投資的資本門檻，回應 1.2 節所述之普惠金融目的。就團隊學習而言，成員透過本專題累積了 ERC-3643 權限型代幣合約開發、鏈上鏈下狀態一致性對帳設計、AMM 定價演算法，以及多角色 RBAC 權限系統設計等跨領域技術整合經驗，並在故障注入測試的過程中，理解到金融系統的可靠度來自對異常路徑的處理而非正常路徑的完成度。

6.2 對未來研究或發展的建議

6.2.1 AI 智慧資產推薦配置（延伸自房產代幣市場與投資組合管理）

將投資人目前手頭的閒置資金或目標年化收益（APY）作為輸入，系統可自動推薦適合入手的不動產代幣份額，以及未來資產升級可考慮的優質高科技廠辦標的。此功能可延伸自現有之投資組合管理模組，其研究重點在於如何在資料量有限的情況下建立可解釋的推薦邏輯，避免形成無法向投資人說明理由的黑箱建議。

6.2.2 多源異質數據動態鑑價（延伸自房產估值爬蟲與 AMM 動態定價）

優化 AMM 價格修正曲線，除現有的混合式動態定價模型外，可進一步納入區域都市計畫與大眾運輸建設數據，協助判斷哪些碎化房產具有更高的潛在增值空間，作為買賣雙方定價決策之參考。此方向亦可直接改善 5.5.1 節所述「定價權重缺乏實證校準」之問題。

6.2.3 房產投資願景清單（延伸自房產代幣市場）

允許投資人將想要的特定地段、建案類型（如商辦或住宅）以及期望的每幣單價加入清單，當市場上出現符合條件的次級市場限價單或新募資物件時，系統即時通知該投資人。此功能可提升次級市場的媒合效率，間接改善募資初期流動性不足的問題。

6.2.4 實價登錄資料整合（延伸自房產估值爬蟲）

現階段 AMM 定價僅結合 591 新建案平台爬蟲資料，未來可進一步串接政府實價登錄資料，作為代幣參考價格之額外校準來源，降低單一資料來源被操縱或失準之風險，同時分散 5.5.2 節所述對單一第三方平台的依賴。

6.3 對於計劃的進一步研究或改進，以及預期的挑戰和機會

本專題未來仍面臨數項挑戰與機會。以下將 5.5 節所列之問題與挑戰，依可行性與相依條件分配至短期、中期與長期三個階段，並說明各階段的預期挑戰與機會。

6.3.1 短期（3 至 6 個月）

短期著重於團隊可自行控制、不需外部機構配合的技術缺口，包含四項工作。

（一）金鑰管理強化：將 ADMIN_KEY 與 IMAGE_ENCRYPTION_KEY 依用途分離，並由環境變數改為第三方金鑰管理服務（KMS），同時將 KYC 解密所用之 adminKey 改為獨立簽發之管理金鑰，不再沿用資料庫連線密碼，對應 5.5 節與 2.3 節所列之金鑰限制。

（二）效能調校：導入 Redis 快取層攔截 K 線等高頻重複查詢、設定 TypeORM 連線池上限，並評估將 OHLC 聚合邏輯下放至資料庫端，對應 5.2.4 節所測得之延遲瓶頸。

（三）爬蟲資料品質閘門：於爬蟲層加入數值合理性檢核（如坪數與價格須為正數、日期格式須可解析）與「未通過驗證即不寫入」之攔截機制，並接上排程器使爬蟲不再依賴人工執行，對應 5.5.1 節第二項問題。

（四）職務分離機制：對白名單調整、鏈上暫停與資產強制轉移等高風險操作，實作「提出→核准→執行」之雙人核准流程，對應 2.3.7 節所列之限制。

此階段的預期挑戰在於效能調校須同時處理應用層與部署環境兩端，若僅調整程式而未改變資料庫地理位置，改善幅度可能有限。機會則在於上述四項均可在現有架構內完成，不需重寫核心邏輯。

6.3.2 中期（6 個月至 1 年）

中期著重於架構層面的改動與外部服務串接，包含四項工作。

（一）交易一致性架構改造：將鏈上呼叫前置一筆「已送出」意圖紀錄並改為兩階段提交，從根本解決 5.5.1 節第五項所述「服務重啟空窗期重複轉帳」之風險，使系統不再僅依賴事後對帳。

（二）導入鏈上事件訂閱與 reorg 處理：現行對帳為全量比對式讀取，未使用事件監聽亦未實作區塊重組偵測。中期可改為事件驅動並加入確認數（confirmations）判斷，降低對帳延遲並正確處理鏈重組情境。

（三）第三方 KYC 與 AML 服務串接：導入外部身分查核服務與國際制裁名單比對，解決 5.5.2 節第四項所述人工審核量能與標準一致性問題。

（四）大規模併發壓力測試與效能基準：於效能調校完成後，執行長時間、多使用者之壓力測試，建立可比較的效能基準，作為產品化前的驗收依據。

此階段的預期挑戰在於兩階段提交會改變既有交易流程，須重新設計測試案例以確保原有 89 項測試仍能守住行為。機會則在於第三方 KYC 服務已有成熟商用方案，導入後可同時提升審核品質與速度。

6.3.3 長期（1 年以上）

長期工作多需外部機構配合或法遵資格，非團隊可單獨完成，包含四項。

（一）正式部署至公鏈或測試鏈：需重新評估 Gas 成本對交易體驗與 AMM 定價頻率的影響。在測試網環境中每筆轉帳的成本可忽略，但在公鏈上，合規檢查所增加的 Gas 消耗將直接反映為投資人的交易成本，可能需要改採 Layer 2 或批次結算設計。

（二）法幣交割與信託金流串接：需取得銀行測試環境與相應法遵資格，才能將現行以 bank_trust_accounts 與 bank_trust_transactions 模擬之金流，替換為真實撥款 API，對應 5.5.2 節第二項挑戰。

（三）產權法定登記串接與權利定性：需與內政部地政系統建立介接管道，並釐清代幣所表彰之權利在現行民法與信託法下的定性，才能使代幣自「平台內部收益權益紀錄」提升為具法律效力之權利憑證。

（四）多源鑑價與專業估價整合：房地產估值涉及高度在地化的市場資訊，如何持續擴充可信

賴的外部資料來源（如都市計畫、大眾運輸建設資訊），並確保其資料品質，將是動態鑑價模型能否長期維持準確性的關鍵。此階段亦須評估引入不動產估價師之專業意見作為模型校準基準，以及估價責任歸屬如何在演算法與專業人員之間劃分等制度性問題。

此階段的預期挑戰在於法遵與制度面的不確定性高，時程不完全由團隊掌握。機會則在於金管會已就證券型代幣發行提出相關規範說明，制度環境正逐步明朗，若能在規範成形階段即具備可運作的技術原型，將較晚進者更具先行優勢。

7. 參考文獻

7.1 中文文獻

金融監督管理委員會（2025）。監理門診 FAQ：證券型代幣發行（STO）相關規範之說明。
<https://www.fsc.gov.tw/>

富邦建築經理股份有限公司（無日期）。富邦一號不動產投資信託基金。2026 年 9 月 10 日，
取自 <https://www.fubonrem.com.tw/archives/category/650457883904f>

591 房屋交易網（無日期）。新建案。2026 年 9 月 10 日，取自 <https://www.591.com.tw/>

7.2 英文文獻

Adams, H., Zinsmeister, N., & Robinson, D. (2020). Uniswap v2 core. Uniswap.
<https://uniswap.org/whitepaper.pdf>

Aspen Digital. (n.d.). Aspen Coin. Retrieved September 10, 2026, from <https://www.aspencoin.io/>

Docker. (n.d.). Docker Compose documentation. Retrieved September 10, 2026, from
<https://docs.docker.com/compose/>

ERC-3643 Association. (n.d.). ERC-3643: The T-REX protocol. Retrieved September 10, 2026, from
<https://www.erc3643.org/>

Ethereum Improvement Proposals. (2021). EIP-3643: T-REX — Token for regulated exchanges.
<https://eips.ethereum.org/EIPS/eip-3643>

Ethers. (n.d.). Ethers.js documentation. Retrieved September 10, 2026, from <https://docs.ethers.org/>

Forbes. (2026, March 17). How real estate tokenization opens new paths for ownership and income.
<https://www.forbes.com/sites/digital-assets/2026/03/17/how-real-estate-tokenization-opens-new-paths-for-ownership-and-income/>

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc7519/>

Meta Open Source. (n.d.). React documentation. Retrieved September 10, 2026, from <https://react.dev/>

Microsoft. (n.d.). Playwright documentation. Retrieved September 10, 2026, from
<https://playwright.dev/>

NestJS. (n.d.). NestJS documentation. Retrieved September 10, 2026, from <https://docs.nestjs.com/>

Nomic Foundation. (n.d.). Hardhat documentation. Retrieved September 10, 2026, from
<https://hardhat.org/docs>

OnchainID. (n.d.). OnchainID documentation. Retrieved September 10, 2026, from
<https://docs.onchainid.com/>

PostgreSQL Global Development Group. (2022). PostgreSQL 15 documentation.
<https://www.postgresql.org/docs/15/>

Tailwind Labs. (n.d.). Tailwind CSS documentation. Retrieved September 10, 2026, from <https://tailwindcss.com/docs>

Tokenising real-world assets: Transforming real estate and beyond. (2024). ResearchGate. <https://www.researchgate.net/publication/383118168>

Tokenization of real estate assets using blockchain. (2022). Journal of Global Information Management. <https://www.sciencedirect.com/org/science/article/pii/S1548365722000188>

TypeORM. (n.d.). TypeORM documentation. Retrieved September 10, 2026, from <https://typeorm.io/>

Vite. (n.d.). Vite documentation. Retrieved September 10, 2026, from <https://vite.dev/>

Wang, Y., et al. (2025). Exploration on real world assets (RWAs) and tokenization (arXiv:2503.01111). arXiv. <https://arxiv.org/abs/2503.01111>

附錄

附錄 A 自動化測試執行紀錄

以下為本專題使用 Jest（後端單元、整合與權限測試）與 Hardhat（智能合約測試）之完整終端機輸出，共涵蓋 89 項測試，對應 5.2 節「實驗數據」中所引用之測試證據。

```

=====
【一】後端 單元／負向／併發／故障注入
=====

後端單元／負向／併發／故障注入測試 (jest, rootDir=src)
指令 : npx jest --verbose --reporters=default --ci
路徑 : RWA_Project-main/rwa-backend
基準 commit : 909db44 test: update fault-injection assertion for reconcile offline node message
| | (另含尚未提交的 src/blockchain/fault-injection.spec.ts 新增 4 筆)
時間 : 2026-08-18 01:56:06
Node : v24.18.0 npx jest : 30.4.1
OS : Windows 10 Pro
=====

PASS src/properties/properties.service.spec.ts
  PropertiesService.getKLineData - 單元測試：台北時區歸類
    ✓ UTC 16:30 (台北時間隔天 00:30) 應該被歸類到「隔天」的 K 棒，而不是當天 (8 ms)
    ✓ UTC 23:59 (台北時間 07:59，同一個台北日) 不應該被推到隔天 (1 ms)
    ✓ 同一個台北日的多筆交易應合併成一根 K 棒，並正確計算 open/high/low/close/volume (1 ms)
    ✓ 沒有任何交易紀錄時應回傳空陣列，而不是報錯 (1 ms)
  PropertiesService.getKLineData - 合約測試：回傳格式
    ✓ 每個元素都必須包含 time/open/high/low/close/volume 六個欄位，型別正確 (1 ms)
  PropertiesService.getKLineData - 查詢範圍
    ✓ appTxRepo.find 呼叫時應帶入 take: 3000，避免無上限查詢全部歷史
    ✓ 時間下限應與造市資料的 31 天歷史對齊，使視窗內每一天都有資料、真實交易不會落單 (3 ms)

[Nest] 6212 - 2026/08/18 上午1:56:12 WARN [TransactionsService] ⚠ 鏈上轉帳失敗 (DB 仍會記錄，狀態標記為 CHAIN_FAILED) : SIMULATED_CHAIN_TIMEOUT
PASS src/transactions/transactions.service.spec.ts
  故障注入測試 - DB 連線在 commit 階段中斷
    ✓ commitTransaction 失敗時，應該呼叫 rollbackTransaction 並回傳失敗，而不是讓例外往外爆 (35 ms)
  故障注入測試 - 鏈上呼叫逾時/失敗
    ✓ executeOnChainBuy 拋錯時，DB 仍應完成交易並標記為 CHAIN_FAILED，而不是整筆請求失敗 (3 ms)

PASS src/transactions/fault-injection.spec.ts
  console.log
    [併發測試 log] 2 個併發請求 → 成功 2/2 筆，資料庫實際寫入相同 idempotency_key 共 2 次
      at Object.<anonymous> (transactions/transactions.service.spec.ts:366:15)

[Nest] 6292 - 2026/08/18 上午1:56:12 ERROR [TransactionsService] 唯一約束違反 (非冪等性) : constraint=UQ_user_holdings_user_property_detail=無 table=user_holdings
PASS src/transactions/transactions.service.spec.ts
  TransactionsService - 負向測試
    ✓ 系統暫停時下單應被拒絕 (403) (28 ms)
    ✓ 未通過 KYC (is_whitelisted=false) 的使用者下單應被拒絕 (1 ms)
    ✓ 數量為 0 應回傳 400 錯誤
    ✓ 數量為負數 應回傳 400 錯誤 (1 ms)
    ✓ 價格為 0 應回傳 400 錯誤 (1 ms)
    ✓ 價格為負數 應回傳 400 錯誤
    ✓ 賣出數量超過實際持倉時應被拒絕 (3 ms)
    ✓ AMM 池子被抽乾時 (流通量 = 總發行量) 應拒絕買入 (1 ms)
    ✓ 限價買單若 AMM 滑價超過使用者限價應被拒絕 (1 ms)
  TransactionsService - 單元測試：AMM 定價公式 (k = x*y)
    ✓ 買入 100 枚時，服務算出的成交均價應該精確符合 k=x*y 公式 (而不是約等於現貨價) (17 ms)
    ✓ 買入量越大，均價應該越高 (滑價方向正確) (3 ms)
  TransactionsService - 併發測試
    ✓ ● 已知風險：相同 idempotency_key 併發送出兩筆市價單，目前的檢查 (先 findOne 再寫入) 存在 TOCTOU 競態條件，可能兩筆都成交 (62 ms)
    ✓ 不同 idempotency_key 的併發請求應該都能各自正常成交 (2 ms)
  TransactionsService - 併發測試：AMM 定價的鎖定策略
    ✓ 讀取建案資料時應帶入 pessimistic_write 排他鎖，使同一建案的交易所序列化 (1 ms)
    ✓ 資料庫以 UNIQUE 約束擋下重複的 idempotency_key 時，應轉為明確的重複交易訊息而非 500 (1 ms)
    ✓ 其他唯一約束的衝突不應被誤報成重複交易，訊息要指出實際違反的約束 (2 ms)
    ✓ 撮合引擎應以含滑價的成交均價判斷觸發，不可只看現貨價 (1 ms)
    ✓ 限價單結算時若沒有鏈上報價，tx_hash 應維持未設定而非空字串
    ✓ 鎖應在資料庫交易開啟之後才取得，否則鎖不會生效 (1 ms)

PASS src/auth/fault-injection.spec.ts
  故障注入測試 - 必要環境變數缺失時應 fail-fast

```

```

    ✓ 其他唯一約束的開天價應被映報成里後又勿不訊息要指出實際違反的約束 (2 ms)
    ✓ 撮合引擎應以含滑價的成交均價判斷觸發，不可只看現貨價 (1 ms)
    ✓ 限價單結算時若沒有鏈上雜湊，tx_hash 應維持未設定而非空字串
    ✓ 鎖應在資料庫交易開啟之後才取得，否則鎖不會生效 (1 ms)

PASS src/auth/fault-injection.spec.ts
  故障注入測試 – 必要環境變數缺失時應 fail-fast
    ✓ 缺少 SUPABASE_SERVICE_KEY 時，AuthService 應該在建構時直接拋錯，而不是等到呼叫 API 才炸 (1297 ms)
    ✓ 缺少 IMAGE_ENCRYPTION_KEY 時，整個 auth module 在載入當下就應該拋錯 (271 ms)

PASS src/blockchain/fault-injection.spec.ts
  BlockchainService – 故障注入：區塊鏈節點無法連線
    ✓ RPC 位址連不上時，初始化不應拋出例外讓整個服務起不來 (71 ms)
    ✓ isNodeReachable() 面對連不上的節點應回傳 false，而不是拋例外 (29 ms)
    ✓ getStatus() 應照常回應並標示節點不可達，不因節點故障而讓 API 掛掉 (24 ms)
    ✓ setupBlockchain() 應以明確訊息拒絕，而不是靜默失敗或卡住 (39 ms)
    ✓ 節點故障應寫入 system_alerts 供技術端稽核台顯示 (24 ms)
    ✓ reconcile() 在節點不可達時應拒絕執行，避免以殘缺的鏈上資料誤判持倉 (30 ms)
  BlockchainService – 故障注入：節點失憶的自動重建判斷
    ✓ 節點不可達時排程應直接跳過，不得觸發重建 (15 ms)
    ✓ 🛡️ getCode 因網路異常拋錯時不得重建 — RPC 瞬斷不等於合約消失 (2 ms)
    ✓ getCode 確實回傳 0x (合約真的不存在) 時才應觸發重建 (1 ms)
    ✓ 前一次重建尚未結束時，下一次排程應被 isRecovering 擋下，且結束後要能再次執行 (2 ms)
  BlockchainService – 負向：正式環境缺少 ADMIN_KEY
    ✓ 正式環境未設定 ADMIN_KEY 時，應拒絕以公開的 Hardhat 測試私鑰啟動鏈上功能 (1 ms)
    ✓ 開發環境未設定 ADMIN_KEY 時仍可使用 Hardhat 預設帳戶，不影響本機開發 (1 ms)

Test Suites: 5 passed, 5 total
Tests:       42 passed, 42 total
Snapshots:   0 total
Time:        3.98 s
Ran all test suites

```

```
=====
路徑 : RWA_Project-main/rwa-backend
基準 commit : 909db44 test: update fault-injection assertion for reconcile offline node message
時間 : 2026-08-18 01:56:14
Node : v24.18.0 npx jest : 30.4.1
OS : Windows 10 Pro
=====
```

PASS test/integration-auth.e2e-spec.ts

整合測試 + 權限測試 (e2e, mocked DB)

權限測試 - JWT 驗證

- ✓ 未帶 Authorization header 打受保護的 API 應回 401 (29 ms)
- ✓ 帶壞掉/篡改過的 token 應回 401 (7 ms)
- ✓ 帶合法 token 應該通過驗證 (不是 401) (9 ms)

權限測試 - 資源擁有者檢查

- ✓ user_id 跟 token 裡的 id 不一致時應回 403 (不能幫別人下單) (11 ms)
- ✓ user_id 跟 token 一致時應正常進入下單流程 (不是 403) (7 ms)

權限測試 - BUSINESS 角色專屬 API

- ✓ 一般投資人 (INVESTOR) 呼叫 get /api/users 應回 403 (4 ms)
- ✓ 一般投資人 (INVESTOR) 呼叫 patch /api/users/2/whitelist 應回 403 (4 ms)
- ✓ 一般投資人 (INVESTOR) 呼叫 patch /api/users/2/kyc 應回 403 (5 ms)
- ✓ 一般投資人 (INVESTOR) 呼叫 post /api/kyc/2/decrypt 應回 403 (3 ms)
- ✓ 管理員 (BUSINESS) 呼叫 get /api/users 不應被擋在權限這關 (不是 403) (2 ms)
- ✓ 管理員 (BUSINESS) 呼叫 patch /api/users/2/whitelist 不應被擋在權限這關 (不是 403) (3 ms)
- ✓ 管理員 (BUSINESS) 呼叫 patch /api/users/2/kyc 不應被擋在權限這關 (不是 403) (2 ms)
- ✓ 管理員 (BUSINESS) 呼叫 post /api/kyc/2/decrypt 不應被擋在權限這關 (不是 403) (1 ms)

整合測試 - 下單完整路徑

- ✓ 合法下單應該回傳 success:true, 並帶有交易結果 (3 ms)
- ✓ 系統暫停時, 就算通過所有權限檢查, 下單也應該在 Service 層被擋下 (7 ms)

Test Suites: 1 passed, 1 total

Tests: 15 passed, 15 total

Snapshots: 0 total

Time: 1.642 s, estimated 2 s

```
=====
智能合約測試 (Hardhat 內建網路, chainId 31337)
指令 : npm test      (= hardhat test)
路徑 : RWA_Project-main/blockchain
基準 commit : 909db44 test: update fault-injection assertion for reconcile offline node message
時間 : 2026-08-18 01:56:33
Node : v24.18.0      Hardhat : 2.28.6
OS   : Windows 10 Pro
=====
```

```
> test
> hardhat test
```

SimpleRWA – 合約測試：部署與初始狀態

- ✓ 代幣的名稱、代號、精度、總發行量應與部署參數一致 (725ms)
- ✓ 鑄造出的全部代幣應在 admin 錢包中，作為未售出庫存
- ✓ 代幣應綁定正確的 IdentityRegistry 與 Compliance

SimpleRWA – 合約測試：KYC 合規閘門 (ERC-3643 核心)

- ✓ 已登記身分且持有 KYC Claim 的地址，應被 IdentityRegistry 判定為已驗證
- ✓ 完全未登記的地址應被判定為未驗證
- ✓ 轉帳給已通過 KYC 的投資人應該成功，且雙方餘額正確變動
- ✓ 🔒 轉帳給未通過 KYC 的地址必須被鏈上拒絕 (就算後端權限被繞過也擋得住)
- ✓ 🔒 未通過 KYC 的地址也不能被鑄造代幣
- ✓ 身分登記被撤銷後，該地址應立即無法再收到代幣

SimpleRWA – 權限測試：agent 與 owner 角色

- ✓ admin 應具備 agent 身分，一般投資人不應具備
- ✓ 🔒 非 agent 呼叫 mint() 應被拒絕
- ✓ 🔒 非 agent 呼叫 pause() 應被拒絕

SimpleRWA – 權限測試：agent 與 owner 角色

- ✓ admin 應具備 agent 身分，一般投資人不應具備
- ✓ 🔒 非 agent 呼叫 mint() 應被拒絕
- ✓ 🔒 非 agent 呼叫 pause() 應被拒絕
- ✓ 🔒 非 agent 呼叫 forcedTransfer() 應被拒絕（賣出流程依賴此權限）
- ✓ agent 可用 forcedTransfer 收回投資人代幣，使用者錢包無需持有 ETH 付 gas
- ✓ 🔒 非 owner 不能新增 agent

SimpleRWA – 負向測試：暫停與餘額邊界

- ✓ pause() 之後所有轉帳都應被拒絕（技術端緊急煞車）
- ✓ unpause() 之後轉帳應恢復正常
- ✓ 重複 pause() 應被拒絕，避免狀態被誤改
- ✓ 轉帳金額超過餘額時應被拒絕
- ✓ 投資人賣超過自己持有的數量時應被拒絕

20 passing (861ms)

附錄 B 效能測試原始輸出

以下為資料庫查詢效能測試（5 萬筆資料規模）與區塊鏈 RPC 效能測試之原始終端機輸出，對應 5.2.4 節所引用之數據。

```

PS C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main> cd C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main\RWA_Project-main\rwa-backend
PS C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main\RWA_Project-main\rwa-backend> node test/perf/run-explain.js
◇ injected env (7) from .env // tip: Ⓜ multiple files { path: ['.env.local', '.env'] }
連線成功，正在執行 EXPLAIN ANALYZE...

===== 執行計畫結果 =====
✓ Limit (cost=0.28..2.42 rows=1 width=588) (actual time=0.089..0.089 rows=0 loops=1)
  -> Index Scan using idx_transactions_property_time on transactions (cost=0.28..2.42 rows=1 width=588) (actual time=0.088..0.088 rows=0 loops=1)
        Index Cond: ((property_id = 1) AND (created_at >= (now() - '31 days'::interval)))
        Filter: ((status)::text = 'SUCCESS'::text)
Planning Time: 1.262 ms
Execution Time: 0.126 ms

=====
===== 區塊鏈 RPC 響應測試 [本機 Hardhat 節點] =====
Connecting to: http://127.0.0.1:8545
[Req 1/10] 延遲: 67.95 ms (當前區塊: 0)
[Req 2/10] 延遲: 4.63 ms (當前區塊: 0)
[Req 3/10] 延遲: 2.88 ms (當前區塊: 0)
[Req 4/10] 延遲: 3.01 ms (當前區塊: 0)
[Req 5/10] 延遲: 2.72 ms (當前區塊: 0)
[Req 6/10] 延遲: 2.28 ms (當前區塊: 0)
[Req 7/10] 延遲: 1.85 ms (當前區塊: 0)
[Req 8/10] 延遲: 1.94 ms (當前區塊: 0)
[Req 9/10] 延遲: 3.08 ms (當前區塊: 0)
[Req 10/10] 延遲: 4.10 ms (當前區塊: 0)

本機 Hardhat 節點 測試完成！平均響應時間: 9.45 ms
=====

```

```

===== 區塊鏈 RPC 響應測試 [正式雲端節點 (Render)] =====
Connecting to: https://rwa-blockchain-node.onrender.com
[Req 1/10] 延遲: 1439.33 ms (當前區塊: 0)
[Req 2/10] 延遲: 106.34 ms (當前區塊: 0)
[Req 3/10] 延遲: 105.64 ms (當前區塊: 0)
[Req 4/10] 延遲: 105.19 ms (當前區塊: 0)
[Req 5/10] 延遲: 107.27 ms (當前區塊: 0)
[Req 6/10] 延遲: 109.94 ms (當前區塊: 0)
[Req 7/10] 延遲: 108.95 ms (當前區塊: 0)
[Req 8/10] 延遲: 93.73 ms (當前區塊: 0)
[Req 9/10] 延遲: 107.54 ms (當前區塊: 0)
[Req 10/10] 延遲: 109.48 ms (當前區塊: 0)

正式雲端節點 (Render) 測試完成！平均響應時間: 239.34 ms
=====

PS C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main> cd C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main\RWA_Project-main\rwa-backend
PS C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main\RWA_Project-main\rwa-backend> node test/perf/autocannon-rpc.js
Running 20s test @ https://rwa-blockchain-node.onrender.com with 20 connections...
Running 20s test @ https://rwa-blockchain-node.onrender.com
20 connections

```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	74 ms	84 ms	135 ms	204 ms	92.4 ms	26.83 ms	466 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	149	149	221	228	215.35	18.31	149
Bytes/Sec	78 kB	78 kB	116 kB	119 kB	113 kB	9.58 kB	77.9 kB

```

PS C:\Users\reyliou\Desktop\412630567-RWA-DEMO-main\RWA_Project-main\rwa-backend> node test/perf/scale-explain.js
◇ injected env (7) from .env // tip: Ⓜ multiple files { path: ['.env.local', '.env'] }
連線成功，正在開啟資料庫交易 (Transaction)...
正在注入 50000 筆虛擬交易資料... (請稍候)
注入完成！

===== 大規模資料 EXPLAIN ANALYZE 結果 =====
✓ Limit (cost=0.29..2.43 rows=1 width=588) (actual time=0.034..3.398 rows=3000 loops=1)
✓ -> Index Scan using idx_transactions_property_time on transactions (cost=0.29..2.43 rows=1 width=588) (actual time=0.033..3.051 rows=3000 loops=1)
        Index Cond: ((property_id = 9999) AND (created_at >= (now() - '31 days'::interval)))
        Filter: ((status)::text = 'SUCCESS'::text)
Planning Time: 0.155 ms
Execution Time: 3.677 ms
=====

```

附錄 C 後端 API 負載測試原始輸出

以下為以 autocannon 對 K 線查詢與掛單簿查詢兩支端點所做之負載測試原始輸出，區分保留限流與解除限流兩種情境，對應 5.2.4 節之表 9 與表 10。

三-1 K 線查詢 · 保留限流 (`limit = 100`)

~~~

Running 20s test @ <http://localhost:3001/api/properties/1/kline>  
20 connections

| Stat    | 2.5%    | 50%     | 97.5%    | 99%      | Avg        | Stdev      | Max      |
|---------|---------|---------|----------|----------|------------|------------|----------|
| Latency | 1543 ms | 2258 ms | 12096 ms | 12617 ms | 4071.69 ms | 3487.66 ms | 13105 ms |

| Stat      | 1%  | 2.5% | 50%     | 97.5%   | Avg      | Stdev    | Min     |
|-----------|-----|------|---------|---------|----------|----------|---------|
| Req/Sec   | 0   | 0    | 35      | 7,259   | 3,155.15 | 3,434.86 | 4       |
| Bytes/Sec | 0 B | 0 B  | 12.7 kB | 2.82 MB | 1.23 MB  | 1.34 MB  | 1.45 kB |

Req/Bytes counts sampled once per second.  
200 2xx responses, 62900 non 2xx responses  
79k requests in 20.05s, 24.5 MB read  
16k errors (0 timeouts)

~~~

三-2 掛單簿查詢 · 保留限流 (`limit = 100`)

~~~

Running 20s test @ <http://localhost:3001/api/orderbook/1>  
20 connections

| Stat    | 2.5% | 50%  | 97.5% | 99%  | Avg     | Stdev    | Max     |
|---------|------|------|-------|------|---------|----------|---------|
| Latency | 1 ms | 2 ms | 3 ms  | 4 ms | 3.95 ms | 56.04 ms | 3221 ms |

| Stat      | 1%  | 2.5% | 50%     | 97.5%   | Avg     | Stdev    | Min     |
|-----------|-----|------|---------|---------|---------|----------|---------|
| Req/Sec   | 0   | 0    | 6,631   | 7,103   | 4,400.7 | 3,235.47 | 10      |
| Bytes/Sec | 0 B | 0 B  | 2.58 MB | 2.76 MB | 1.71 MB | 1.26 MB  | 3.84 kB |

Req/Bytes counts sampled once per second.  
100 2xx responses, 87901 non 2xx responses  
88k requests in 20.08s, 34.2 MB read

~~~

三-3 K 線查詢 · 解除限流 (`limit = 100000`)

...

Running 20s test @ <http://localhost:3001/api/properties/1/kline>
20 connections

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	550 ms	577 ms	2001 ms	2370 ms	632.88 ms	291.29 ms	2581 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	3	3	34	40	31.25	9.79	3
Bytes/Sec	1.11 kB	1.11 kB	12.6 kB	14.8 kB	11.5 kB	3.61 kB	1.11 kB

Req/Bytes counts sampled once per second.
645 requests in 20.14s, 231 kB read

...

三-4 掛單簿查詢 · 解除限流 (`limit = 100000`)

...

Running 20s test @ <http://localhost:3001/api/orderbook/1>
20 connections

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	1127 ms	1157 ms	2993 ms	2998 ms	1277.33 ms	437.38 ms	3211 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	0	0	17	20	15.15	6.12	8
Bytes/Sec	0 B	0 B	6.63 kB	7.8 kB	5.91 kB	2.39 kB	3.12 kB

Req/Bytes counts sampled once per second.
323 requests in 20.16s, 118 kB read

...

三-5 純 TCP 連線測試 (對應 7.3 步驟 3)

隔離「應用層運算」與「底層網路連線」兩種可能性，直接對資料庫主機建立 TCP 連線並計時。

```

...
第 1 次 TCP Connect Time: 2128.444ms
第 2 次 TCP Connect Time: 706.373ms
第 3 次 TCP Connect Time: 548.819ms
第 4 次 TCP Connect Time: 534.578ms
第 5 次 TCP Connect Time: 556.849ms
...

```

第 1 次包含 DNS 解析與 TLS 初始化，屬冷啟動；第 3~5 次穩定在 ****530~560 ms****，即為跨國網路連線的基線成本。此數值與三-3 的 K 線平均延遲 632.88 ms 相比，佔比約 87%。

附錄 D 資料庫 Schema 設計

本附錄接續 3.6 節之五張 ER 子圖，依序列出 16 張核心資料表之欄位、資料型態、鍵位與約束。各表之排列順序與 3.6 節各子圖所標示之實體一一對應，可逐一對照查閱。

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
roles	id	integer	PK	NOT NULL	自動遞增	角色 ID
	role_name	varchar		UNIQUE	NULL	角色名稱

表 13：資料表欄位說明－roles

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
users	id	integer	PK	NOT NULL	自動遞增	使用者 ID
	username	varchar		UNIQUE	NULL	帳號
	email	varchar		UNIQUE	NULL	電子郵件
	phone_number	varchar			NULL	電話
	is_email_verified	boolean			false	信箱驗證狀態
	password_hash	text			NULL	密碼雜湊值
	role_id	integer	FK→roles.id		1	角色外鍵
	is_whitelisted	boolean			false	是否已通過鏈上白名單
	kyc_status	varchar			PENDING	KYC 狀態
	kyc_document_path	text			NULL	KYC 文件正面路徑
	kyc_document_back_path	text			NULL	KYC 文件反面路徑
	kyc_reviewed_by	integer			NULL	審核人員 ID
	kyc_reviewed_at	timestamp			NULL	審核時間

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	onchain_id_address	varchar			NULL	鏈上身份合約地址
	total_asset_value	numeric			100000	可用現金餘額（實作上承載現金，非資產總值。資產總值於 API 層由現金加計持倉市值即時計算）
	total_profit_loss	numeric			0	總損益
	wallet_address	varchar			NULL	錢包地址
	wallet_private_key	varchar			NULL	錢包私鑰
	bank_account_number	varchar			NULL	銀行帳號
	created_at	timestamp		NOT NULL	建立時間	建立時間

表 14：資料表欄位說明－users

註：wallet_private_key 已採 AES-256-CBC 加密儲存（格式為「IV:密文」之十六進位字串，並保留舊明文格式之向後相容處理）。kyc_document_path 與 bank_account_number 目前仍為明文欄位，屬原型階段已知限制，正式產品化應一併加密或改為遮罩顯示。另 total_asset_value 於實作上承載「可用現金餘額」，新註冊帳號預設為 100000（展示用體驗金），並非資產總值。資產總值係由現金餘額加計持倉市值於 API 層即時計算。

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
properties	id	integer	PK	NOT NULL	自動遞增	建案 ID
	title	varchar			NULL	建案名稱
	location	varchar			NULL	地區
	complete_address	text			NULL	完整地址
	main_image	varchar			NULL	主圖路徑
	token_symbol	varchar			‘RWA’	代幣代號
	total_supply_x	numeric			100000	代幣總發行量（AMM 公式 X 儲備）
	current_price	numeric			0	目前報價

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	fundraising_goal	numeric			0	募資初始總額 (AMM 公式 Y 儲備)
	status	varchar			'交易中'	建案狀態
	expected_apy	numeric			4.5	預期年化報酬率
	payout_cycle_days	integer			30	租金分潤週期天數
	size_ping	numeric			35.0	建案坪數
	yesterday_close_price	numeric			NULL	前一交易日收盤價 (供漲跌幅計算)
	token_address	varchar			NULL	鏈上代幣合約地址
	landlord_id	integer			NULL	業主 ID

表 15：資料表欄位說明—properties

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
user_holdings	id	integer	PK	NOT NULL	自動遞增	持倉紀錄 ID
	user_id	integer	FK→users.id	NOT NULL, UNIQUE(組合)		使用者 ID
	property_id	integer	FK→properties.id	NOT NULL, UNIQUE(組合)		建案 ID
	balance	numeric			0	持有代幣數量
	holder_type	varchar		UNIQUE(組合)	'INVESTOR'	持有人類型

表 16：資料表欄位說明—user_holdings

註:UNIQUE(組合)：(user_id, property_id, holder_type) 三欄組合唯一

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
transactions	id	integer	PK	NOT NULL		交易 ID
	user_id	integer	FK→users.id			使用者 ID
	property_id	integer	FK→properties.id			建案 ID

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	tx_type	varchar		NOT NULL		交易類型（買/賣）
	order_type	varchar			NULL	市價/限價
	token_amount	numeric		NOT NULL		代幣數量
	price_per_token	numeric		NOT NULL		每枚成交價
	status	varchar			'SUCCESS'	交易狀態（含CHAIN_FAILED）
	tx_hash	varchar			NULL	鏈上交易雜湊
	idempotency_key	uuid		UNIQUE	NULL	冪等鍵（見 5.2 節併發 TOCTOU 限制）
	created_at	timestamp		NOT NULL		建立時間
	is_simulated	boolean			false	是否為模擬交易

表 17：資料表欄位說明－transactions

註：user_id、property_id 於資料庫層已建立正式外鍵約束（transactions_user_id_fkey、transactions_property_id_fkey），分別參照 users.id 與 properties.id。

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
user_notifications	id	integer	PK	NOT NULL	自動遞增	通知 ID
	user_id	integer	FK→users.id	NOT NULL		使用者 ID
	title	text			NULL	標題
	message	text			NULL	內容
	is_read	boolean			false	已讀狀態
	created_at	timestamp		NOT NULL		建立時間

表 18：資料表欄位說明－user_notifications

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
system_alerts	id	integer	PK	NOT NULL	自動遞增	告警 ID
	alert_type	varchar			NULL	類型
	severity	varchar			NULL	嚴重程度

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	message	text			NULL	內容
	created_at	timestamp		NOT NULL		建立時間

表 19：資料表欄位說明－system_alerts

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
crawler_metrics	id	integer	PK	NOT NULL	人工指定	指標 ID（非自動遞增）
	last_run_at	timestamp			NULL	最後執行時間
	consecutive_failures	integer			0	連續失敗次數
	average_integrity	numeric			100	平均資料完整度
	status	varchar			'HEALTHY'	爬蟲健康狀態

表 20：資料表欄位說明－crawler_metrics

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
property_ownership	id	integer	PK	NOT NULL	自動遞增	產權紀錄 ID
	property_id	integer	FK→properties.id	NOT NULL		建案 ID
	holder_name	varchar		NOT NULL		持有人姓名
	ownership_percentage	numeric		NOT NULL		持有比例
	legal_document_ref	varchar			NULL	法律文件參照
	entity_type	varchar			'INDIVIDUAL'	持有人類型
	created_at	timestamp		NOT NULL		建立時間

表 21：資料表欄位說明－property_ownership

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
valuation_logs	id	integer	PK	NOT NULL	自動遞增	估值紀錄 ID
	property_id	integer	FK→properties.id	NOT NULL		建案 ID
	value	numeric			NULL	估值

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	recorded_at	timestamp		NOT NULL		記錄時間

表 22：資料表欄位說明－valuation_logs

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
bank_trust_accounts	id	integer	PK	NOT NULL	自動遞增	信託帳戶 ID
	property_id	integer	FK→properties.id	NOT NULL		建案 ID
	current_cash_balance	numeric			0	目前現金餘額
	pending_rent_amount	numeric			0	待入帳租金

表 23：資料表欄位說明－bank_trust_accounts

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
bank_trust_transactions	id	integer	PK	NOT NULL	自動遞增	流水 ID
	trust_account_id	integer	FK→bank_trust_accounts.id	NOT NULL		信託帳戶外鍵
	tx_type	varchar		NOT NULL		類型
	amount	numeric		NOT NULL		金額
	reference_note	text			NULL	備註
	recorded_at	timestamp		NOT NULL		記錄時間

表 24：資料表欄位說明－bank_trust_transactions

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
rent_payout_batches	id	integer	PK	NOT NULL	自動遞增	批次 ID
	property_id	integer	FK→properties.id	NOT NULL		建案外鍵
	payout_period	date		NOT NULL		分潤期間
	total_rent_collected	numeric		NOT NULL		該期收取總租金
	status	varchar			'PENDING'	狀態

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
	created_at	timestamp		NOT NULL		建立時間

表 25：資料表欄位說明－rent_payout_batches

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
rent_payout_details	id	integer	PK	NOT NULL	自動遞增	明細 ID
	batch_id	integer	FK→rent_payout_batches.id	NOT NULL		批次外鍵
	user_id	integer	FK→users.id	NOT NULL		使用者外鍵
	holding_percentage	numeric		NOT NULL		持有比例
	payout_amount	numeric		NOT NULL		分潤金額
	status	varchar			'CALCULATED'	狀態
	tx_hash	varchar			NULL	撥款交易雜湊
	created_at	timestamp		NOT NULL		建立時間

表 26：資料表欄位說明－rent_payout_details

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
chat_logs	id	integer	PK	NOT NULL	自動遞增	聊天訊息 ID
	sender_id	integer	FK→users.id	NOT NULL		發送者
	channel	varchar		NOT NULL		頻道
	message	text		NOT NULL		訊息內容
	created_at	timestamp		NOT NULL		建立時間

表 27：資料表欄位說明－chat_logs

表格名稱	欄位名稱	資料型態	鍵位	約束	預設值	說明
blockchain_config	key	varchar	PK	NOT NULL		設定鍵名
	value	varchar			NULL	設定值

表 28：資料表欄位說明－blockchain_config

附錄 D-1 物件關聯對映 (Object-Relational Mapping, ORM) 設計

本系統以 TypeORM 作為 ORM 層，將上述 16 張資料表分別定義為對應之實體類別 (Entity)，並透過屬性裝飾器 (如 @Column、@PrimaryGeneratedColumn、@ManyToOne) 宣告欄位型態與外鍵關聯，使程式端得以物件屬性存取取代原生 SQL 語句拼接。各模組 (如 TransactionsModule、PropertiesModule) 之 Service 層透過 Repository 介面對實體進行查詢與寫入，並利用 QueryRunner API 包裝多筆寫入操作，確保跨資料表更新之原子性 (例如接收鏈上轉帳確認後，同時寫入交易紀錄與更新持倉餘額兩項操作)。正式環境設定 synchronize: false，資料庫結構變更一律透過 Migration 腳本管理，避免 ORM 自動同步結構所帶來之非預期資料遺失風險。已知限制：目前實體類別與資料表為一對一對映，尚未導入視圖 (View) 或跨模組聚合查詢之進階 ORM 設計，複雜報表查詢仍以原生 SQL 輔助。

附錄 E 功能與循序圖對應表

為確保所有核心業務與高風險例外皆具備架構設計依據，下表建立功能需求與循序圖之對應關係，各功能均可追溯至對應之設計圖與實作驗證機制。

功能項目	對應 USE case 圖	異常／例外處理路徑	實作檔案與驗證機制
投資人市價買入	圖 2：投資人買入代幣流程	餘額不足回滾、超過 5% 持倉上限拒絕、鏈上超時標記 CHAIN FAILED	transactions.service.ts (悲觀鎖 + 冪等性)
限價單背景撮合	圖 3 之衍生掛單分支 (每 5 秒輪詢)	均價滑價過高跳過本輪、無法滿足條件時轉為 CANCELLED	transactions.service.ts
業務端 KYC 審核	圖 9：KYC 審核流程	鏈上部署或 Claim 簽發失敗時僅記錄警告，資料庫狀態維持 VERIFIED 不回捲，須由技術端手動補登 (已知限制)	users.service.ts + blockchain.service.ts
租金收益批次分潤	圖 10：租金分潤批次處理	鏈上撥款失敗標記 FAILED、信託專戶扣款。已知限制：無 CALCULATED 中間態、批次未包單一交易、未實作除不盡餘額結轉	properties.service.ts
鏈上鏈下一致性對帳	圖 3：對帳排程	節點離線回傳 503、發現未追蹤轉帳時標記需人工介入	blockchain.service.ts (reconcile)
用戶白名單管理	圖 18：UseCase 圖-業務端	未通過審核者無法交易，白名單狀態變更即時反映於鏈上 IdentityRegistry	users.service.ts

表 29：功能與循序圖對應表

註：本表涵蓋本系統核心業務與主要高風險例外情境，各功能均可追溯至對應設計圖與實作驗證機制。